COMP0411 중간고사 · 10월 21일(수) 10:30

컴퓨터구조 벼락치기

시험에 나올 것만 모은 압축판입니다. 처음 배우는 사람이 앞에서부터 읽기만 해도 모든 문제를 풀 수 있게 "왜 그런지" 설명을 넣었습니다. 그래서 첫 회독은 읽기만 약 5시간, 점검까지 풀면 이틀이 걸립니다. 두 번째부터는 "먼저 알아 둘 말"과 긴 설명 문단을 건너뛰고 외울 것의 굵은 줄, 함정, 점검만 보면 2시간 안팎입니다. 시험 전까지 **세 번 이상** 읽는 것이 목표입니다. 자세한 설명이 필요하면 이론 노트를 여세요: 이론 노트 열기

중간고사 공지	
시간	10월 21일(수) 오전 10:30 ~ 오후 12:00 (90분)
장소	IT5호관(융복합관) B101호
형식	Closed Book 필기시험, 단답형 위주(T/F, 빈칸, 계산, 코드), 한글 시험
범위	Ch1 ~ Ch3 전체 + Ch4 단일 사이클(p.1–29)

단원 하나를 읽는 법

1. **외울 것** 상자를 읽습니다. 단원의 중심입니다.
2. **푸는 순서**가 있으면 순서 번호까지 외웁니다. 계산 문제는 이 순서대로 풉니다.
3. **함정**을 읽습니다. 시험에서 자주 틀리는 곳입니다.
4. **예제**와 **회독 점검**은 답을 펼치기 전에 연습장이나 종이에 먼저 씁니다. 맞았으면 [맞음], 틀렸으면 [틀림]을 누릅니다. 틀린 문제는 **오답 노트**에 모입니다.

회독 계획 (하루 5시간)

한 번 읽을 때마다 단원 끝의 "이 단원 다 봤어요"를 체크하세요. 다음 회독을 시작할 때 체크를 다시 풀면 됩니다.

날짜	회독	할 것
10/17 (토)	1회독	필수 기초 1 ~ 4 → Ch1 → Ch2 · 1 ~ 3. 회독 점검을 모두 직접 풉니다. 막히면 바로 질문.
10/18 (일)	1회독	Ch2 · 4 ~ 5 → Ch3 → Ch4 → 시험 전날 한 장 → 오답 노트.
10/19 (월)	2회독	처음부터 끝까지, 설명 상자는 건너뛰고 외울 것의 굵은 줄·함정·점검만. 외울 것은 **가리고 먼저 떠올린 뒤** 확인. 끝나면 오답 노트, Ch4 제어 신호 표를 백지에 써 보기.
10/20 (화)	3회독 + 실전	오전: 3회독(외울 것·함정·틀렸던 점검만, 약 1시간 30분). 오후: 이론 노트의 모의고사 1회(90분) → 채점 → 오답 노트.
10/21 (수)	4회독	아침: **시험 전날 한 장** + 오답 노트만. 시험 10:30.

단원별 진도

상자 읽는 법

외울 것 그대로 외울 정의·공식·표

함정 시험에서 자주 틀리는 지점

[Ch2 p.14] 근거 슬라이드: 2장 PDF 14쪽

PDF 외 보충 강의 PDF에 없지만 기출에 나왔거나 이해를 돕는 내용

교재 기준 슬라이드에 그림으로만 있어 교재로 보충한 세부

기초 · 1 슬라이드 Ch2 p.17–18 (쓰이는 곳: Ch2 · 4, Ch3 · 2)

진법 변환과 16진수

시험에 나오는 모양: 10진 ↔ 2진 ↔ 16진 바꾸기, 32비트 값을 16진 8자리로 쓰기, 10진 소수를 2진으로(IEEE 754의 첫 단계). 이 단원이 막히면 인코딩·IEEE 754 문제를 하나도 못 푼다.

쉽게 말하면

2진수는 자릿값이 **오른쪽 끝 자리부터** 1, 2, 4, 8, … 로 두 배씩 커지는 수다(10진수가 1, 10, 100, … 로 열 배씩 커지는 것과 같은 원리). 2진수 한 자리를 **비트(bit)**라 부른다. 1011_2는 4비트다. 16진수는 2진 4자리를 한 글자로 줄여 쓴 것이라 2진과 16진 사이는 계산 없이 표만 보고 바꾼다.

외울 것

n	0	1	2	3	4	5	6	7	8
2^n	1	2	4	8	16	32	64	128	256

n	9	10	11	12	13	14	15	16	32
2^n	512	1024	2048	4096	8192	16384	32768	65536	4,294,967,296 (약 43억)

n	−1	−2	−3	−4	−5
2^n	0.5	0.25	0.125	0.0625	0.03125

16진	0	1	2	3	4	5	6	7
2진	0000	0001	0010	0011	0100	0101	0110	0111

16진	8	9	A(10)	B(11)	C(12)	D(13)	E(14)	F(15)
2진	1000	1001	1010	1011	1100	1101	1110	1111

- **2진 → 10진**: 1이 있는 자리의 자릿값을 더한다. 1011_2 = 8 + 2 + 1 = 11. [Ch2 p.17]
- **10진 → 2진(정수)**: 2로 나누기를 **몫이 0이 될 때까지** 반복한다(마지막은 1 ÷ 2 = 몫 0, 나머지 1). 나머지를 **아래에서 위로** 읽는다. 작은 수는 **자릿값 빼기(분해)**도 된다: 13 = 8 + 4 + 1 → 8·4·1 자리에 1 → 1101_2(Ch3에서 이 방법을 씀).

  왜 아래 → 위인가: 2로 나눈 나머지는 그 수가 홀수(1)인지 짝수(0)인지, 곧 **1의 자리 비트**다. 몫은 1의 자리를 떼어 내고 남은 비트들을 한 칸 오른쪽으로 당긴 수라, 몫을 다시 나누면 2의 자리, 그다음은 4의 자리 비트가 나온다. 먼저 나온 나머지일수록 오른쪽 자리이므로 쓸 때는 마지막(맨 아래) 나머지를 맨 왼쪽에 둔다. 예 6: 6 ÷ 2 = 3 … 0(1의 자리 0), 3 ÷ 2 = 1 … 1(2의 자리 1), 1 ÷ 2 = 0 … 1(4의 자리 1) → 110_2 = 4 + 2 = 6.

  왜 몫 0까지인가: 몫이 1이면 아직 쓰지 않은 자리(값 1)가 하나 남아 있다는 뜻이다. 1 ÷ 2 = 0 … 1을 해야 그 1이 맨 왼쪽 비트로 나온다. 몫이 0이면 남은 자리가 없으니 멈춘다.
- **10진 → 2진(소수)**: 소수 부분에 2를 곱하고 나온 정수 부분(0 또는 1)을 떼어 적는다. 소수 부분이 0이 될 때까지 반복하고 **위에서 아래로** 읽는다.

  왜 위 → 아래인가: 2진 소수의 자릿값은 소수점 바로 오른쪽부터 0.5, 0.25, 0.125, …다. 2를 곱하면 0.5 자리가 1의 자리로 올라와 정수 부분에 드러난다. 그래서 처음 나온 정수 부분이 소수점 바로 오른쪽(0.5 자리), 다음이 0.25 자리다. 예 0.75: 0.75 × 2 = 1.5 → 1(0.5 자리), 0.5 × 2 = 1.0 → 1(0.25 자리) → 0.11_2 = 0.5 + 0.25.

- **2진 ↔ 16진**: 소수점(정수면 오른쪽 끝)에서부터 4비트씩 묶어 표로 바꾼다. 모자란 왼쪽은 0으로 채운다.
  4비트씩인 이유: 4비트로 만들 수 있는 패턴은 0000 ~ 1111, 곧 0 ~ 15의 16가지(2^4 = 16)라 16진 한 자리와 하나씩 짝이 맞는다.
- **16진 자릿값**: 오른쪽부터 1, 16, 256, 4096(16배씩). 0x2BC = 2 × 256 + 11 × 16 + 12 = 700.
- **10진 → 16진**: ÷16 반복(나머지 아래 → 위), 또는 256·16 단위로 빼기. 700 = 2 × 256 + 188, 188 = 11 × 16 + 12 → 2, B, C → 0x2BC.
- **32비트 = 16진 8자리**. 16진 한 자리가 4비트라 32 ÷ 4 = 8자리다. 앞의 0도 써서 8자리를 채운다: 42 = 0x2A → 0x0000002A. 16진은 0x를 붙이고 A~F는 대문자.
- **2진 덧셈**: 0 + 1 = 1, 1 + 1 = 10_2(0 쓰고 1 올림), 1 + 1 + 1 = 11_2(1 쓰고 1 올림). 예 0110_2 + 0111_2: 오른쪽부터 1, 0(올림 1), 1(올림 1), 1 → 1101_2(6 + 7 = 13).
- **16진 덧셈**: 자리 합이 16 이상이면 **16을 빼고** 왼쪽 자리에 1을 올린다(15가 아님). **16진 뺄셈**: 모자라면 왼쪽에서 1을 빌려 **16을 더한** 뒤 뺀다.
  왜 16인가: 16진은 한 자리 왼쪽으로 갈 때마다 자릿값이 16배다(1, 16, 256, …). 그래서 어떤 자리에 16이 모이면 그것은 바로 왼쪽 자리의 1과 같은 양이다. 10진에서 7 + 5 = 12일 때 10을 왼쪽 1로 올리고 2를 쓰는 것과 똑같은 일이다(10진은 10배씩이라 10을 뺀다). 15(F)는 한 자리에 쓸 수 있는 가장 큰 숫자일 뿐 올림 단위가 아니다. 뺄셈도 같다: 왼쪽 자리에서 빌려 온 1은 이 자리로 오면 16이다.

푸는 순서

1. 정수 부분과 소수 부분을 나눈다.
2. 정수는 ÷2 반복 또는 자릿값 빼기, 소수는 ×2 반복(위 규칙대로 읽는다).
3. 둘을 소수점으로 잇는다. 16진이 필요하면 4비트씩 묶는다.
4. 검산: 자릿값을 더해 원래 10진이 나오는지 본다.

함정

- 나누기를 몫 1에서 멈추면 맨 앞 1이 빠진다. 1 ÷ 2 = 몫 0, 나머지 1까지 해야 한다.
- 소수 ×2의 결과를 아래 → 위로 읽으면 틀린다. 0.375 = 0.011_2이지 0.110_2가 아니다.
- 16진 덧셈에서 0xA + 0x8 = 18이면 18 − 16 = 2, 올림 1 → 0x12. 15를 빼서 3으로 쓰는 실수가 잦다.

예제 (시험 수준)

37.375를 2진수로 쓰시오.

풀이 보기

100101.011_2

① 정수 37을 2로 나눈다: 37 ÷ 2 = 18 … 1, 18 ÷ 2 = 9 … 0, 9 ÷ 2 = 4 … 1, 4 ÷ 2 = 2 … 0, 2 ÷ 2 = 1 … 0, 1 ÷ 2 = 0 … 1. 몫이 0이 되었으니 멈춘다.

② 나머지를 아래에서 위로 읽는다: 1, 0, 0, 1, 0, 1 → 100101_2. 처음 나온 나머지(1)가 소수점 바로 왼쪽에 온다.

③ 소수 0.375에 2를 곱한다: 0.375 × 2 = 0.75 → 정수 부분 0, 0.75 × 2 = 1.5 → 1(남은 0.5), 0.5 × 2 = 1.0 → 1(남은 0이라 멈춤).

④ 정수 부분을 위에서 아래로 읽는다: 0.011_2. 처음 나온 0이 소수점 바로 오른쪽에 온다.

⑤ 합치면 100101.011_2.

검산: 32 + 4 + 1 = 37, 0.25 + 0.125 = 0.375.

회독 점검

1. 89를 2진수와 16진수로 쓰시오.

   답 보기

   1011001_2 = 0x59

   ① 89 ÷ 2 = 44 … 1, 44 ÷ 2 = 22 … 0, 22 ÷ 2 = 11 … 0, 11 ÷ 2 = 5 … 1, 5 ÷ 2 = 2 … 1, 2 ÷ 2 = 1 … 0, 1 ÷ 2 = 0 … 1.

   ② 나머지를 아래 → 위로 읽으면 1011001_2.

   ③ 오른쪽 끝부터 4비트씩 묶는다: 101 1001 → 0101 1001(왼쪽을 0으로 채움) → 5, 9 → 0x59.

   검산: 64 + 16 + 8 + 1 = 89.

2. 1101 0110_2를 10진수와 16진수로 쓰시오.

답 보기

214 = 0xD6

① 오른쪽 끝 자리를 0번째로 세면 k번째 자리의 자릿값은 2^k다. 1이 있는 자리: 7번째(128), 6번째(64), 4번째(16), 2번째(4), 1번째(2).

② 128 + 64 + 16 + 4 + 2 = 214.

③ 4비트 묶음: 1101 = D, 0110 = 6 → 0xD6.

검산: 13 × 16 + 6 = 214.

---

3. 10.625를 2진수로 쓰시오.

답 보기

1010.101_2

① 정수 10: 10 ÷ 2 = 5 … 0, 5 ÷ 2 = 2 … 1, 2 ÷ 2 = 1 … 0, 1 ÷ 2 = 0 … 1 → 아래 → 위로 1010_2.

② 소수 0.625: 0.625 × 2 = 1.25 → 1(남은 0.25), 0.25 × 2 = 0.5 → 0, 0.5 × 2 = 1.0 → 1 → 위 → 아래로 0.101_2.

③ 합쳐서 1010.101_2.

검산: 8 + 2 = 10, 0.5 + 0.125 = 0.625.

---

4. 16진 덧셈 0x3A7 + 0x2C9를 계산하시오.

답 보기

0x670

① 맨 오른쪽: 7 + 9 = 16. 16 이상이므로 16을 빼면 0, 왼쪽에 1을 올린다.

② 가운데: A(10) + C(12) + 올림 1 = 23. 23 − 16 = 7, 다시 1을 올린다.

③ 맨 왼쪽: 3 + 2 + 올림 1 = 6.

④ 왼쪽부터 6, 7, 0 → 0x670.

검산: 0x3A7 = 935, 0x2C9 = 713, 935 + 713 = 1648 = 6 × 256 + 7 × 16 + 0 = 0x670.

---

5. 16진 뺄셈 0x5A2 − 0x1B7을 계산하시오.

답 보기

0x3EB

① 맨 오른쪽: 2 − 7은 모자란다. 왼쪽 자리(A)에서 1을 빌리면 이 자리에 16이 더해진다: 2 + 16 − 7 = 11 = B. 왼쪽 A는 빌려 줬으니 9가 된다.

② 가운데: 9 − B(11)도 모자란다. 왼쪽(5)에서 빌려 9 + 16 − 11 = 14 = E. 왼쪽 5는 4가 된다.

③ 맨 왼쪽: 4 − 1 = 3.

④ 0x3EB.

검산: 0x5A2 = 1442, 0x1B7 = 439, 1442 − 439 = 1003 = 3 × 256 + 14 × 16 + 11 = 0x3EB.

---

6. 빈칸: 2^{12} = ( ), 2^{16} = ( ), 300을 32비트 값으로 보고 16진 8자리로 쓰면 ( ).

답 보기

4096, 65536, 0x0000012C

① $2^{12} = 2^{10} \times 2^2 = 1024 \times 4 = 4096$. $2^{16} = 2^8 \times 2^8 = 256 \times 256 = 65536$.

② 300 = 256 + 44, 44 = 2 × 16 + 12 → 1 × 256 + 2 × 16 + 12(C) → 0x12C.

③ 32비트 = 16진 8자리이므로 앞에 0을 5개 채워 0x0000012C.

더 자세히: 이론 노트 기초 · 1, 2, 3

기초 · 2 슬라이드 Ch2 p.18–20, 31 / Ch4 p.13

2의 보수와 부호 확장

시험에 나오는 모양: 음수를 2진·16진으로 쓰기, 0xFFFFFFFC 같은 값을 10진으로 읽기, 12비트 즉시값을 32비트로 부호 확장하기, 같은 비트를 부호 있는 수·무부호 수로 읽기.

먼저 알아 둘 말

- **명령어(instruction)**: CPU에게 시키는 일 하나. 예 "더해라", "메모리에서 읽어라". 하나가 32비트 기계어로 저장된다. [Ch2 p.2]
- **레지스터(register)**: CPU 안에 있는 32개의 저장 칸 x0~x31. 칸 하나에 32비트 값이 들어간다. 계산은 이 칸들 사이에서 한다. [Ch2 p.10]
- **즉시값(immediate)**: 명령어 안에 직접 적힌 상수. 기본 12비트라서 계산 전에 32비트로 늘려야 한다. 그 일을 하는 장치가 ImmGen(즉시값 생성기)이다. [Ch2 p.16, Ch4 p.13]

쉽게 말하면

2의 보수(two's complement)에서는 맨 왼쪽 비트(MSB, Most Significant Bit)의 자릿값만 음수다. 맨 오른쪽 비트는 LSB(Least Significant Bit)라 부른다. 8비트라면 맨 왼쪽 자리가 +128이 아니라 −128이다. 나머지 자리는 무부호와 똑같다.

왜 이렇게 되나

- **"보수(complement)"라는 말**: 어떤 수에 더해서 정해진 기준 수를 채워 주는 짝이다. 10진 예: 10을 기준으로 하면 7의 보수는 3이다(7 + 3 = 10). **2의 보수**는 n비트 수 x에 더해서 2^n(2진으로 1 뒤에 0이 n개)을 채우는 짝, 곧 **$2^n - x$**다. 기준이 2의 거듭제곱이라 "2의" 보수라 부른다. PDF 외 보충
- **왜 그 짝이 −x 역할을 하나**: 4비트 예. 3 = 0011의 2의 보수는 16 − 3 = 13 = 1101. 0011 + 1101 = 1 0000인데 칸이 4개뿐이라 맨 앞 1은 칸 밖으로 버려지고 0000이 남는다. 3과 더해 0이 되니 1101이 −3이다. 그래서 "n비트에서 −x는 $2^n - x$의 비트 패턴으로 저장한다"고 약속한 방식이 **2의 보수 표현**이다. 음수를 나타내는 방법은 부호-크기(sign and magnitude), 1의 보수, 2의 보수 세 가지가 있었고 최종적으로 2의 보수를 쓴다. [Ch2 p.19]
- **MSB가 -2^{n-1}이라는 규칙과 같은 말이다**: 1101을 MSB 규칙으로 읽으면 −8 + 4 + 1 = −3. 무부호로 읽으면 8 + 4 + 1 = 13. MSB 자리를 +8 대신 −8로 읽으면 값이 16(= 2^4) 작아지므로 13 − 16 = −3, 곧 $2^n - x$에서 2^n을 빼는 것과 같다.
- **왜 "뒤집고 1을 더하면" −x인가**: x와 ~x(모든 비트를 뒤집은 것)를 더하면 자리마다 0 + 1이나 1 + 0이라 올림 없이 전부 1, 곧 $2^n - 1$이 된다(1111_2 = 16 − 1 = 15처럼 1이 n개면 $2^n - 1$). 그래서 ~x = $2^n - 1 - x$이고, 1을 더하면 $2^n - x = -x$의 패턴이 된다. 4비트 3: ~0011 = 1100(= 12 = 15 − 3), 1을 더하면 1101. 뒤집기만 한 ~x는 $2^n - 1$(모두 1)에 대한 짝이라 1의 보수라고 부르고, 16진 한 자리씩 뒤집으면 F(1111)에서 빼는 것이라 **15 − (그 자리 숫자)**다. 16은 1을 더한 뒤에야 나오는 수다. [Ch2 p.20]
- **범위가 왜 $-2^{n-1} \sim 2^{n-1} - 1$인가**: 가장 작은 값은 음수 자리(MSB)만 1이고 양수 자리는 모두 0인 패턴 = -2^{n-1}. 가장 큰 값은 MSB가 0이고 나머지 n − 1개가 모두 1인 패턴 = $2^{n-1} - 1$. 4비트: 1000 = −8, 0111 = 7. 무부호는 n개가 모두 1인 $2^n - 1$이 최대다.
- **같은 비트, 두 가지 읽기(약속)**: 레지스터·메모리에 들어 있는 것은 0과 1뿐이고 "이 값은 음수"라는 표시는 따로 없다. 같은 비트를 무부호(모든 자리 양수)로 읽을지 부호 있는 수(MSB 자리만 음수)로 읽을지는 **그 값을 쓰는 명령어가 정한다**. 두 읽기의 차이는 MSB 자리 하나뿐이다: 무부호로는 $+2^{n-1}$, 부호 있는 수로는 -2^{n-1}이라 차이는 2^n이다. 그래서 MSB가 1이면 무부호 값 = 부호 있는 값 + 2^n, MSB가 0이면 두 값이 같다.
- **32비트에서 "2^{32}에서 얼마 모자라나" 구하기**: 2^{32} = 0x1 0000 0000(1 뒤에 0이 8개, 9자리)이고 0xFFFFFFFF는 그보다 딱 1 작다. 그래서 앞이 F로 채워진 값은 **(0xFFFFFFFF까지 남은 양) + 1**만큼 모자란다. 예 0xFFFFFFF0: 0xFFFFFFFF − 0xFFFFFFF0 = 0xF = 15, 여기에 1을 더해 16 → −16. 15에서 멈추면 −15로 틀린다(뒤집기만 하고 +1을 빼먹은 것과 같은 실수).

- **부호 확장이 왜 값을 지키나**: 4비트 1110 = −8 + 4 + 2 = −2. 8비트로 늘려 1111 1110이 되면 −128 + 64 + 32 + 16 + 8 + 4 + 2 = −2로 같다. 새 MSB(−128)와 1로 채운 자리(64, 32, 16)와 옛 MSB 자리(이제 +8)를 합하면 −128 + 64 + 32 + 16 + 8 = −8이라 옛 MSB의 −8과 똑같기 때문이다. 0으로 채웠다면 0000 1110 = 14가 되어 값이 바뀐다. 원래 4비트(1110)는 손대지 않는다. 슬라이드 예: −2 = 1111 1110 → 1111 1111 1111 1110. [Ch2 p.20]

외울 것

- **n비트 부호 있는 수 범위**: $-2^{n-1} \sim 2^{n-1} - 1$. 무부호 수(unsigned)는 $0 \sim 2^n - 1$. [Ch2 p.18–19]
- MSB가 부호 비트(sign bit): 1이면 음수, 0이면 양수. [Ch2 p.19]
- **부정(= 부호 반전, x → −x)**: ~x + 1 = −x (모든 비트를 뒤집고 1을 더함). [Ch2 p.20]
- **음수 읽기**: 부호 비트가 1이면 -2^{n-1} + (나머지 비트의 값). 또는 뒤집고 1을 더해 크기를 구한 뒤 − 를 붙인다.
- **부호 확장(sign extension)**: 비트 수를 늘릴 때 부호 비트를 왼쪽 빈칸에 복사한다. 값이 그대로 유지된다. [Ch2 p.20]
- ImmGen(즉시값 생성기)은 명령어의 12비트 즉시값을 32비트로 부호 확장한다. [Ch4 p.13]

비트 수	부호 있는 범위	무부호 범위
4	−8 ~ 7	0 ~ 15
8	−128 ~ 127	0 ~ 255
12	−2048 ~ 2047	0 ~ 4095
32	$-2^{31} \sim 2^{31} - 1$	$0 \sim 2^{32} - 1$

32비트 16진	부호 있는 값	무부호 값
0xFFFFFFFF	−1	$2^{32} - 1$
0xFFFFFFFE	−2	$2^{32} - 2$
0xFFFFFFFC	−4	$2^{32} - 4$
0x80000000	-2^{31}	2^{31}
0x7FFFFFFF	$2^{31} - 1$	$2^{31} - 1$

일반형: n비트에서 −a의 비트 패턴을 무부호로 읽으면 $2^n - a$다. 8비트 −37 = 256 − 37 = 219 = 0xDB.

거꾸로 32비트 0xFFFFFFFC는 2^{32}에서 4 모자라므로 −4.

푸는 순서

1. 비트 수 n을 확인한다(8, 12, 32 ...). 16진이면 2진으로 펼친다.
2. MSB가 0이면 무부호처럼 읽고, 1이면 위 "음수 읽기"대로 읽는다.
3. 부호 확장이면 늘어난 왼쪽 칸을 전부 MSB 값으로 채운다(16진으로는 F 또는 0).

함정

- 뒤집기만 하고 +1을 빼먹는다. ~x는 −x − 1이다. 0xFFFFFFF0을 −15로 읽는 것이 이 실수다: 뒤집으면 0x0000000F = 15, 여기에 1을 더한 16이 크기라 −16이다.
- 자리마다 16 − (숫자)를 한다. 16은 덧셈 넘침·뺄셈 빌림에만 쓴다. 뒤집기는 15 − (숫자) 후 끝에 +1 한 번이고, 헷갈리면 크기는 **다음 큰 수에서 한 번에 빼기**로 구한다: 0xFE8의 크기 = 0x1000 − 0xFE8 = 0x18 = 24(자리마다 빼면 0x128 = 296이 되어 틀림).
- 부호 확장하면서 원래 숫자를 바꾼다. 부호 확장은 **옮겨 적기**라 원래 자리는 그대로 두고 왼쪽 빈칸만 MSB로 채운다: 0xFE8 → 0xFFFFFFE8. MSB가 1인데 0으로 채우면 큰 양수가 된다.
- 같은 비트도 읽는 방식에 따라 값이 다르다. 두 레지스터를 비교해 "작으면 다른 줄로 건너뛰는" 명령어가 둘 있는데, blt는 부호 있는 수로, bltu는 무부호 수로 읽고 비교한다. 그래서 0xFFFFFFFF는 blt에서 −1, bltu에서 2^{32} − 1이다(자세히는 Ch2 · 2). [Ch2 p.31]

예제 (시험 수준)

−6을 32비트 2의 보수로 나타내 16진 8자리로 쓰고, 결과를 다시 10진으로 읽어 확인하시오.

풀이 보기

0xFFFFFFFA

① +6을 32비트로 쓴다: 0x00000006, 마지막 바이트는 0000 0110.

② 모든 비트를 뒤집는다(~x): 0은 1로, 1은 0으로. 16진으로 한꺼번에 하려면 자리마다 **15** − (그 자리 숫자)다(F = 1111에서 빼는 것이라 15, 16이 아님). 0x00000006 → 0xFFFFFFF9.

③ 1을 **맨 끝에 한 번만** 더한다(자리마다 더하지 않음): 0xFFFFFFF9 + 1 = 0xFFFFFFFA. 맨 끝 9 + 1 = 10 = A, 올림 없음.

④ 확인: MSB가 1이라 음수. 0xFFFFFFFF까지 F − A = 15 − 10 = 5가 남고, 2^{32}까지는 1을 더한 6이 모자라므로 −6이다(위 "2^{32}에서 얼마 모자라나").

회독 점검

1. −37을 8비트 2의 보수로 2진과 16진으로 쓰시오.

   답 보기

   1101 1011 = 0xDB

   ① +37 = 32 + 4 + 1 → 0010 0101.

   ② 모든 비트를 뒤집는다: 1101 1010.

   ③ 1을 더한다: 1101 1010 + 1 = 1101 1011.

   ④ 4비트 묶음: 1101 = D, 1011 = B → 0xDB.

   검산: −128 + (0101 1011 = 91) = −37.

---

2. 8비트 2의 보수 1010 0110을 10진으로 쓰시오.

   답 보기

   −90

   ① MSB가 1이므로 음수. MSB 자릿값은 -2^7 = −128.

   ② 나머지 7비트 010 0110 = 32 + 4 + 2 = 38.

   ③ −128 + 38 = −90.

   검산(반전 + 1): 0101 1001 + 1 = 0101 1010 = 90 → −90.

---

3. 32비트 값 0xFFFFFFF0을 부호 있는 수로 읽으면? 무부호 수로 읽으면?

답 보기

부호 있는 수 −16, 무부호 수 $2^{32} - 16 = 4{,}294{,}967{,}280$

① MSB(맨 앞 F = 1111의 첫 비트)가 1이므로 부호 있는 수로는 음수.

② 크기를 구한다. 0xFFFFFFFF(= $2^{32} - 1$)까지 0xF = 15가 남고, 2^{32}까지는 1이 더 남으므로 15 + 1 = 16(= 0x10) 모자란다. 그래서 부호 있는 값은 −16(−15가 아님).

②′ 같은 일을 "뒤집고 1 더하기"로: 0xFFFFFFF0을 자리마다 15 − (숫자)로 뒤집으면 0x0000000F = 15, 1을 더하면 16 → −16.

③ 무부호로는 MSB 자리도 양수로 읽으니 부호 있는 값에 2^{32}을 더한 $2^{32} - 16 = 4{,}294{,}967{,}296 - 16 = 4{,}294{,}967{,}280$.

---

4. 12비트 즉시값 1111 1110 1000을 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

답 보기

0xFFFFFFE8 = −24

① 12비트의 MSB(11번 비트)가 1이므로 늘어나는 위쪽 20비트를 모두 1로 채운다.

② 위쪽 20비트(모두 1) = 16진 FFFFF(5자리), 원래 12비트 1111 1110 1000 = FE8(3자리). 이어 붙이면 0xFFFFFFE8.

③ 크기를 구한다. 앞 6자리는 둘 다 F라 차이가 없고 아래 두 자리만 다르다: 0xFF − 0xE8 = 255 − 232 = 23. 그래서 0xFFFFFFFF까지 23이 남고, 2^{32}까지는 1을 더한 24(= 0x18) 모자란다 → −24.

③′ 뒤집고 1 더하기로도: 0xFFFFFFE8 → 자리마다 15 − (숫자) → 0x00000017 = 23, 1을 더해 24 → −24.

검산(12비트에서): −2048 + (111 1110 1000 = 2024) = −24. 확장 전후 값이 같다.

---

5. 8비트 패턴 1100 1000을 부호 있는 수(2의 보수)로 읽으면? 무부호 수로 읽으면?

답 보기

부호 있는 수 −56, 무부호 수 200

① 무부호: 1이 있는 자리의 자릿값을 더한다. 128 + 64 + 8 = 200.

② 부호 있는 수: MSB가 1이라 음수. MSB 자릿값을 −128로 바꾼다: −128 + (100 1000 = 64 + 8 = 72) = −56.

검산(일반형): 무부호 값 = 2^8 − 56 = 256 − 56 = 200. 두 값이 맞물린다.

---

6. 빈칸: 12비트 2의 보수가 나타낼 수 있는 범위는 ( ) ~ ( )이고, 32비트 부호 있는 수의 최솟값은 16진으로 ( )이다.

답 보기

−2048 ~ 2047, 0x80000000

① n = 12를 공식 -2^{n-1} ~ 2^{n-1} − 1에 넣는다. 2^{11} = 2048이므로 −2048 ~ 2047.

② 최솟값은 MSB만 1이고 나머지가 0인 패턴: -2^{31} + 0. 32비트로 1000 0000 … 0000 → 0x80000000.

더 자세히: 이론 노트 기초 · 3 / Ch2 · 4, 7 / Ch4 · 3

기초 · 3 슬라이드 Ch2 p.25–27, 30

비트 연산과 시프트

시험에 나오는 모양: and·or·xor 결과를 16진으로 쓰기, sll·srl·sra 결과 쓰기, [19:15] 같은 비트 범위 읽기, slli 2가 왜 ×4인지(배열 인덱스).

먼저: 어셈블리 한 줄 읽는 법

이 단원 예제와 점검에는 `and x9, x10, x11` 같은 줄이 나온다. 읽는 규칙은 세 가지다(자세한 표는 기초 · 4).

1. **맨 앞 단어 = 할 일(명령어 이름)**. `and` = AND 하라, `srli` = 오른쪽으로 밀어라(shift right logical immediate).
2. **쉼표 뒤 첫 번째 = 결과를 넣을 칸**, 그 뒤 = 계산에 쓸 값. `and x9, x10, x11` = "x10과 x11을 AND해서 x9에 넣어라", 즉 x9 ← x10 and x11. x10과 x11은 읽기만 하므로 값이 그대로다. [Ch2 p.9]
3. **이름 끝에 i가 붙으면 마지막 값이 숫자 상수**(즉시값)다. `srli x9, x9, 8` = "x9를 오른쪽으로 8칸 밀어서 다시 x9에 넣어라", 즉 x9 ← x9 >> 8. 결과 칸과 읽는 칸이 같으면 자기 값을 바꾼 것이다.

"실행했다"는 맨 앞 칸(결과 칸)의 값이 바뀌었다는 뜻이다. 문제가 묻는 것은 보통 그 칸의 새 값이다.

외울 것

a	b	and	or	xor
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

세 연산의 뜻(위 표는 이 뜻을 네 경우에 적용한 것):

- **and(그리고)**: 두 비트가 **둘 다 1일 때만** 1. 하나라도 0이면 0이다. 그래서 0 and 0 = 0이다(1이 하나도 없으니 "둘 다 1"이 아님). 스위치 두 개를 일렬로 이은 전등과 같다: 둘 다 켜야 불이 들어온다.
- **or(또는)**: 두 비트 중 **하나라도 1이면** 1. 둘 다 0일 때만 0이다.
- **xor(배타적 또는, exclusive or)**: 두 비트가 **서로 다를 때만** 1. 같으면(0과 0, 1과 1) 0이다.
- 여러 비트짜리 값끼리 할 때는 **같은 번호의 비트끼리 따로따로** 위 규칙을 적용한다. 예 1100 and 1010: 자리마다 1·1 → 1, 1·0 → 0, 0·1 → 0, 0·0 → 0 → 1000.
- **and = 마스크(mask)**: 마스크 1인 자리만 남기고 나머지는 0으로 지운다. [Ch2 p.26]
- **or = 1로 설정**: 마스크 1인 자리를 1로 켜고 나머지는 그대로. [Ch2 p.26]
- **xor = 뒤집기**: 같으면 0, 다르면 1. 마스크 1인 자리만 뒤집힌다. 모두 1인 값과 xor하면 NOT이 된다(RISC-V에는 NOT 명령이 없음). [Ch2 p.27]
- 비트 연산은 자리끼리만 계산하고 올림이 없다. 그래서 **16진 자리 단위**로 계산해도 된다. 16진 F = 1111(네 비트 모두 1), 0 = 0000(모두 0)이라 아래 표가 나온다. x는 16진 한 자리(네 비트)다.

16진 자리 계산	결과	비트 단위 이유 (b = x의 비트 하나)	예
x and F	x 그대로	b and 1 = b (b가 1이면 둘 다 1이라 1, b가 0이면 0). 네 비트 모두 1과 만나니 x가 그대로 남는다	B and F = 1011 and 1111 = 1011 = B
x and 0	0	b and 0 = 0 (한쪽이 0이면 "둘 다 1"이 될 수 없음)	B and 0 = 0000 = 0
x or F	F	b or 1 = 1 (한쪽이 이미 1)	B or F = 1111 = F
x or 0	x 그대로	b or 0 = b (0은 1을 보태지 않음)	B or 0 = 1011 = B
x xor F	15 − x	b xor 1 = b의 반대(다르면 1). 네 비트가 모두 뒤집히고, x와 뒤집힌 x를 더하면 1111 = 15이므로 뒤집힌 값 = 15 − x	B xor F = 0100 = 4 = 15 − 11
x xor 0	x 그대로	b xor 0 = b (b가 0이면 같아서 0, 1이면 달라서 1)	B xor 0 = 1011 = B

마스크 자리가 0도 F도 아니면(예 C and 6) 두 숫자를 2진으로 펼쳐 비트마다 계산한다: 1100 and 0110 = 0100 = 4.

- **비트 번호**: 맨 오른쪽(LSB)이 0번, 32비트면 맨 왼쪽(MSB)이 31번. **[hi:lo]** = hi번~lo번 비트(슬라이드의 [19-15]와 같은 뜻). [19:15]는 19, 18, 17, 16, 15번의 5개. imm[12:1]은 즉시값의 12번~1번 비트(0번은 칸에 없음).
- **레지스터 번호**: 레지스터 32개(기초 · 2)는 x0, x1, ..., x31처럼 **x 뒤의 숫자가 번호**다. `add x9, x20, x21`은 9번, 20번, 21번 레지스터를 쓴다는 뜻이다. [Ch2 p.10]
- CPU가 실제로 읽는 32비트 기계어에는 "x20" 같은 글자를 쓸 수 없어서 **번호만 2진수로** 적는다. 번호는 0~31이고 2^5 = 32이므로 레지스터 하나에 **5비트 칸**이면 충분하다. 계산에 읽는 첫 번째 레지스터 칸을 **rs1**, 두 번째를 **rs2**, 결과를 넣을 칸을 **rd**라 부른다(형식은 Ch2 · 4). [Ch2 p.22]

예: `add x9, x20, x21`의 기계어 0x015A04B3을 칸으로 나누면 아래와 같다. 각 레지스터 칸의 5비트를 10진으로 바꾸면 그대로 레지스터 번호가 나온다. [Ch2 p.22]

칸	비트 위치	들어 있는 2진	10진	뜻
rs2	[24:20]	10101_2	16 + 4 + 1 = 21	두 번째로 읽는 레지스터 = x21
rs1	**[19:15]**	10100_2	16 + 4 = 20	첫 번째로 읽는 레지스터 = **x20**
rd	[11:7]	01001_2	8 + 1 = 9	결과를 넣는 레지스터 = x9

그래서 "[19:15]가 rs1 칸이라면 몇 번 레지스터인가"는 **그 5비트를 10진으로 바꿔 앞에 x를 붙이라**는 뜻이다.

시프트	방향	빈칸 채우기	값으로 보면
sll, slli	왼쪽	오른쪽에 0	k칸 = × 2^k(넘친 비트는 버림)
srl, srli	오른쪽	왼쪽에 0	무부호 ÷ 2^k(몫)
sra, srai	오른쪽	왼쪽에 부호 비트(MSB) 복사	부호 있는 ÷ 2^k(내림)

슬라이드 예: 1100 0011을 sra 1칸 → 1110 0001. [Ch2 p.25] 4칸 시프트 = 16진 한 자리 이동(16진 한 자리가 4비트이므로).

왜 왼쪽 k칸이 × 2^k인가: 왼쪽으로 한 칸 밀면 모든 1이 자릿값이 두 배인 자리로 옮겨 가므로 합도 두 배가 된다. 10진에서 오른쪽에 0을 하나 붙이면(왼쪽으로 한 칸) × 10이 되는 것과 같다. 예 0011(3) → 0110(6) → 1100(12). 두 칸이면 × 2 × 2 = × 4. 오른쪽으로 밀면 반대로 자릿값이 절반이 되고, 칸 밖으로 버려지는 오른쪽 비트가 나머지라 ÷ 2^k의 몫만 남는다. 예 1101(13) srl 1 → 0110(6) = 13 ÷ 2의 몫. sra가 빈칸을 MSB로 채우는 것은 부호 확장(기초 · 2)과 같은 이유다: 음수를 밀어도 음수로 남게 하려는 것이다.

시프트가 가장 많이 쓰이는 곳은 배열 주소 계산이다. 메모리(memory)는 바이트마다 0부터 번호가 붙은 긴 칸 줄이고, 그 번호가 **주소(address)**다. [Ch2 p.12] 배열(array)은 같은 종류의 값을 메모리에 차례로 붙여 놓은 것이고, A[i]는 배열 A의 i번 원소(0번부터 셈)다. int(정수) 하나는 4바이트라 A[i]는 배열 시작 주소에서 i × 4바이트 떨어져 있다. 이 거리(바이트 수)를 **오프셋(offset)**이라 한다. i × 4는 `slli x10, x22, 2` (x22 = i를 왼쪽으로 2칸)로 구한다. [Ch2 p.30]

푸는 순서

1. 두 값을 같은 자리 수(32비트면 8자리)로 맞춰 위아래로 놓는다.
2. 마스크 자리가 0이나 F면 위 요령으로 바로 쓴다. 섞인 자리만 2진으로 펼쳐 진리표를 적용한다.
3. 시프트는 2진으로 펼쳐 옮기고, 빈칸을 0(sll, srl) 또는 MSB(sra)로 채운 뒤 다시 16진으로 묶는다.

유형별 풀이 예 (점검 문제와 같은 모양, 숫자만 다름)

① NOT 만들기 (xor로 모든 비트 뒤집기)

NOT은 "모든 비트를 반대로 바꾸기"(0 → 1, 1 → 0)다. RISC-V에는 not 명령이 없어서, **모든 비트가 1인 값과 xor**해서 만든다. 이유: b xor 1은 "b와 1이 다르면 1"이라 b가 0이면 1, b가 1이면 0, 즉 b의 반대가 된다. [Ch2 p.27]

예: 8비트 0xA5를 0xFF와 xor. 0xA5 = 1010 0101, 0xFF = 1111 1111. 자리마다 뒤집으면 0101 1010 = **0x5A**. 16진 자리로 해도 같다: 15 − A(10) = 5, 15 − 5 = 10 = A → 0x5A. 이 결과를 "NOT(모든 비트 반전)과 같다"고 답한다.

② 여러 칸 시프트 (8비트, 2진)

k칸 시프트는 세 단계로 한다. (가) 미는 방향의 끝에서 k개 비트를 버린다. (나) 남은 비트를 순서 그대로 반대쪽 끝까지 붙인다. (다) 비어 있는 k칸을 **모두 같은 값**으로 채운다: sll·srl은 0, sra는 원래 MSB.

1001 0110을 2칸	(가) 버리는 비트	(나) 남은 비트	(다) 빈칸 2개	결과
sll 2 (왼쪽)	왼쪽 끝 10	01 0110 → 왼쪽으로 붙임	오른쪽에 00	**0101 1000**
srl 2 (오른쪽)	오른쪽 끝 10	1001 01 → 오른쪽으로 붙임	왼쪽에 00	**0010 0101**
sra 2 (오른쪽)	오른쪽 끝 10	1001 01 → 오른쪽으로 붙임	원래 MSB가 1 → 왼쪽에 11	**1110 0101**

srl과 sra는 (다)만 다르다. 원래 MSB가 0이면 둘의 결과가 똑같다.

③ 32비트 16진 값을 4칸 시프트

16진 한 자리 = 4비트이므로 **4칸 시프트 = 16진 숫자 한 자리 이동**이다. 2진으로 펼칠 필요가 없다.

- srli 4: 맨 오른쪽 16진 숫자 하나를 버리고 나머지를 한 자리 오른쪽으로, 맨 왼쪽에 **0**. 0x80000120 → **0x08000012**.
- srai 4: 같지만 맨 왼쪽 새 자리를 MSB 4개로 채운다. MSB는 맨 앞 16진 숫자의 첫 비트다(8 = **1**000). 1이 4개면 1111 = **F**, 0이 4개면 0. 0x80000120 → **0xF8000012**. 부호 확장(기초 · 2)에서 F로 채우던 것과 같은 원리다.
- slli 4: 맨 왼쪽 숫자 하나를 버리고 한 자리 왼쪽으로, 맨 오른쪽에 0. 0x00000120 → 0x00001200.

④ slli로 배열 오프셋 구하기

x5 = 7(인덱스 i)일 때 `slli x6, x5, 2`. 먼저 읽기: "x5를 왼쪽으로 2칸 밀어 x6에 넣어라". 왼쪽 2칸 = × 2^2 = × 4이므로 x6 = 7 × 4 = **28**(2진으로 0111 → 1 1100 = 16 + 8 + 4 = 28). 뜻: int는 4바이트라 A[7]은 배열 시작 주소에서 28바이트 떨어져 있다. 즉 28은 **A[7]의 오프셋(시작 주소에서 A[7]까지의 바이트 수)**이고, 시작 주소 + 28이 A[7]의 주소다. [Ch2 p.30]

함정

- 0 and 0 = 1이 아니다. and는 "둘 다 1일 때만 1"이라 0 and 0 = 0이다. "둘이 같으면 1"이라는 규칙은 위 세 연산 어디에도 없다(xor는 반대로 "다르면 1").
- and를 "작은 쪽 고르기"로 외우면 틀린다. F and 3 = 3인 이유는 F = 1111이 3 = 0011의 비트를 하나도 지우지 않기 때문이다. 6 and 3 = 0110 and 0011 = 0010 = 2(6도 3도 아님), 5 and A = 0101 and 1010 = 0000 = 0이다. 항상 비트로 펼쳐 확인한다.
- 음수(MSB 1)를 srl하면 왼쪽에 0이 들어와 큰 양수가 된다. 부호를 지키는 것은 sra뿐이다.
- 왼쪽 시프트로 1이 칸 밖으로 넘치면 $\times 2^k$가 성립하지 않는다(8비트 1100 0000을 sll 1 → 1000 0000).
- sra는 음수 홀수에서 내림이다: −7 sra 1 = −4(1111 1001 → 1111 1100). 부호 있는 나눗셈 몫 −7 ÷ 2 = −3과 다르다. PDF 외 보충

예제 (시험 수준)

x10 = 0x12345678이다. x11 = 0x0000FF00일 때 `and x9, x10, x11` 을 실행한 뒤 `srli x9, x9, 8` 을 실행했다. 각 단계의 x9를 16진 8자리로 쓰시오.

풀이 보기

and 뒤 0x00005600, srli 뒤 0x00000056

⓪ 두 줄을 말로 읽는다. `and x9, x10, x11` = x9 ← x10 and x11(0x12345678 and 0x0000FF00). `srli x9, x9, 8` = x9 ← (방금 만든 x9)를 오른쪽으로 8비트 민 값.

① 자리를 맞춘다: 1 2 3 4 5 6 7 8 과 0 0 0 0 F F 0 0.

② and는 마스크가 0인 자리를 0으로, F인 자리를 그대로 남긴다. F는 5번째·6번째 자리(5, 6)에만 있으므로 0x00005600.

③ srli 8칸 = 16진 두 자리 오른쪽 이동. 맨 오른쪽 00이 버려지고 왼쪽에 0이 두 자리 들어온다: 0x00000056.

시험과 연결: and로 원하는 바이트만 남기고 시프트로 맨 아래로 내리면 그 바이트를 꺼낼 수 있다. [Ch2 p.25–26]

회독 점검

1. 0x0000ABCD AND 0x00000FF0을 16진 8자리로 쓰시오.

   답 보기

   0x00000BC0

   ① 아래 4자리만 보면 A B C D 와 0 F F 0.

   ② 마스크 0인 자리는 0, F인 자리는 그대로: A → 0, B → B, C → C, D → 0.

   ③ 0x00000BC0.

---

2. 32비트 값 0x000A4000의 [19:15] 비트 5개를 2진으로 쓰고, 이 5비트가 rs1 칸이라면 몇 번 레지스터인지 쓰시오.

   답 보기

   10100_2, x20

   ① 16진 한 자리 = 4비트. 맨 오른쪽 자리가 3~0번, 다음이 7~4번, 11~8번, 15~12번, 19~16번 비트다.

   ② 0x000A4000을 오른쪽부터 보면 0(3~0번), 0(7~4번), 0(11~8번), 4(15~12번), A(19~16번).

   ③ 19~16번 = A = 1010. 15번은 4 = 0100의 맨 왼쪽 비트라 0.

   ④ 19번부터 15번까지 이어 쓰면 1010 + 0 → 10100_2.

   ⑤ 레지스터 칸의 5비트는 레지스터 번호를 2진으로 적은 것이다. 10100_2 = 16 + 4 = 20이므로 20번 레지스터, 즉 **x20**이다. 이 명령어가 계산에 읽는 첫 번째 레지스터가 x20이라는 뜻이다.

   검산: 15칸 srli 한 뒤 0x1F(아래 5비트만 1)와 and해도 20이 나온다.

---

3. 8비트 값 0xC3을 0xFF와 xor한 결과를 16진과 2진으로 쓰고, 이 연산이 무엇과 같은지 쓰시오.

   답 보기

   0x3C = 0011 1100, NOT(모든 비트 반전)과 같다

   ① 0xC3 = 1100 0011.

   ② 1111 1111과 xor하면 각 비트가 1과 비교되므로 0은 1, 1은 0이 된다: 0011 1100.

   ③ 16진 자리로도: C(12) → 15 − 12 = 3, 3 → 15 − 3 = 12 = C. 0x3C.

   시험과 연결: RISC-V는 NOT을 "모두 1인 값과 XOR"로 만든다. [Ch2 p.27]

---

4. 8비트 1011 0100을 2칸 sll, srl, sra 한 결과를 각각 2진으로 쓰시오.

답 보기

sll 1101 0000, srl 0010 1101, sra 1110 1101

① sll 2: 모든 비트를 왼쪽으로 2칸. 왼쪽 끝 10은 칸 밖으로 버리고 오른쪽 빈 2칸에 0 → 1101 0000.

② 오른쪽으로 2칸 옮기면 오른쪽 끝 00이 버려지고 원래 왼쪽 6비트 1011 01이 오른쪽 6칸에 온다.

③ srl: 왼쪽 빈 2칸을 0으로 → 0010 1101.

④ sra: 원래 MSB가 1이므로 왼쪽 빈 2칸을 1로 → 1110 1101.

검산: 부호 있는 값 1011 0100 = −128 + 52 = −76, −76 ÷ 4 = −19, 1110 1101 = −128 + 109 = −19.

---

5. 32비트 0xF0000040을 4칸 srai 한 결과와 4칸 srli 한 결과를 16진 8자리로 쓰시오.

답 보기

srai 0xFF000004, srli 0x0F000004

① 4칸 = 16진 한 자리. 맨 오른쪽 0이 버려지고 F000004가 한 자리 오른쪽으로 간다: 0x?F000004.

② 맨 앞 자리 F = 1111이므로 MSB = 1.

③ srai는 빈 4비트를 MSB(1)로 채워 1111 = F → 0xFF000004.

④ srli는 0으로 채워 0x0F000004.

---

6. x22 = 9(인덱스 i)일 때 `slli x10, x22, 2` 실행 뒤 x10은? 이 값은 int 배열에서 무엇인가?

답 보기

36, 배열 시작 주소에서 A[9]까지의 바이트 수(i × 4)

① 왼쪽 2칸 시프트 = × 2^2 = × 4. 9 × 4 = 36.

② 2진으로: 1001 → 10 0100 = 32 + 4 = 36.

③ int는 4바이트라 i번 원소는 시작 주소에서 i × 4바이트 떨어져 있다. 그래서 곱셈 대신 slli 2를 쓴다.

[Ch2 p.30]

---

7. (다시 풀기, 3번 유형) 8비트 값 0x6E를 0xFF와 xor한 결과를 16진과 2진으로 쓰고, 이 연산이 무엇과 같은지 쓰시오.

   답 보기

   0x91 = 1001 0001, NOT(모든 비트 반전)과 같다

   ① 0x6E를 2진으로: 6 = 0110, E = 1110 → 0110 1110.

   ② 0xFF = 1111 1111. 모든 비트가 1과 xor되므로 각 비트가 반대로 바뀐다(b xor 1 = b의 반대): 1001 0001.

   ③ 4비트씩 16진으로: 1001 = 9, 0001 = 1 → 0x91.

   검산(16진 자리로): 15 − 6 = 9, 15 − E(14) = 1 → 0x91. 모든 비트를 뒤집었으니 NOT과 같다. [Ch2 p.27]

---

8. (다시 풀기, 4번 유형) 8비트 1010 1101을 3칸 sll, srl, sra 한 결과를 각각 2진으로 쓰시오.

   답 보기

   sll 0110 1000, srl 0001 0101, sra 1111 0101

   ① sll 3: (가) 왼쪽 끝 3비트 101을 버린다. (나) 남은 0 1101을 왼쪽으로 붙인다. (다) 오른쪽 빈 3칸에 000 → 0110 1000.

   ② srl 3: (가) 오른쪽 끝 3비트 101을 버린다. (나) 남은 1010 1을 오른쪽으로 붙인다. (다) 왼쪽 빈 3칸에 000 → 0001 0101.

   ③ sra 3: (가)(나)는 srl과 같다. (다) 원래 MSB가 1이므로 왼쪽 빈 3칸을 모두 1로 → 1111 0101.

   검산: 무부호 1010 1101 = 173, 173 ÷ 8 = 21(몫), 0001 0101 = 21. 부호 있는 값 −128 + 45 = −83, −83 ÷ 8 = −10.375를 내림하면 −11, 1111 0101 = −128 + 117 = −11.

---

9. (다시 풀기, 5번 유형) 32비트 0xC00000F0을 8칸 srai 한 결과와 8칸 srli 한 결과를 16진 8자리로 쓰시오.

   답 보기

   srai 0xFFC00000, srli 0x00C00000

   ① 8칸 = 4비트 × 2 = 16진 두 자리 이동. 맨 오른쪽 두 숫자 F0이 버려지고 나머지 C00000이 두 자리 오른쪽으로 간다: 0x??C00000.

   ② 맨 앞 숫자 C = **1**100이므로 MSB = 1.

   ③ srai는 새로 생긴 두 자리(8비트)를 모두 MSB(1)로 채운다. 1111 1111 = FF → 0xFFC00000.

   ④ srli는 0으로 채운다 → 0x00C00000.

---

10. (다시 풀기, 6번 유형) x7 = 12(인덱스 i)일 때 `slli x8, x7, 2` 실행 뒤 x8은? int 배열 A의 시작 주소가 0x1000이면 A[12]의 주소는?

    답 보기

    x8 = 48, A[12]의 주소 = 0x1030

    ⓪ 읽기: "x7을 왼쪽으로 2칸 밀어 x8에 넣어라".

    ① 왼쪽 2칸 = × 4. 12 × 4 = 48. 2진으로 1100 → 11 0000 = 32 + 16 = 48.

    ② 48은 시작 주소에서 A[12]까지의 바이트 수(오프셋)다. int 하나가 4바이트이기 때문이다.

    ③ 주소 = 시작 주소 + 오프셋. 48 = 3 × 16 = 0x30이므로 0x1000 + 0x30 = 0x1030.

더 자세히: 이론 노트 기초 · 4 / Ch2 · 6, 7

기초 · 4 슬라이드 Ch1 p.26 / Ch2 p.9–14, 30, 35

단위와 어셈블리 읽는 법

시험에 나오는 모양: GHz ↔ ps 환산과 10의 거듭제곱 계산(성능 공식), 어셈블리 한 줄을 C 문장으로 읽기, 배열 원소 주소 계산.

먼저 알아 둘 말

- **클럭(clock)**: CPU가 일정한 박자에 맞춰 일하게 하는 신호. 박자 한 번이 **클럭 사이클(clock cycle)**, 그 한 번의 길이(시간)가 **클럭 주기(clock period)**, 1 s에 박자가 몇 번인지가 **클럭 속도(clock rate)**다. 속도 단위 Hz = 1 s에 1번. [Ch1 p.26]
- **C 문장**: 시험의 "C로 쓰시오"는 `A[7] = A[5] - 1;` 같은 한 줄을 말한다. `=` 는 "오른쪽을 계산해 왼쪽에 넣어라", `A[5]` 는 배열 A의 5번 원소(기초 · 3), 끝의 `;` 는 문장의 끝 표시다. PDF 외 보충

외울 것

큰 쪽	값	작은 쪽	값
k(킬로)	10^3	m(밀리)	10^{-3}
M(메가)	10^6	μ(마이크로)	10^{-6}
G(기가)	10^9	n(나노)	10^{-9}
		p(피코)	10^{-12}

- **음수 지수와 0 지수**: $10^{-k} = 1 / 10^k$(예 $10^{-3} = 1 / 1000 = 0.001$), $10^0 = 1$. 기초 · 1의 $2^{-1} = 0.5$와 같은 규칙이다. 그래서 1 ns = 10^{-9} s = 1000 × 10^{-12} s = **1000 ps**, 1 s = 1000 ms다.
- **클럭 주기 = 1 / 클럭 속도**. 4 GHz → 1 / (4 × 10^9) s = 0.25 ns = 250 ps. [Ch1 p.26] 이유: 1 s에 박자가 f번이면 1 s를 f조각으로 나눈 것이 박자 하나의 길이다(1 s에 4번이면 한 번에 1/4 s).
- **지나간 클럭 사이클 수 = 시간 × 클럭 속도**. 1 s에 f번 박자가 오면 t s 동안에는 f × t번 온다(1 s에 10번이면 3 s에 30번). 같은 말로 시간 ÷ 클럭 주기다.
- 빠른 환산: 주기(ps) = 1000 / 속도(GHz 숫자). 2 GHz → 500 ps, 500 ps → 1000 / 500 = 2 GHz. 이유: 속도가 f GHz면 주기 = 1 / (f × 10^9) s = (1 / f) × 10^{-9} s = (1 / f) ns이고, 1 ns = 1000 ps라 (1000 / f) ps다.
- 지수 계산: $10^a \times 10^b = 10^{a+b}$, $10^a / 10^b = 10^{a-b}$. 앞의 숫자끼리 따로, 지수끼리 따로 계산한다. a, b가 음수여도 같다: $10^{-3} \times 10^9 = 10^6$, $1 / 10^{-12} = 10^{0-(-12)} = 10^{12}$.
- 한국어 큰 수: 1만 = 10^4, 1억 = 10^8. 예 6억 = 6 × 10^8, 600만 = 6 × 10^6.
- 바이트 = 8비트, **word = 32비트 = 4바이트**, doubleword = 64비트 = 8바이트. [Ch2 p.10] KiB(2^{10} = 1024)와 KB(10^3)는 섞지 않는다.

어셈블리	뜻
`add x5, x6, x7`	x5 ← x6 + x7. 꼴 `add rd, rs1, rs2` 에서 **맨 앞 rd = 결과를 받는 레지스터(destination)**, rs1·rs2 = 값을 읽는 레지스터(source). [Ch2 p.9]
`sub x5, x6, x7`	x5 ← x6 − x7 (순서 그대로 뺀다)
`addi x5, x6, -3`	x5 ← x6 + (−3). 즉시값(immediate)은 상수
`lw x9, 32(x22)`	주소 x22 + 32에서 word를 읽어 x9에 넣는다. [Ch2 p.14]
`sw x9, 48(x22)`	x9를 주소 x22 + 48에 쓴다(x9는 그대로)
`beq x5, x6, L`	x5 == x6이면 L로 건너뛰고, 아니면 다음 줄. 이렇게 조건에 따라 다른 줄로 건너뛰는 것을 **분기(branch)**라 한다. [Ch2 p.28] bne = 같지 않으면, blt = 작으면, bge = 크거나 같으면 L로
`Loop:`	라벨(label). 명령어 위치의 이름, 분기 목적지로 쓴다
`// …`	주석. 실행되지 않는다

- **PC(Program Counter)**: 실행할 명령어의 주소를 담는 레지스터. 명령어 하나가 4바이트라 보통 다음은 PC + 4. [Ch2 p.35]
- **배열 주소**: int 하나 = word = 4바이트라 int A[i]의 주소 = 시작 주소 + i × 4. `lw` · `sw` 의 괄호 앞 숫자(32, 48)가 이 i × 4, 즉 오프셋이다. 연속된 word의 주소는 4씩 차이 난다. [Ch2 p.12, 14]

푸는 순서

1. 단위 문제: 모든 값을 10의 거듭제곱 꼴로 바꾼 뒤 숫자와 지수를 따로 계산하고, 마지막에 원하는 단위로 바꾼다.
2. 어셈블리: rd(맨 앞)를 먼저 표시하고 오른쪽을 계산한다.
3. lw·sw: 괄호 안 레지스터 값 + 앞의 숫자 = 주소. 배열이면 (앞의 숫자) ÷ 4 = 인덱스.

함정

- `lw x9, 32(x22)` 는 A[32]가 아니라 A[8]이다. 32는 바이트 수라 4로 나눠야 인덱스가 된다.
- sub의 소스 순서를 바꾸면 부호가 바뀐다. `sub x5, x7, x6` 은 x7 − x6이다.
- GHz는 1 s에 10^9번, 1 ns = 10^{-9} s. 주기와 속도는 역수라 한쪽이 2배면 다른 쪽은 절반이다.

예제 (시험 수준)

int 배열 A의 시작 주소는 x23에 있고 x23 = 0x00000200이다. 아래 코드가 읽고 쓰는 주소를 쓰고, C 문장 하나로 쓰시오.

```
lw   x9, 20(x23)     // ?
addi x9, x9, -1
sw   x9, 28(x23)
```

풀이 보기

읽는 주소 0x00000214, 쓰는 주소 0x0000021C, C 문장 `A[7] = A[5] - 1;`

① lw의 주소: `20(x23)`은 "x23에 든 주소에서 20바이트 더 간 곳"이다. 괄호 앞 숫자 **20이 오프셋**(시작 주소에서 떨어진 바이트 수)이다. 주소 = 0x200 + 20. 20 = 1 × 16 + 4 = 0x14이므로 0x200 + 0x14 = **0x214**.

①′ 몇 번 원소인가: 오프셋 = 인덱스 × 4(int 하나가 4바이트)이므로 거꾸로 **인덱스 = 오프셋 ÷ 4**다. 오프셋 20은 명령어 괄호 앞에 그대로 적혀 있다(주소만 주어졌다면 0x214 − 0x200 = 0x14 = 20으로 구한다). 20 ÷ 4 = 5 → A[5]. 그래서 x9 = A[5]. 확인: A[5]는 A[0](0x200~0x203), A[1](0x204~), …를 지나 0x200 + 5 × 4 = 0x214에서 시작한다.

② addi: 목적지 x9 ← x9 + (−1) = A[5] − 1. 즉시값 −1은 상수다.

③ sw: 주소 = 0x200 + 28. 28 = 1 × 16 + 12 = 0x1C이므로 0x21C. 28 ÷ 4 = 7이므로 A[7]에 x9를 쓴다.

④ 합치면 `A[7] = A[5] - 1;`

회독 점검

1. 클럭 속도 2.5 GHz인 CPU의 클럭 주기는 몇 ps인가?

답 보기

400 ps

① 주기 = 1 / 속도 = 1 / (2.5×10^{9}) s.

② 1 / 2.5 = 0.4이므로 0.4×10^{-9} s = 0.4 ns.

③ 1 ns = 1000 ps이므로 0.4 ns = 400 ps.

검산(빠른 환산): 1000 / 2.5 = 400.

---

2. 클럭 주기가 800 ps인 CPU의 클럭 속도는 몇 GHz인가?

답 보기

1.25 GHz

① 속도 = 1 / 주기 = 1 / (800×10^{-12}) Hz.

② 숫자와 지수를 나눈다: 1 / 800 = 0.00125 = 1.25×10^{-3}.

③ 지수 쪽은 $1 / 10^{-12} = 10^{0} / 10^{-12}$. 외울 것의 $10^{a} / 10^{b} = 10^{a-b}$에 a = 0, b = −12를 넣으면 $10^{0 - (-12)} = 10^{12}$.

④ 합치면 $1.25 \times 10^{-3} \times 10^{12} = 1.25 \times 10^{9}$ Hz. 10^{9} Hz = 1 GHz이므로 1.25 GHz.

검산(빠른 환산): 1000 / 800 = 1.25.

---

3. 클럭 속도 2 GHz인 CPU에서 3 ms 동안 지나가는 클럭 사이클 수를 10의 거듭제곱 꼴과 "만" 단위로 쓰시오.

답 보기

6×10^{6}번 = 600만 번

① 단위를 10의 거듭제곱으로: 3 ms = 3×10^{-3} s, 2 GHz = 1 s에 2×10^{9}번.

② 사이클 수 = 시간 × 클럭 속도(1 s에 몇 번 × 몇 s).

③ 숫자끼리 3 × 2 = 6, 지수끼리 $10^{-3} \times 10^{9} = 10^{-3 + 9} = 10^{6}$ → 6×10^{6}.

④ 1만 = 10^{4}이므로 $6 \times 10^{6} = 600 \times 10^{4}$ = 600만.

---

4. x6 = 10, x7 = 4일 때 `sub x5, x7, x6` 실행 뒤 x5는? 이 명령을 C 문장 꼴로 쓰시오.

   답 보기

   x5 = −6, `x5 = x7 - x6;`

   ① 맨 앞 x5가 목적지, 계산은 두 번째 − 세 번째 순서.

   ② x7 − x6 = 4 − 10 = −6.

5. x22 = 0x00001000이고 x22가 int 배열 A의 시작 주소다. `lw x9, 24(x22)`는 몇 번지를 읽고, C로는 무엇인가?

   답 보기

   0x00001018, x9 = A[6];

   ① 주소 = x22 + 24. 24 = 1 × 16 + 8 = 0x18이므로 0x1000 + 0x18 = 0x1018.

   ② int는 4바이트라 인덱스 = 24 ÷ 4 = 6. 그래서 A[6].

6. int 배열 A의 시작 주소가 0x00002000이다. A[10]의 주소를 16진 8자리로 쓰시오. 주소 0x00002034에 있는 원소는 A의 몇 번 원소인가?

   답 보기

   A[10]은 0x00002028, 0x00002034는 A[13]

   ① A[10]: 시작 + 10 × 4 = 0x2000 + 40. 40 = 2 × 16 + 8 = 0x28 → 0x00002028.

   ② 거꾸로: 0x2034 − 0x2000 = 0x34 = 3 × 16 + 4 = 52바이트.

   ③ int는 4바이트라 인덱스 = 52 ÷ 4 = 13 → A[13].

더 자세히: 이론 노트 기초 · 5, 7 / Ch1 · 8 / Ch2 · 3

Ch1 · 1 슬라이드 p.3–34

Ch1 외울 문장

시험에 나오는 모양: T/F, 빈칸, 나열. (기출: 21-2 "성능을 결정하는 3가지"·"프로세서를 구성하는 두 가지", 26-1 빈칸 "캐시 메모리", 21-2·26-1 T/F "임베디드" 등 Ch1 한두 줄)

외울 것

분류와 정의

용어	외울 문장	근거
개인용 컴퓨터(PC)	범용, 다양한 SW, 비용·성능 균형	[Ch1 p.3]
서버(Server)	보통 네트워크로 접근, 대용량·고성능·고신뢰성	[Ch1 p.3]
임베디드(Embedded)	다른 기기 안에서 미리 정해진 응용 실행, 전력·성능·비용 제약이 엄격	[Ch1 p.3]
컴퓨터 구조	ISA + 조직(Machine Organization)	[Ch1 p.9]
명령어 집합 구조(ISA, Instruction Set Architecture)	하드웨어와 최하위 소프트웨어 사이의 추상 인터페이스 = 아키텍처(architecture)	[Ch1 p.17]
ABI	ISA + OS 인터페이스 (Application Binary Interface)	[Ch1 p.17]
구현(Implementation)	아키텍처를 따르는 하드웨어	[Ch1 p.17]

- (하) 개인 휴대 기기(PMD, Personal Mobile Device) = 배터리·무선 인터넷. 클라우드 = 창고 규모 컴퓨터(WSC, Warehouse Scale Computer). 서비스형 소프트웨어(SaaS, Software as a Service) = 클라우드로 배포되는 SW. ECU = 자동차 전장의 임베디드. [Ch1 p.4, 7]

프로그램 아래 계층과 언어 [Ch1 p.13–14]

- 계층: 응용 소프트웨어 → 시스템 소프트웨어(OS, 컴파일러) → 하드웨어. [Ch1 p.13]
- 운영체제(OS) 역할 3 = 기본 입출력 처리, 저장공간·메모리 할당, 자원 공유·작업 스케줄링. [Ch1 p.13]
- 고급 언어 장점 = 생산성(productivity)·이식성(portability). 컴파일러 = 고급 언어(C 등) 프로그램을 하드웨어가 실행하는 명령어로 번역. [Ch1 p.13] 어셈블리어 = 기계어 명령의 기호 표현. 어셈블러 = 기호 버전 → 2진 버전으로 번역. [Ch1 p.14]

8대 위대한 아이디어 [Ch1 p.12]

#	한국어	영어	한 줄
1	무어의 법칙을 고려한 설계	Design for Moore's Law	집적회로 자원이 18~24개월마다 2배
2	추상화로 설계 단순화	Use abstraction to simplify design	여러 수준을 추상화로 표현
3	자주 생기는 경우를 빠르게	Make the common case fast	드문 경우 최적화보다 효과가 큼
4	병렬성을 통한 성능	Performance via parallelism	동시에 계산
5	파이프라이닝을 통한 성능	Performance via pipelining	병렬성의 특정 형태
6	예측을 통한 성능	Performance via prediction	기다리기보다 추측해 시작하는 게 평균적으로 빠름
7	메모리 계층	Hierarchy of memories	크기와 속도의 충돌을 계층으로 해결
8	중복을 통한 신뢰성	Dependability via redundancy	여분 부품으로 믿을 만하게

구성 요소와 메모리

- 5대 구성 요소 = 입력, 출력, 메모리, 데이터패스, 제어. 모든 컴퓨터에 같다. [Ch1 p.15]
- **프로세서 = 데이터패스(산술 연산 수행) + 제어(데이터패스·메모리·입출력에 언제 무엇을 할지 동작 신호를 보냄).** 메모리·입력·출력은 프로세서 밖이다. [Ch1 p.15–16]
- 입력은 메모리에 **쓰고**, 출력은 메모리에서 **읽는다**. [Ch1 p.15]
- DRAM(동적 RAM, Dynamic RAM) = 주기억. **캐시 메모리 = DRAM의 버퍼인 작고 빠른 메모리**, SRAM(정적 RAM, Static RAM. 빠르지만 집적도가 낮아 비쌈)으로 만든다. [Ch1 p.17]
- 휘발성 = 주(1차) 기억(DRAM·SRAM). 비휘발성 = 2차 기억(디스크·플래시), 더 크고 느림. [Ch1 p.18]
- (하) 네트워크 장점 = 통신·자원 공유·원격 접근(사용자가 컴퓨터 가까이 있을 필요가 **없다(NOT)**). LAN = 한 건물 안(예 이더넷), WAN = 수백 km~대륙(예 인터넷). 실리콘은 전기가 잘 통하지 **않아(NOT)** 반도체. [Ch1 p.19, 21]

칩 비용 [Ch1 p.22]

웨이퍼(wafer) = 칩을 찍어 내는 둥근 실리콘 판, 다이(die) = 웨이퍼에서 잘라 낸 칩 한 개, 수율(yield) = 만든 다이 중 정상인 것의 비율. [Ch1 p.21–22]

```
다이당 비용 = 웨이퍼당 비용 / (웨이퍼당 다이 수 × 수율)
```

```
수율 = 1 / (1 + 면적당 결함 수 × 다이 면적 / 2)^N
```

- 지수: 슬라이드 N 슬라이드 확인 필요, 교재 예제 N = 2 PDF 외 보충. 결함률 = 제조 공정, 다이 면적 = 구조·회로 설계. [Ch1 p.22]

성능

- 응답 시간(response time, = 실행 시간) = 작업 하나의 시작부터 끝까지 걸린 시간. 처리량(throughput, = 대역폭) = 정해진 시간에 해낸 일의 총량. 프로세서 **교체** → 둘 다 개선, **추가** → 처리량만. [Ch1 p.24]
- 경과 시간(wall clock) = I/O·OS 오버헤드까지 모두 포함. CPU 시간은 I/O·다른 프로그램 시간을 포함하지 **않는다(NOT)**. CPU 시간 = 사용자(user) CPU 시간 + 시스템(system) CPU 시간. [Ch1 p.25]
- 성능 개선 = 클럭 사이클 수 줄이기 또는 클럭 주기 줄이기(클럭 속도 올리기). 둘은 트레이드오프. [Ch1 p.27]
- **성능을 결정하는 3요소 = 명령어 수(IC), CPI, 클럭 주기.** IC(Instruction Count) = 실행되는 명령어 개수, CPI = 명령어 하나에 드는 평균 클럭 사이클 수(계산은 Ch1 · 2). 세 요소를 곱하면 CPU 시간이 되므로(명령어 수 × 명령어당 사이클 × 사이클 하나의 길이) 셋 중 하나만 줄어도 시간이 준다. [Ch1 p.28]
  누가 정하나는 슬라이드 두 곳이 조금 다르다. **Ch4 p.2: IC = ISA·컴파일러, 클럭 주기 = CPU 하드웨어, CPI = CPU 하드웨어.** 여기서 CPU 하드웨어 = 마이크로아키텍처(내부 회로 설계)·공정 기술 등(예 Intel과 AMD의 차이). [Ch4 p.2] Ch1 p.28은 IC를 정하는 것에 **프로그램**을 더 넣고(프로그램·ISA·컴파일러), 클럭 주기는 말하지 않는다. 문제가 "Ch4 기준"이면 앞의 것, 출처를 안 밝히면 IC에 프로그램까지 쓰면 안전하다. 상세 표는 Ch1 · 2. [Ch1 p.28]

에너지·전력과 MIPS

- 동적 에너지 ∝ 용량성 부하 × 전압^2 (전력은 여기에 × 주파수(= 클럭 속도), 계산은 Ch1 · 2). [Ch1 p.30]
- 전력 장벽 ① 전압을 더 낮출 수 **없다(NOT)**: 누설(leakage). [Ch1 p.31]
- 전력 장벽 ② 열을 더 제거할 수 **없다(NOT)**: 냉각 비용(cooling cost). [Ch1 p.31]
- 2006년부터 멀티코어(칩 하나에 프로세서 코어 여러 개), 이득은 응답 시간보다 처리량. [Ch1 p.32]
- MIPS(초당 백만 명령어 수, 계산은 Ch1 · 2)가 성능 지표로 부적절한 이유 ① ISA 차이 ② 명령어 복잡도 차이를 반영 못 함 ③ 같은 CPU도 프로그램마다 CPI가 다름. [Ch1 p.34]

함정

- "프로세서를 추가하면 응답 시간도 준다"는 X. 추가는 처리량만. [Ch1 p.24]
- "입력은 메모리에서 읽는다", "SRAM이 더 싸다", "메모리도 프로세서에 포함"은 X.

회독 점검

1. O/X: 임베디드 컴퓨터는 범용으로 다양한 소프트웨어를 실행하며 비용과 성능의 균형이 가장 중요하다.

답 보기

X

① 문장은 개인용 컴퓨터(PC)의 정의다.

② 임베디드 = 다른 기기 안에 들어가 미리 정해진 응용을 실행하고, 전력·성능·비용 제약이 엄격한 컴퓨터. [Ch1 p.3]

---

2. 빈칸: (　　)는 DRAM의 버퍼 역할을 하는 작고 빠른 메모리이며, (　　)으로 만든다.

답 보기

캐시 메모리, SRAM

① DRAM은 주기억장치를 만드는 칩이다.

② 캐시는 DRAM 앞에서 버퍼 역할을 하고, 더 빠르지만 집적도가 낮아 비싼 SRAM으로 만든다. [Ch1 p.17]

---

3. 컴퓨터의 5대 구성 요소 중 프로세서를 이루는 두 가지를 쓰고, 각각 하는 일을 한 줄로 쓰시오.

답 보기

데이터패스(산술 연산 수행), 제어(데이터패스·메모리·입출력에 동작 신호를 보냄)

① 5대 구성 요소 = 입력, 출력, 메모리, 데이터패스, 제어.

② 그중 프로세서 = 데이터패스 + 제어. 메모리는 프로세서 밖이다. [Ch1 p.15–16]

---

4. 프로그램의 CPU 실행 시간을 결정하는 세 요소를 쓰고, 각각 무엇이 정하는지 Ch4 슬라이드 기준으로 쓰시오.

답 보기

명령어 수 = ISA·컴파일러, 클럭 주기 = CPU 하드웨어, CPI = CPU 하드웨어

① CPU 시간 = 명령어 수 × CPI × 클럭 주기이므로 세 요소가 곱으로 들어간다. [Ch1 p.28]

② Ch4 p.2는 CPU 하드웨어를 마이크로아키텍처·공정 기술이라고 풀어 쓴다. Ch1 p.28은 명령어 수에 "프로그램"도 넣는다. [Ch4 p.2]

---

5. O/X: CPU 시간은 프로그램이 디스크 입출력을 기다린 시간까지 포함한 전체 시간이다.

   답 보기

   X

   ① 그 설명은 경과 시간(wall clock time, 응답 시간)이다.

   ② CPU 시간은 CPU가 그 작업에 쓴 시간만이며 I/O 시간·다른 프로그램 시간은 포함하지 않는다(NOT). [Ch1 p.25]

6. 8대 아이디어 중 영어 이름이 "Performance via ~"인 것 세 가지를 한국어로 쓰고, 그중 "병렬성의 특정 형태"는 무엇인가?

   답 보기

   병렬성·파이프라이닝·예측을 통한 성능. 병렬성의 특정 형태 = 파이프라이닝

   ① Performance via parallelism / pipelining / prediction.

   ② 슬라이드는 파이프라이닝을 병렬성의 특정 형태라고 적는다. [Ch1 p.12]

더 자세히: 이론 노트 Ch1 · 1~10 / 실전 1, 8

Ch1 · 2 슬라이드 p.24, 26–29, 31, 33–34

성능 계산

시험에 나오는 모양: CPI·클럭을 주고 MIPS 계산(단위 주의), 실행 명령어·시간 표를 주고 개선율 계산. (기출: 21-2·26-1 MIPS 계산, 26-1 개선율)

외울 것

- 클럭 사이클(clock cycle) = 클럭이 한 번 똑딱하는 단위. 그 길이가 클럭 주기. [Ch1 p.26]
- CPI(Cycles Per Instruction) = 명령어 하나에 드는 평균 클럭 사이클 수. [Ch1 p.28]
- MIPS(Million Instructions Per Second) = 초당 백만 명령어 수. [Ch1 p.34]
- 명령어 종류별로 CPI가 다르다. 문제에 나오는 종류: 산술(add 등), load/store(메모리 읽기·쓰기 명령어, lw·sw), 분기(beq 등). 캐시(cache)는 메모리 앞의 작고 빠른 메모리라(Ch1 · 1) 개선하면 load/store가 빨라진다. [Ch1 p.17, 28]

CPU 시간 = 명령어 수(IC) × CPI × 클럭 주기 = IC × CPI / 클럭 속도 [Ch1 p.28]

사이클 수 = 실행 시간 × 클럭 속도 = IC × CPI [Ch1 p.27-28]

클럭 속도 = 1 / 클럭 주기 (250 ps ↔ 4 GHz, 0.5 ns ↔ 2 GHz) [Ch1 p.26]

성능 = 1 / 실행 시간, "X가 n배 빠르다" ⇔ n = 실행 시간$_Y$ / 실행 시간$_X$ [Ch1 p.24]

평균 CPI = Σ(CPI$_i$ × 개수$_i$) / 전체 명령어 수 (가중 평균) [Ch1 p.28]

MIPS = 클럭 속도 / (CPI × 10^6) = IC / (실행 시간 × 10^6) [Ch1 p.34]

암달: $T_{개선\ 후} = T_{영향} / n + T_{비영향}$, 최대 속도 향상 = $T_{전} / T_{비영향}$ [Ch1 p.33]

전력 ∝ 용량성 부하 × 전압^2 × 주파수 (비율끼리 곱한다) [Ch1 p.31]

- 암달의 법칙(Amdahl's Law): 일부만 빨라지면 전체 개선은 빨라지지 않는 나머지 부분에 묶인다. $T_{영향}$ = 개선이 적용되는 부분의 시간, $T_{비영향}$ = 그대로 남는 시간, n = 영향 부분이 빨라진 배수(improvement factor). 각 항의 이유: 영향 부분은 n배 빨라졌으니 걸리는 시간이 1/n로 줄어 $T_{영향}$ / n, 나머지는 손대지 않았으니 $T_{비영향}$ 그대로, 두 시간을 더하면 새 전체 시간이다. 예: 10 s 중 6 s를 3배 빠르게 → 6 / 3 + 4 = 6 s. 최대 속도 향상: n을 아무리 키워도 $T_{영향}$ / n은 0에 가까워질 뿐이라 시간은 $T_{비영향}$ 밑으로 못 내려간다. 그래서 최대 = $T_{전}$ / $T_{비영향}$(위 예는 10 / 4 = 2.5배). [Ch1 p.33]
- 슬라이드 암달: 100 s 중 곱셈 80 s, 전체를 2.5배 빠르게. 목표 = 100 ÷ 2.5 = 40 s, 비영향 = 100 − 80 = 20 s. 40 = 80/n + 20 → 80/n = 20 → **n = 4**. 곱셈 외만 개선 → 80 s가 남아 최대 100 / 80 = 1.25배, **불가능**. [Ch1 p.33]

- 교재 전력 예제: 부하·전압·주파수 셋 다 0.85배. 전압만 제곱이라 $0.85 \times 0.85^2 \times 0.85 = 0.85^4 \approx 0.52$배. 클럭 1000배에 전력 30배뿐인 이유 = 전압 5 V → 1 V(세대당 약 15%). [Ch1 p.31]

"n배 빠르다"의 방향. 실행 시간이 짧을수록 좋은 컴퓨터이므로 성능은 시간의 역수(1 / 시간)로 정한다. 그러면 $\text{성능}_X / \text{성능}_Y = (1 / \text{시간}_X) / (1 / \text{시간}_Y) = \text{시간}_Y / \text{시간}_X$가 되어, **느린 쪽 시간을 빠른 쪽 시간으로 나눈다**(답이 1보다 크게). 예: X 10 s, Y 15 s → X가 15 / 10 = 1.5배 빠르다. 속도 향상(speedup) = 개선 전 시간 / 개선 후 시간도 같은 방향이다. [Ch1 p.24]

평균 CPI가 가중 평균인 이유. 평균 CPI는 "전체 사이클 수 ÷ 전체 명령어 수"다. 종류 i의 명령어가 개수_i개이고 하나에 CPI_i 사이클이 들면 그 종류가 쓴 사이클은 $\text{CPI}_i \times \text{개수}_i$이고, 모든 종류를 더한 것이 전체 사이클 수다. 그래서 많이 실행되는 종류가 평균에 더 크게 반영된다. 작은 예: CPI 1인 명령어 3개, CPI 5인 명령어 1개 → 사이클 = 3 × 1 + 1 × 5 = 8, 평균 CPI = 8 / 4 = 2. CPI 값만 평균 낸 (1 + 5) / 2 = 3은 개수를 무시해 틀린다. [Ch1 p.28]

MIPS 공식 두 개가 같은 이유. MIPS의 뜻 그대로 쓰면 "1 s에 실행한 명령어 수 ÷ 10^6(백만)" = IC / (실행 시간 × 10^6)이다. 여기에 실행 시간 = IC × CPI / 클럭 속도를 넣으면 IC / (IC × CPI / 클럭 속도 × 10^6) = 클럭 속도 / (CPI × 10^6)이 된다(IC가 위아래에서 약분). 그래서 IC와 시간을 주면 앞 식, 클럭과 CPI를 주면 뒤 식을 쓰고, 하나로 구한 답을 다른 식으로 검산한다. 클럭 속도는 Hz(1 s에 몇 번)로 넣는다. [Ch1 p.34]

전력 문제에서 비율끼리 곱하는 이유. 새 전력 / 옛 전력 = (부하' × 전압'^{2} × 주파수') / (부하 × 전압^2 × 주파수) = (부하'/부하) × (전압'/전압)^2 × (주파수'/주파수). $\propto$(비례)에 숨은 상수는 위아래에서 약분되어 사라지므로, 실제 값을 몰라도 "몇 배"만으로 답이 나온다. 전압만 제곱인 것은 에너지 $\propto$ 부하 × 전압^{2}이고 전력 = 에너지 × 주파수이기 때문이다. 예: 전압만 0.5배 → 0.5^2 = 0.25배. [Ch1 p.30–31]

요소	누가 정하나 [Ch1 p.28, Ch4 p.2]	영향 표 [Ch1 p.29]
명령어 수(IC)	프로그램(p.28만), ISA, 컴파일러	알고리즘·언어·컴파일러·ISA
CPI (평균)	CPU 하드웨어	알고리즘·언어·컴파일러·ISA
클럭 주기 (속도)	CPU 하드웨어(마이크로아키텍처, 공정 기술). p.28은 언급 없음	ISA·기술(Technology) 슬라이드 확인 필요

단위(ns = 10^{-9} s, GHz = 10^9 Hz 등)는 기초 · 4 단위표.

푸는 순서

1. 값을 s와 Hz로 바꾼다(6억 = 6×10^8, 500 MHz = $500 \times 10^6 = 5 \times 10^2 \times 10^6 = 5 \times 10^8$ Hz, 3 ms = 3×10^{-3} s). 앞 숫자를 1~10 사이로 맞추면 지수 계산이 쉽다. 두 컴퓨터를 비교할 때 같은 프로그램·같은 ISA(같은 컴파일러)면 IC가 같으므로 명령어 하나당 시간 = CPI / 클럭 속도(= CPI × 클럭 주기)만 비교하면 된다.

2. 사이클 수 = Σ(개수 × CPI), 평균 CPI = 사이클 수 / IC, CPU 시간 = 사이클 수 / 클럭 속도.

3. MIPS는 두 식으로 구해 검산한다.

4. 개선 문제: 영향·비영향으로 나눠 암달 식에 넣고 속도 향상 = $T_{전}$ / $T_{후}$. 목표 시간이 $T_{비영향}$ 이하면 불가능.

함정

- MIPS 분모의 10^6을 빠뜨리거나, 500 MHz를 500으로 넣고 10^6까지 나누면 백만 배 틀린다.
- 평균 CPI는 개수(비율)로 가중한다. CPI 값만 단순 평균하면 틀린다. 명령어 수가 많은 코드도 CPI가 낮으면 더 빠를 수 있다(사이클 수 = IC × CPI로 비교). 교재 기준 [Ch1 p.28 → 교재 pp.36–37]
- 암달에서 n은 영향 받는 부분에만 나눈다. 전력에서 전압은 제곱, 부하·주파수는 1제곱.

예제 (시험 수준)

클럭 2 GHz인 컴퓨터에서 프로그램이 다음 명령어를 실행한다. 산술 40×10^6개(CPI 1), load/store 30×10^6개(CPI 5), 분기 30×10^6개(CPI 2). (1) 평균 CPI (2) CPU 시간 (3) MIPS (4) 캐시를 개선해 load/store의 CPI가 2가 되면 속도 향상은?

풀이 보기

(1) 2.5 (2) 0.125 s (3) 800 MIPS (4) 약 1.56배

① 단위: 2 GHz = 2×10^9 Hz. 개수는 모두 $\times 10^6$이니 앞의 수만 계산하고 10^6은 붙여 둔다. 전체 명령어 수 = 40 + 30 + 30 = 100 → 10^8개.

② 사이클 수 = 40 × 1 + 30 × 5 + 30 × 2 = 40 + 150 + 60 = 250 → 250×10^6 사이클.

③ (1) 평균 CPI = 250×10^6 / (100×10^6) = 2.5. 개수로 가중했으므로 (1 + 5 + 2) / 3 ≈ 2.67이 아니다.

④ (2) CPU 시간 = 사이클 수 / 클럭 속도 = 250×10^6 / (2×10^9). 250 ÷ 2 = 125, $10^{6-9} = 10^{-3}$ → $125 \times 10^{-3} = 0.125$ s.

⑤ (3) MIPS = 클럭 속도 / (CPI × 10^6) = 2×10^9 / (2.5×10^6) = $0.8 \times 10^3 = 800$. 검산: IC / (시간 × 10^6) = 10^8 / (0.125×10^6) = 10^8 / 125,000 = 800. 일치.

⑥ (4) 개선 후 사이클 = 40 × 1 + 30 × 2 + 30 × 2 = 40 + 60 + 60 = 160 → 160×10^6. 시간 = 160×10^6 / (2×10^9) = 0.08 s.

⑦ 속도 향상 = $T_{전} / T_{후}$ = 0.125 / 0.08 = 1.5625 ≈ 1.56배(성능 약 56% 향상, 시간 36% 감소).

검산(암달): load/store가 쓰던 시간 = 150×10^6 / (2×10^9) = 0.075 s, 나머지 = 0.125 − 0.075 = 0.05 s. CPI 5 → 2는 그 부분이 5/2 = 2.5배 빨라진 것이므로 0.075 / 2.5 + 0.05 = 0.03 + 0.05 = 0.08 s. 일치.

[Ch1 p.28, 33]

회독 점검

1. 클럭 주기가 0.4 ns이고 CPI가 1.25인 컴퓨터가 있다. (1) MIPS는? (2) 명령어 5×10^9개를 실행하는 시간은?

   답 보기

   (1) 2000 MIPS (2) 2.5 s

   ① 주기 → 속도: $1 / (0.4 \times 10^{-9}) = 2.5 \times 10^9$ Hz(1 ÷ 0.4 = 2.5, 10^{-9}으로 나누면 10^9을 곱함).

   ② MIPS = $2.5 \times 10^9 / (1.25 \times 10^6) = 2 \times 10^3 = 2000$.

   ③ 시간 = IC × CPI × 주기 = $5 \times 10^9 \times 1.25 \times 0.4 \times 10^{-9}$. $10^9 \times 10^{-9} = 1$, $5 \times 1.25 \times 0.4 = 2.5$ → 2.5 s.

   검산: $5 \times 10^9 / (2.5 \times 10^6) = 2000$. 일치.

---

2. 클럭 800 MHz인 컴퓨터가 4억 개의 명령어를 0.8 s에 실행했다. MIPS와 평균 CPI를 구하시오.

   답 보기

   500 MIPS, CPI 1.6

   ① 단위: 4억 = 4×10^8(1억 = 10^8). 800 MHz = $800 \times 10^6 = 8 \times 10^2 \times 10^6 = 8 \times 10^8$ Hz.

   ② 명령어 수와 시간이 있으니 MIPS = IC / (시간 × 10^6) = $4 \times 10^8 / (0.8 \times 10^6)$. 숫자 4 / 0.8 = 5, 지수 $10^{8-6} = 10^2$ → $5 \times 10^2 = 500$.

   ③ CPI: 사이클 수 = 시간 × 클럭 속도 = $0.8 \times 8 \times 10^8 = 6.4 \times 10^8$. 평균 CPI = 사이클 수 / IC = $6.4 \times 10^8 / (4 \times 10^8) = 1.6$.

   검산(MIPS 다른 식): 클럭 속도 / (CPI × 10^6) = $8 \times 10^8 / (1.6 \times 10^6) = 5 \times 10^2 = 500$. 일치.

---

3. 같은 ISA인 A(3 GHz, CPI 1.5)와 B(2 GHz, CPI 0.8)가 같은 프로그램을 실행한다. 어느 쪽이 몇 배 빠른가?

답 보기

B가 A보다 1.25배 빠르다

① 같은 프로그램을 같은 ISA로 실행하니(같은 컴파일러로 번역했다고 본다) 명령어 수 IC가 같다. CPU 시간 = IC × CPI / 클럭 속도에서 IC가 양쪽에 똑같이 곱해지므로, 비율을 구할 때 약분되어 명령어 하나당 시간 = CPI / 클럭 속도만 비교하면 된다.

② A: $1.5 / (3 \times 10^9) = 0.5 \times 10^{-9}$ s = 0.5 ns. B: $0.8 / (2 \times 10^9) = 0.4$ ns.

③ n = 느린 쪽 시간 / 빠른 쪽 시간 = 0.5 / 0.4 = 1.25. 클럭이 낮은 B가 CPI가 작아 이긴다. [Ch1 p.24, 28]

---

4. 200 s 걸리는 프로그램 중 메모리 접근이 120 s다. (1) 전체를 2배 빠르게 하려면 메모리 접근을 몇 배 개선해야 하는가? (2) 메모리 접근 외 부분만 개선해서 2배가 가능한가?

답 보기

(1) 6배 (2) 불가능(최대 약 1.67배)

① 목표 시간 = 200 / 2 = 100 s. 비영향 = 200 − 120 = 80 s.

② 100 = 120 / n + 80 → 120 / n = 20 → n = 6.

③ (2) 이번엔 120 s가 그대로 남는다. 나머지 80 s를 0 s로 만들어도 120 s → 최대 200 / 120 ≈ 1.67배 < 2. 목표 100 s가 120 s보다 작으므로 불가능.

검산: 120 / 6 + 80 = 100 s, 200 / 100 = 2배. [Ch1 p.33]

---

5. 새 프로세서는 용량성 부하가 1.1배, 전압이 0.8배, 주파수가 1.25배다. 동적 전력은 기존의 몇 배인가?

답 보기

0.88배

① 전력 $\propto$ 부하 × 전압^2 × 주파수. 새 전력 / 옛 전력을 구하면 비례 상수가 약분되어 (부하 비율) × (전압 비율)^2 × (주파수 비율)만 남으므로 비율끼리 곱한다.

② 전압만 제곱: $0.8^2 = 0.64$.

③ 1.1 × 0.64 × 1.25 = 1.1 × 0.8 = 0.88. 주파수를 올려도 전압을 낮추면 전력이 줄 수 있다. [Ch1 p.31]

---

6. 어떤 프로그램이 클럭 2.5 GHz인 컴퓨터 A에서 12 s 걸린다. 컴퓨터 B는 클럭을 올리는 대신 이 프로그램에 A보다 1.25배 많은 클럭 사이클이 필요하다. B에서 7.5 s에 끝내려면 B의 클럭 속도는 얼마여야 하는가?

   답 보기

   5 GHz

   ① A의 사이클 수 = 실행 시간 × 클럭 속도 = $12 \times 2.5 \times 10^9 = 30 \times 10^9$ 사이클. 클럭 속도는 1 s에 도는 사이클 수이므로 시간을 곱하면 사이클 수가 된다.

   ② B의 사이클 수 = $1.25 \times 30 \times 10^9 = 37.5 \times 10^9$ 사이클.

   ③ B의 클럭 속도 = 사이클 수 / 목표 시간 = $37.5 \times 10^9 / 7.5 = 5 \times 10^9$ Hz = 5 GHz.

   검산: B의 시간 = $37.5 \times 10^9 / (5 \times 10^9) = 7.5$ s. 클럭은 2배인데 사이클이 1.25배 늘어 12 / 7.5 = 1.6배만 빨라진다(트레이드오프). 교재 기준 [Ch1 p.27 → 교재 pp.34–35]

더 자세히: 이론 노트 Ch1 · 7~10 / 실전 1

Ch2 · 1 슬라이드 p.10–16, 29, 34–44

레지스터·메모리·엔디언

시험에 나오는 모양: 배열 오프셋 계산, 코드 실행 뒤 레지스터 결과값, 리틀/빅 엔디언 바이트 배치. (기출: 26-1 레지스터 결과값·little/big endian)

외울 것

- 32비트 레지스터 32개(x0~x31). word = 32비트, doubleword = 64비트. [Ch2 p.10]

레지스터	역할	근거
x0	항상 0 (배선으로 고정)	[Ch2 p.16]
x1	복귀 주소(return address), jal이 저장	[Ch2 p.34–35]
x2 (sp)	스택 포인터	[Ch2 p.37]
x8 (fp)	프레임 포인터	[Ch2 p.44]
x10~x17	인자 8개, 그중 x10·x11 = 반환값	[Ch2 p.34]

- 표의 x1·sp·fp·인자·반환값은 함수(프로시저)를 부를 때 쓰는 레지스터다. 인자 = 함수에 넘기는 값, 반환값 = 함수가 돌려주는 결과, jal = 함수로 건너가는 명령어, 스택 = 값을 잠시 맡겨 두는 메모리 영역. 이 단원 문제에는 x0만 필요하고, 나머지는 Ch2 · 3에서 다룬다. 임시(temporary)·저장(saved) 레지스터 구분도 Ch2 · 3. [Ch2 p.32, 37]
- x0 활용: x0은 읽으면 언제나 0이므로 상수 0이 필요한 곳에 쓴다. `sub x1, x0, x1` = 0 − x1 = −x1, 즉 부정(= 부호 반전). [Ch2 p.16] `add x7, x6, x0` = x6 + 0 = x6이라 x6을 x7로 복사한 것과 같다. `beq x0, x0, L` = 0과 0은 늘 같으니 항상 L로 건너뛰는 무조건 분기. [Ch2 p.29] 거꾸로 x0을 결과 칸(rd)에 두면 계산은 하지만 값이 들어가지 않는다(배선으로 0에 고정되어 있어 쓰기가 무시됨). 그래서 `addi x0, x5, 3` 뒤에도 x0 = 0이다. PDF 외 보충
- 산술은 레지스터에서만, 메모리는 load·store로. [Ch2 p.11] 바이트 주소, 연속한 word는 4씩 차이. [Ch2 p.12]

```
A[i]의 주소 = 시작 주소(베이스) + i × 4 (4바이트 정수 배열)
```

- rd = 결과를 받는 레지스터, rs1 = 주소의 기준(베이스) 레지스터(읽는 법은 기초 · 4). `lw rd, 오프셋(rs1)` : 메모리 → 레지스터. `sw rs2, 오프셋(rs1)` : 레지스터 → 메모리(rs2 = 저장할 값). 오프셋은 바이트 단위. [Ch2 p.14]
  sw만 읽는 순서가 다르다: "쉼표 뒤 첫 칸 = 결과 칸"(기초 · 3) 규칙은 결과가 레지스터에 들어가는 명령어 얘기다. sw는 결과가 메모리에 쓰이므로 맨 앞 레지스터(rs2)도 읽기만 하고 값이 그대로다. `sw x9, 48(x22)` = "x9의 값을 주소 x22 + 48에 써라". lw는 반대로 맨 앞 x9가 메모리에서 읽은 값을 받는다. 오프셋이 i × 4인 것은 int 하나가 4바이트(기초 · 4)이고 주소는 바이트마다 하나씩 붙기 때문이다(A[1]은 A[0]보다 4바이트 뒤).
  바이트와 주소: 1바이트 = 8비트이고 16진 한 자리 = 4비트(기초 · 1)라 **1바이트 = 16진 두 자리**다(예 0x33, 0xFF). 메모리 주소 하나에는 바이트 하나가 들어간다(기초 · 3). 그래서 32비트 값(16진 8자리)은 두 자리씩 끊은 4바이트가 연속한 주소 4개(예 0x100, 0x101, 0x102, 0x103)에 나눠 담기고, word의 주소는 그중 가장 낮은 주소(0x100)로 부른다. [Ch2 p.12–13]
- 32비트 값은 4바이트라 메모리 주소 4개에 나눠 담긴다. 이때 순서를 정하는 규칙이 엔디언(endianness)이다. 최상위 바이트 = 16진 8자리로 썼을 때 맨 왼쪽 두 자리(0x00112233의 00, 자릿값이 가장 큼), 최하위 바이트 = 맨 오른쪽 두 자리(33, 자릿값이 가장 작음). 빅 엔디언 = 최상위 바이트를 가장 낮은 주소에. 리틀 엔디언 = 최하위 바이트를 가장 낮은 주소에. **RISC-V = 리틀 엔디언.** [Ch2 p.13]
  lw로 읽은 값을 16진으로 쓸 때: 수는 최상위 바이트를 맨 왼쪽에 쓴다. 리틀 엔디언에서 최상위 바이트는 **가장 높은 주소**에 있으므로 높은 주소의 바이트부터 왼쪽에 놓아 이어 붙인다. 예: 주소 0~3에 33, 22, 11, 00 → 높은 주소부터 00, 11, 22, 33 → 0x00112233(아래 표의 리틀 엔디언 줄을 거꾸로 읽은 것). sw는 그 반대로 오른쪽 끝 두 자리를 가장 낮은 주소에 놓는다.
- RISC-V는 정렬 제한이 없다: word가 4의 배수 주소에서 시작할 필요 **없다(NOT)**. [Ch2 p.13]
- 레지스터가 훨씬 빠르다. 덜 쓰는 변수만 메모리로(spill). [Ch2 p.15]

```
// A[12] = h + A[8];  h = x21, A의 시작 주소 = x22
lw   x9, 32(x22)     // x9 = A[8]   (8 × 4 = 32)
add  x9, x21, x9     // x9 = h + A[8]
sw   x9, 48(x22)     // A[12] = x9  (12 × 4 = 48)
```

0x00112233 저장	주소 0	주소 1	주소 2	주소 3
빅 엔디언	00	11	22	33
리틀 엔디언 (RISC-V)	33	22	11	00

푸는 순서

1. 배열 오프셋 = 인덱스 × 4. 주소 = 베이스 + 오프셋.

2. 엔디언: 16진수를 두 자리씩 끊는다. 오른쪽 끝이 최하위 바이트.

3. 리틀 엔디언: 최하위 바이트부터 낮은 주소에. lw 값은 높은 주소 바이트부터 이어 붙인다.

4. 결과값은 16진수 8자리, 음수는 2의 보수(−7 = 0xFFFFFFF9).

함정

- A[8]을 `lw x9, 8(x22)` 로 쓰면 X. 바이트 주소라 8 × 4 = 32.
- 바이트 순서만 뒤집는다. 바이트 안의 두 자리(33)는 뒤집지 않는다.
- lw 결과를 낮은 주소부터 그대로 읽으면 빅 엔디언 답이 된다.

예제 (시험 수준)

x22 = 0x00000100이고, 주소 0x104~0x107에 바이트 78, 56, 34, 12가 차례로 들어 있다. 다음을 실행한 뒤 x5의 값과 주소 0x10C~0x10F의 바이트를 쓰시오(RISC-V).

```
lw   x5, 4(x22)
addi x5, x5, 1
sw   x5, 12(x22)
```

풀이 보기

x5 = 0x12345679, 0x10C~0x10F = 79, 56, 34, 12

① lw 주소 = x22 + 4 = 0x100 + 4 = 0x104. 여기서부터 4바이트(0x104~0x107)를 읽는다.

② 리틀 엔디언이라 가장 낮은 주소 0x104의 78이 최하위 바이트다. 높은 주소부터 이어 붙이면 12 34 56 78 → x5 = 0x12345678.

③ addi로 1을 더하면 0x12345679.

④ sw 주소 = 0x100 + 12 = 0x10C(12 = 0xC). 12바이트 = word 3개 뒤이므로 배열로 치면 A[3] 자리다.

⑤ 0x12345679를 두 자리씩 끊으면 12, 34, 56, 79. 최하위 79를 가장 낮은 주소 0x10C에: 0x10C = 79, 0x10D = 56, 0x10E = 34, 0x10F = 12. [Ch2 p.13–14]

회독 점검

1. 정수 배열 A의 시작 주소 x22 = 0x00001000이다. `lw x7, 40(x22)`는 A의 몇 번째 원소를 읽는가? 읽는 4바이트의 주소 범위는?

   답 보기

   A[10], 0x1028~0x102B

   ① 오프셋 40은 바이트 단위다. 원소 하나가 4바이트이므로 인덱스 = 40 ÷ 4 = 10 → A[10].

   ② 시작 주소 = 0x1000 + 40. 40 = 2 × 16 + 8 = 0x28이므로 0x1000 + 0x28 = 0x1028.

   ③ word는 4바이트라 0x1028, 0x1029, 0x102A, 0x102B(마지막 = 0x1028 + 3)를 읽는다. [Ch2 p.12, 14]

---

2. 정수 배열 B의 시작 주소가 x9, g가 x20에 있다. `B[6] = B[2] - g;`를 RISC-V 세 줄로 쓰시오(임시 레지스터 x5).

   답 보기

   `lw x5, 8(x9)` / `sub x5, x5, x20` / `sw x5, 24(x9)`

   ① B[2]의 오프셋 = 2 × 4 = 8. 메모리 값은 먼저 레지스터로 읽어야 계산할 수 있다.

   ② sub rd, rs1, rs2 = rs1 − rs2이므로 B[2] − g는 `sub x5, x5, x20`.

   ③ B[6]의 오프셋 = 6 × 4 = 24. sw는 앞 레지스터(x5)의 값을 메모리에 쓴다. [Ch2 p.14]

---

3. 처음 x5 = 7이다. 다음을 실행한 뒤 x6, x7, x0의 값을 16진수 8자리로 쓰시오.

```
sub  x6, x0, x5
add  x7, x6, x0
addi x0, x5, 3
```

답 보기

x6 = 0xFFFFFFF9, x7 = 0xFFFFFFF9, x0 = 0x00000000

① x0은 항상 0이라 sub x6, x0, x5 = 0 − 7 = −7(부호 반전).

② −7의 2의 보수: 7 = 0x00000007을 모두 반전 → 0xFFFFFFF8, 1을 더함 → 0xFFFFFFF9.

③ add x7, x6, x0 = x6 + 0 = x6 복사 → 0xFFFFFFF9.

④ addi x0, x5, 3은 x5 + 3 = 10을 x0에 넣으라는 뜻이지만, x0은 배선으로 0에 고정되어 쓰기가 무시된다. 그래서 x0 = 0x00000000 그대로다. [Ch2 p.16]

---

4. x10 = 0x00000300이고 주소 0x300~0x303에 바이트 88, 77, 66, 55가 차례로 들어 있다. `lw x5, 0(x10)` 뒤 x5는? 빅 엔디언 기계였다면?

답 보기

RISC-V(리틀): 0x55667788 / 빅: 0x88776655

① 리틀 엔디언은 가장 낮은 주소(0x300)의 88이 최하위 바이트다.

② 높은 주소부터 이어 붙인다: 55, 66, 77, 88 → 0x55667788.

③ 빅 엔디언은 낮은 주소가 최상위라 쓰인 순서 그대로 0x88776655. [Ch2 p.13]

---

5. O/X: RISC-V에서 주소 0x1002에서 시작하는 word는 4의 배수 주소가 아니므로 lw로 읽을 수 없다.

답 보기

X

① RISC-V는 정렬 제한이 없다. word가 4의 배수 주소에서 시작하지 않아도 된다(NOT). [Ch2 p.13]

② 0x1002~0x1005의 4바이트를 같은 리틀 엔디언 규칙으로 읽는다.

---

6. O/X: RISC-V의 add는 메모리에 있는 값을 바로 피연산자로 쓸 수 있으므로, h + A[8]은 add 한 줄로 계산된다.

   답 보기

   X

   ① RISC-V에서 산술 명령은 메모리 데이터에 직접 접근할 수 **없다(NOT)**. 산술은 레지스터에서만 한다. [Ch2 p.11, 15]

   ② 그래서 `lw x9, 32(x22)`로 A[8]을 먼저 레지스터에 읽고 `add x9, x21, x9`를 한다(두 줄). 실행할 명령어가 늘어나므로 컴파일러는 변수를 최대한 레지스터에 두고, 덜 쓰는 변수만 메모리로 내보낸다(spill). [Ch2 p.15]

더 자세히: 이론 노트 Ch2 · 2, 3 / 실전 3, 카드 표 4

Ch2 · 2 슬라이드 p.9, 12–16, 28–31, 46

C ↔ 어셈블리

시험에 나오는 모양: C 한두 줄을 RISC-V로, RISC-V 몇 줄을 C로 바꾸기, 짧은 코드를 실행한 뒤 레지스터 값 쓰기. (기출: 26-1 C → 어셈블리 2문제, 어셈블리 → C 1문제, 실행 후 레지스터 결과값)

먼저 알아 둘 말

- C의 `if (조건) 문장1; else 문장2;` = 조건이 참이면 문장1, 거짓이면 문장2를 실행. `while (조건) 문장;` = 조건이 참인 동안 문장을 되풀이. [Ch2 p.28–30]
- 비교 기호: `==` 같다, `!=` 같지 않다, `<` 작다, `>=` 크거나 같다. `i += 1` 은 `i = i + 1` 과 같다. 조건 분기 명령어 beq·bne·blt·bge는 기초 · 4 표. 비교 순서는 쓴 순서 그대로다: `blt a, b, L` = "a < b이면 L로", `bge a, b, L` = "a >= b이면 L로". RISC-V에는 bgt(>)·ble(<=) 명령어가 없어서, 필요하면 두 레지스터 순서를 바꿔 blt·bge로 만든다(아래 표). PDF 외 보충

쉽게 말하면

C 한 줄은 정해진 틀 몇 가지 중 하나로 바뀐다. 틀을 외우고 레지스터 번호만 끼워 넣는다. 레지스터·메모리 자체는 Ch2 · 1 참고.

외울 것

레지스터 배정은 **슬라이드 예제마다 다르다**. 시험에서도 문제에 주어진 대응을 그대로 쓴다.

- p.14 배열 예제: h = x21, A의 시작 주소 = x22, 중간값 x9. [Ch2 p.14]
- p.29 if-else 예제: f, g, h, i, j = x19, x20, x21, x22, x23. [Ch2 p.29]
- p.30 while 예제: i = x22, k = x24, save의 시작 주소 = x25, 중간값 x10·x9. [Ch2 p.30]
- 임시 레지스터 = 계산 중간값을 잠깐 담는 레지스터. 규약상 임시 레지스터 = x5~x7, x28~x31. x9는 슬라이드 예제가 임시로 쓰지만 규약상 저장 레지스터(Ch2 · 3). [Ch2 p.39]

번역 틀 (아래 표는 p.29 배정 f~j = x19~x23)

C	RISC-V 틀	요점
`f = g + h;` / `f = g - h;`	`add x19, x20, x21` / `sub x19, x20, x21`	소스 2 + 목적지 1 [Ch2 p.9]
`f = (g + h) - (i + j);`	`add x5, x20, x21` · `add x6, x22, x23` · `sub x19, x5, x6`	괄호마다 임시 레지스터
`f = g + 10;` / `f = g - 10;`	`addi x19, x20, 10` / `addi x19, x20, -10`	subi 없음, 음수를 더함 [Ch2 p.16]
`f = A[i];` (이 행만 A = x25, i = x22)	`slli x5, x22, 2` · `add x5, x5, x25` · `lw x19, 0(x5)`	상수 인덱스면 오프셋 = 인덱스 × 4(Ch2 · 1). 변수 인덱스면 × 4 → 시작 주소 더함 → 오프셋 0. 이유: 오프셋은 명령어 안에 적힌 상수라 어셈블할 때 정해지고, 실행 중에 바뀌는 i를 넣을 수 없다. 그래서 주소 전체를 레지스터에 만들어 두고 오프셋은 0. `slli x5, x22, 2`는 i를 왼쪽으로 2칸 = × 2^2 = × 4(기초 · 3 시프트 표). 예 i = 3이면 11_2 → 1100_2 = 12
32비트 큰 상수	`lui`(상위 20비트) + `addi`(하위 12비트)	addi 즉시값은 12비트뿐이라 큰 상수를 한 번에 못 넣는다. `lui rd, 상수` (load upper immediate) = 20비트 상수를 rd의 31~12번 비트에 넣고 아래 12비트는 0으로. 이어서 addi로 아래 12비트를 더한다 [Ch2 p.46]

32비트 상수 만들기 (lui + addi) [Ch2 p.46]

① 16진 8자리를 위 5자리 | 아래 3자리로 자른다. 16진 한 자리 = 4비트(기초 · 1)라 위 5자리 = 20비트(31~12번 비트), 아래 3자리 = 12비트(11~0번 비트)다.

② `lui rd, 위 5자리` : rd = 위 5자리 뒤에 0을 3자리 붙인 값(아래 12비트가 0). 예 `lui x19, 0x003D0` → x19 = 0x003D0000.

③ `addi rd, rd, 아래 3자리` : 아래 3자리의 맨 앞 자리가 0~7이면(11번 비트가 0) 그대로 더하면 끝난다. 슬라이드 예 0x003D0500은 lui 0x003D0 뒤 addi 0x500(= 1280) → 0x003D0000 + 0x500 = 0x003D0500.

④ 아래 3자리의 맨 앞 자리가 8~F이면(11번 비트가 1) 조심한다. addi의 즉시값은 12비트 2의 보수로 읽혀 부호 확장되므로(기초 · 2) 0x800~0xFFF는 음수 −2048 ~ −1이 된다. 그대로 쓰면 결과가 원하는 값보다 0x1000(= 4096) 작아진다. 그래서 lui 값을 1 늘려(= 0x1000을 미리 더해) 맞춘다. 예 0x00012800: 아래 0x800 = 1000 0000 0000은 12비트로 −2048. `lui x5, 0x00013` → 0x00013000, `addi x5, x5, -2048` → 0x00013000 − 0x800 = 0x00012800. 검산 0x13000 − 0x12800 = 0x800 = 2048. PDF 외 보충

왜 조건을 뒤집나: 분기 명령어는 조건이 참일 때만 건너뛰고, 거짓이면 바로 다음 줄을 실행한다. [Ch2 p.28]
슬라이드 틀은 then 부분을 분기 바로 아래에 둔다. 그러면 C 조건이 참일 때는 분기하지 않고 그대로 내려가 then을 실행해야 하고, 거짓일 때만 then을 건너뛰어 Else로 가야 한다. 그래서 분기 조건 = C 조건의 반대다. 예: `if (i == j)` 에서 i == j면 add로 내려가야 하므로, 분기는 반대인 i != j일 때 Else로 가는 `bne` . while도 같다: 조건이 참이면 몸체로 내려가고, 거짓이면 Exit로 나가므로 탈출 분기는 반대 조건이다.

if-else와 while (슬라이드 순서 그대로) [Ch2 p.29–30]

```
// if (i == j) f = g + h; else f = g - h;
      bne  x22, x23, Else   // 조건을 뒤집어 i != j 이면 Else
      add  x19, x20, x21
      beq  x0, x0, Exit     // then 끝나면 무조건 Exit
Else: sub  x19, x20, x21
Exit:

// while (save[i] == k) i += 1;   i = x22, k = x24, save = x25
Loop: slli x10, x22, 2      // i * 4
      add  x10, x10, x25    // save[i]의 주소
      lw   x9, 0(x10)       // save[i]
      bne  x9, x24, Exit    // 조건을 뒤집어 다르면 탈출
      addi x22, x22, 1
      beq  x0, x0, Loop     // 처음으로
Exit:
```

C 조건 (참이면 then)	Else로 건너뛰는 분기 (조건을 뒤집음)
a == b	`bne a, b, Else`
a != b	`beq a, b, Else`
a < b	`bge a, b, Else`
a >= b	`blt a, b, Else`
a > b	`bge b, a, Else` (순서를 바꿔 b >= a)
a <= b	`blt b, a, Else` (b < a)

마지막 두 행: a > b의 반대는 a <= b인데 ble가 없다. a <= b는 b >= a와 같은 말이라 순서를 바꿔 `bge b, a`. a <= b의 반대 a > b도 b < a로 바꿔 `blt b, a`.

blt·bge = 부호 있는 비교, bltu·bgeu = 무부호 비교. 0xFFFFFFFF와 1: blt는 −1 < 1 참, bltu는 $2^{32} - 1 < 1$ 거짓. [Ch2 p.31]

푸는 순서

1. 문제에 주어진 변수 ↔ 레지스터 대응을 옮겨 적는다. 임시값은 문제가 정한 것, 없으면 x5~x7, x28~x31.
2. 배열: 상수 인덱스면 오프셋 = 인덱스 × 4, 변수면 `slli` 2 → `add` → `lw` / `sw` 0(주소).
3. 산술은 레지스터끼리만: 메모리 값은 `lw`로 가져오고 `sw`로 쓴다.
4. if-else: 뒤집은 조건으로 Else → then → `beq x0, x0, Exit` → Else: → Exit:.
5. 반복문: Loop:에서 뒤집은 조건으로 Exit → 몸체 → `beq x0, x0, Loop`.
6. 어셈블리 → C: 오프셋 ÷ 4 = 인덱스, 분기 조건을 다시 뒤집기, 뒤로 가는 분기 = 반복문.
7. 실행 추적: 줄마다 레지스터 표. x0 = 0, 음수는 32비트 2의 보수(−1 = 0xFFFFFFFF).

함정

- A[3]의 오프셋은 12. 변수 i는 오프셋 자리(상수만)에 못 쓴다.
- i == j면 `bne` 로 Else. then 뒤 `beq x0, x0, Exit`를 빠뜨리면 else까지 실행. 상수와 비교하려면 먼저 `addi x6, x0, 10`.

예제 (시험 수준)

i는 x20, 정수 배열 A의 시작 주소는 x21이다. 임시 레지스터는 x5, x6을 쓴다. (1) `while (A[i] != 0) i = i + 1;`을 RISC-V로 쓰시오. (2) 처음 i = 0이고 A = {3, 7, 0, 5}일 때, 끝난 뒤 i의 값과 `addi`가 실행된 횟수를 쓰시오.

풀이 보기

---

(1) 아래 6줄 (2) i = 2, addi 2번

```
Loop: slli x5, x20, 2      // x5 = i * 4
      add  x5, x5, x21     // x5 = A[i]의 주소
      lw   x6, 0(x5)       // x6 = A[i]
      beq  x6, x0, Exit    // A[i] == 0 이면 탈출
      addi x20, x20, 1     // i = i + 1
      beq  x0, x0, Loop    // 처음으로
Exit:
```

① 슬라이드 while과 같은 틀이다. 다른 점은 조건뿐: C 조건 A[i] != 0을 뒤집으면 A[i] == 0이고, 비교할 0은 x0을 쓰면 되므로 `beq x6, x0, Exit`.

② i는 실행 중에 바뀌는 변수라 오프셋에 못 쓴다. 그래서 i × 4를 `slli`로(2칸 왼쪽 = × 4), 시작 주소를 `add`로 더해 주소를 만든 뒤 `lw x6, 0(x5)`.

③ 추적: i = 0 → A[0] = 3, 0이 아니라 계속, i = 1. A[1] = 7, 계속, i = 2. A[2] = 0 → beq가 참이라 Exit.

④ addi는 i = 0, 1일 때 두 번 실행되고, 끝난 뒤 i = 2. lw는 세 번 실행된다(마지막 검사 포함).

회독 점검

1. (26-1 유형) C → 어셈블리: `A[4] = g - A[2];` g는 x19, 정수 배열 A의 시작 주소는 x23, 임시 레지스터는 x9이다.

   답 보기

   `lw x9, 8(x23)` · `sub x9, x19, x9` · `sw x9, 16(x23)`

   ① 오프셋: int 하나가 4바이트라 A[2] = 2 × 4 = 8, A[4] = 4 × 4 = 16.

   ② 산술은 레지스터끼리만 하므로 A[2]를 먼저 `lw`로 x9에 가져온다.

   ③ g − A[2]이므로 첫 소스가 g(x19): `sub x9, x19, x9`. 순서를 바꾸면 A[2] − g가 된다.

   ④ 결과를 A[4] 자리에 `sw x9, 16(x23)`.

---

2. (26-1 유형) C → 어셈블리: `if (a < b) c = a + b; else c = a - 1;` a, b, c는 x18, x19, x20이고 부호 있는 정수다.

   답 보기

   아래 4줄 + Exit:

```
      bge  x18, x19, Else   // a >= b 이면 Else
      add  x20, x18, x19    // c = a + b
      beq  x0, x0, Exit     // 무조건 Exit
Else: addi x20, x18, -1     // c = a - 1
Exit:
```

   ① 분기는 "참이면 건너뛴다"이므로 then을 건너뛸 조건 = a < b의 반대 = a >= b → `bge`. 부호 있는 수라 bgeu가 아니다.

   ② then 뒤 무조건 분기로 else를 건너뛴다. a − 1은 즉시값 빼기가 없으므로 `addi`에 `-1`.

---

3. (26-1 유형) 어셈블리 → C: h, i는 x20, x21이고 정수 배열 A의 시작 주소는 x22이다. C 한 줄로 쓰시오.

```
slli x5, x21, 2
add  x5, x5, x22
lw   x6, 0(x5)
add  x6, x6, x20
sw   x6, 8(x22)
```

답 보기

`A[2] = A[i] + h;`

① 앞 두 줄: x5 = i × 4 + A의 시작 주소 = A[i]의 주소. 셋째 줄로 x6 = A[i].

② 넷째 줄: x6 = A[i] + h.

③ 마지막 줄의 오프셋 8은 시작 주소(x22) 기준이므로 8 ÷ 4 = 2 → A[2]에 저장. 주소 계산한 x5가 아닌 x22를 썼다는 점을 놓치지 말 것.

---

4. (26-1 유형) 다음 코드를 실행한 뒤 x5, x6의 값을 16진수로 쓰고, `add`가 몇 번 실행되는지 쓰시오.

```
      addi x5, x0, 0
      addi x6, x0, 5
Loop: beq  x6, x0, Exit
      add  x5, x5, x6
      addi x6, x6, -1
      beq  x0, x0, Loop
Exit:
```

답 보기

x5 = 0x0000000F (15), x6 = 0x00000000, add 5번

① 처음 x5 = 0, x6 = 5. 매 바퀴 x6이 0인지 먼저 검사하고, 아니면 x5에 x6을 더한 뒤 x6을 1 줄인다.

② 바퀴별 x6: 5, 4, 3, 2, 1에서 더하고, 0이 되면 beq가 참이라 Exit. 더한 횟수 5번.

③ x5 = 5 + 4 + 3 + 2 + 1 = 15 = 0xF → 8자리로 0x0000000F.

C로 쓰면 `s = 0; n = 5; while (n != 0) { s = s + n; n = n - 1; }`

---

5. x5 = 0x00000005, x6 = 0xFFFFFFFB이다. 분기하는 것을 모두 고르시오. (가) `blt x5, x6, L` (나) `bltu x5, x6, L` (다) `bge x6, x5, L` (라) `bgeu x6, x5, L`

   답 보기

   (나), (라)

   ① x6 = 0xFFFFFFFB는 MSB가 1. 부호 있게 읽으면 반전 0x00000004, +1 → 5이므로 −5. 무부호로 읽으면 2^{32} − 5(아주 큰 수).

   ② (가) 5 < −5 거짓. (나) 5 < 2^{32} − 5 참.

   ③ (다) −5 >= 5 거짓. (라) 2^{32} − 5 >= 5 참. [Ch2 p.31]

6. x7에 0x12345FFF를 넣는 lui, addi 두 명령어를 쓰시오. PDF 외 보충

   답 보기

   `lui x7, 0x12346` · `addi x7, x7, -1`

   ① 8자리를 위 5자리 | 아래 3자리로 자른다: 0x12345 | 0xFFF.

   ② 0xFFF = 1111 1111 1111, 11번 비트가 1이라 addi에서는 12비트 2의 보수로 −1이 된다(부호 확장). 그냥 쓰면 0x12345000 − 1 = 0x12344FFF로 틀린다. 외울 것의 "32비트 상수 만들기" ④ 경우다.

   ③ addi가 1을 빼는 만큼 lui를 1 늘린다: 0x12346000 + (−1) = 0x12345FFF.

   검산: 0x12346000 − 0x1 = 0x12345FFF (bash로 확인).

더 자세히: 이론 노트 Ch2 · 2, 3, 7, 12 / 실전 2, 3

Ch2 · 3 슬라이드 p.32–45

프로시저와 스택

시험에 나오는 모양: 레지스터 번호를 주고 "스택에 저장 → 계산 → 복원 → 복귀" 코드 쓰기·빈칸, sp 값 계산, 누가 저장하는지 O/X. (기출: 21-2 스택 할당 → 덧셈 → 원래 주소로 복귀)

먼저 알아 둘 말

- **프로시저(procedure)** = 함수. 다른 코드가 이름으로 불러 쓰는 코드 묶음. 부르는 쪽이 **호출자(caller)**, 불려가는 쪽이 **피호출자(callee)**다. [Ch2 p.32]
- **인자(argument)** = 호출자가 피호출자에게 넘기는 값, **반환값(return value)** = 피호출자가 돌려주는 결과 (C의 `return`). [Ch2 p.32]
- **스택(stack)** = 레지스터 값을 잠시 맡겨 두는 메모리 영역. **sp(stack pointer, x2)**는 스택 맨 위 칸의 주소를 담는 레지스터다. [Ch2 p.37]
- **jal**(jump and link) = 복귀 주소를 x1에 남기고 함수로 건너가는 명령어, **jalr**(jump and link register) = 레지스터에 든 주소로 건너가는 명령어(복귀에 씀). 아래 공식 두 줄이 정확한 동작이다. [Ch2 p.35–36]

쉽게 말하면

호출 = 인자를 x10부터 놓고 `jal` 로 건너가 일한 뒤, 결과를 x10에 놓고 x1의 주소로 돌아오기. 레지스터가 모자라면 스택에 잠시 맡긴다.

외울 것

프로시저 실행 6단계 [Ch2 p.33]

1. 인자 배치(프로시저가 읽을 수 있는 곳에)
2. 제어 이동(`jal`)
3. 저장 공간 확보
4. 작업 수행
5. 결과 배치(호출한 쪽이 읽을 수 있는 곳에)
6. 복귀(호출한 지점으로, `jalr`)

레지스터	역할
x10~x17 (8개)	인자. 그중 x10·x11은 반환값 [Ch2 p.34]
x1 (1개)	복귀 주소(return address). 호출용은 x10~x17 + x1 = **9개**
x2 = sp	스택 포인터, 스택 맨 위(가장 최근 값) [Ch2 p.37]
x8 = fp	프레임 포인터, 프로시저 안에서 변하지 않는 기준 [Ch2 p.44]

```
jal x1, L : x1 = PC + 4(다음 명령어 주소), PC = L
```

```
jalr x0, 0(x1) : PC = x1 + 0, 링크 값은 x0에 써서 버림
```

jal과 jalr은 둘 다 "PC + 4를 rd에 쓰고(link), PC를 새 주소로 바꾼다(jump)". 다른 점은 새 주소를 만드는 법뿐이다: jal은 라벨 L, jalr은 레지스터 + 상수. 호출할 때는 돌아올 주소가 필요하니 rd = x1. 복귀할 때는 더 돌아올 곳이 없으니 rd = x0으로 써서 PC + 4를 버린다(x0은 항상 0이라 써도 바뀌지 않음). [Ch2 p.35–36] 예: 주소 0x1000의 `jal x1, proc` → x1 = 0x1004, PC = proc의 주소. proc 끝의 `jalr x0, 0(x1)` → PC = 0x1004, 호출 다음 줄부터 이어서 실행.

스택: LIFO(last-in-first-out, 나중에 넣은 것을 먼저 꺼냄), push/pop 명령어 없음(`sw` / `lw`), 높은 → 낮은 주소로 자람, 레지스터 하나에 sp 4바이트. [Ch2 p.37]

sp를 왜·얼마나 빼고, sw 오프셋은 왜 그 값인가: 스택은 높은 주소에서 낮은 주소로 자라므로 새 칸을 만들려면 sp를 줄인다. push 명령어가 없어 `addi` 로 직접 뺀다. 레지스터 하나 = 32비트 = 4바이트라 k개면 4k바이트를 뺀다. 그러면 새 sp부터 원래 sp 바로 아래까지 4k바이트가 이 프로시저의 자리이고, 그 안의 칸 주소는 sp + 0, sp + 4, …, sp + 4(k − 1)이다. 이 수가 `sw x?, 오프셋(sp)` 의 오프셋이다. 4k(sp)는 원래 sp가 가리키던 칸(이미 쓰고 있는 스택 맨 위)이라 쓰면 안 된다. 어느 레지스터를 어느 칸에 둘지는 자유지만, 복원하는 `lw` 는

저장한 칸과 같은 오프셋이어야 한다(슬라이드는 먼저 저장하는 것을 가장 높은 칸에). 끝나면 4k를 더해 sp를 원래대로 돌려 자리를 반납한다.

예: 원래 sp = 0x00001000, 3개 저장	주소
`addi sp, sp, -12` 뒤 sp	0x1000 − 12 = 4096 − 12 = 4084 = 15 × 256 + 15 × 16 + 4 = 0x00000FF4
8(sp) (가장 높은 칸)	0xFF4 + 8 = 0xFFC (= 원래 sp − 4)
4(sp)	0xFF8
0(sp)	0xFF4

16진 뺄셈으로 해도 같다: 0x1000 − 0xC는 맨 오른쪽 0에서 C를 못 빼므로 왼쪽에서 빌리는데 가운데 두 자리도 0이라 맨 앞 1까지 빌림이 이어진다. 0x1000 = 0x0FF0 + 0x10이므로 0x0FF0 + (16 − 12) = 0x0FF4.

leaf 예(다른 프로시저를 안 부름). C 코드: `int leaf_example(int g, int h, int i, int j) { int f; f = (g + h) - (i + j); return f; }` , g~j = x10~x13, f = x20 [Ch2 p.38]

```
leaf_example:
    addi sp, sp, -12      // 3개 x 4바이트 자리
    sw   x5, 8(sp)
    sw   x6, 4(sp)
    sw   x20, 0(sp)
    add  x5, x10, x11     // g + h
    add  x6, x12, x13     // i + j
    sub  x20, x5, x6      // f
    addi x10, x20, 0      // 반환값은 x10
    lw   x20, 0(sp)
    lw   x6, 4(sp)
    lw   x5, 8(sp)
    addi sp, sp, 12       // 자리 반납
    jalr x0, 0(x1)        // 복귀
```

임시·저장 레지스터 구분 (레지스터 저장 규약)

구분	p.39 정의 (두 그룹 합 19개)	p.41·43 호출 규약에서 추가
caller-saved = 임시(temporary). 피호출자가 보존 안 함(NOT preserved)	x5~x7, x28~x31 (3 + 4 = 7개)	x10~x17
callee-saved = 저장(saved). 쓰면 피호출자가 저장·복원, 호출 전후 값 유지	x8~x9, x18~x27 (2 + 10 = 12개)	x1

그래서 위 예에서 x5·x6의 sw/lw는 불필요, x20은 필요. [Ch2 p.39]

누가 저장하나 판단법 [Ch2 p.39, 41, 43]

① callee-saved(x8~x9, x18~x27, 그리고 x1): 피호출자가 이 레지스터를 쓰려면 쓰기 전에 스택에 저장하고, 복귀 전에 복원한다. 그래서 호출자는 호출 뒤에도 값이 그대로라고 믿고 쓴다.

② caller-saved(x5~x7, x28~x31, x10~x17): 피호출자는 마음대로 바꿔도 된다. 그래서 호출자가 호출 뒤에도 그 값이 필요할 때만 호출 전에 직접 저장한다. 호출 뒤 필요 없으면 아무도 저장하지 않는다.

③ 한 프로시저가 두 역할을 함께 한다. 자기를 부른 쪽에게는 피호출자라 ①(쓰는 callee-saved 저장)을 하고, 안에서 다른 프로시저를 부르면 호출자라 ②(호출 뒤 필요한 caller-saved 저장)를 한다. 다른 프로시저를 부르면 `jal` 이 x1을 새 복귀 주소로 덮으므로 자기 x1도 저장한다.

나누는 이유: 오래 쓸 값은 저장 레지스터에, 잠깐 쓰고 버릴 값은 임시 레지스터에 두면 꼭 필요한 것만 저장하게 되어 sw/lw가 줄어든다. PDF 외 보충

nonleaf(안에서 다른 프로시저를 부름)·재귀(recursion, 자기 자신을 부름): 다시 호출하면 `jal` 이 x1을 덮으므로 자기 x1(과 필요한 인자)을 저장. 재귀 예 fact = 팩토리얼(n!): `addi sp, sp, -8` 로 x1(복귀 주소)·x10(인자 n) 두 개 저장. [Ch2 p.41–42]

text	static data	heap	stack
기계어 코드	상수·정적(전역) 변수	동적 할당(실행 중에 malloc으로 빌리고 free로 반납), 위로 자람	지역 변수·레지스터 저장, 아래로 자람

스택과 힙은 서로를 향해 자란다. [Ch2 p.44–45]

푸는 순서 (스택 코드 쓰기)

1. 저장할 레지스터 수 k를 센다("사용한 것 저장"이면 그대로, "규약대로"면 callee-saved만).
2. `addi sp, sp, -4k` (바이트 수, 빼기).
3. `sw` 로 오프셋 4(k – 1), …, 4, 0에 저장.
4. 계산 후 결과를 x10으로 옮긴다.
5. 같은 오프셋에서 `lw` 복원 → `addi sp, sp, 4k`.
6. `jalr x0, 0(x1)`.
7. sp 값: 16진 끝 두 자리만 10진으로 바꿔 계산. 빌림이 생기면 16진 뺄셈(빌림 16) 규칙으로 앞자리까지(기초 · 1).

함정

- "19개"를 물으면 p.39 기준(7 + 12, x10~x17·x1은 없음). "호출 규약에서 피호출자가 저장하는 것"이면 x1을 넣는다. [Ch2 p.39, 43]
- 호출용 레지스터는 9개(11개 아님). x1 = jal 다음 주소(PC + 4). `jalr x0, 0(x1)` 의 x0은 "0번지"가 아니라 링크를 버리는 곳.
- sp 조정은 바이트 수(개수 × 4), 시작은 빼기. 반환값은 반드시 x10.

예제 (시험 수준)

(21-2 유형) `int mix(int a, int b, int c, int d) { return (a + b) - (c - d); }`를 RISC-V로 쓰시오. a~d는 x10~x13, a + b는 x18, c − d는 x19, 결과는 x20에 계산하고, 사용한 x18, x19, x20은 스택에 저장했다가 복원한다. 처음 sp = 0x7FFFFF10일 때 저장 직후 sp와 x18이 저장된 주소도 쓰시오.

풀이 보기

아래 코드, sp = 0x7FFFFF04, x18은 0x7FFFFF0C

```
mix:
    addi sp, sp, -12     // 3개 x 4바이트
    sw   x18, 8(sp)
    sw   x19, 4(sp)
    sw   x20, 0(sp)
    add  x18, x10, x11   // a + b
    sub  x19, x12, x13   // c - d
    sub  x20, x18, x19   // (a + b) - (c - d)
    addi x10, x20, 0     // 반환값
    lw   x20, 0(sp)
    lw   x19, 4(sp)
    lw   x18, 8(sp)
    addi sp, sp, 12
    jalr x0, 0(x1)
```

① 저장할 것 3개 → 3 × 4 = 12바이트. 스택은 아래로 자라므로 `addi sp, sp, -12`, 오프셋은 8, 4, 0.

② sp: 0x7FFFFF10의 끝 두 자리 0x10 = 16, 16 − 12 = 4 = 0x04 → sp = 0x7FFFFF04.

③ x18은 8(sp)에 있으므로 0x04 + 8 = 12 = 0x0C → 0x7FFFFF0C. 원래 sp 바로 아래 칸(원래 sp − 4)이다.

④ 계산 후 결과를 x10으로 옮기고(5단계), 같은 오프셋에서 복원, 12를 더해 sp를 되돌리고 x1로 복귀(6단계).

시험과 연결: 규약만 따지면 x18~x20은 모두 callee-saved라 셋 다 저장이 맞다. [Ch2 p.39]

회독 점검

1. 프로시저 실행 6단계를 순서대로 쓰고, ②와 ⑥에서 쓰는 RISC-V 명령어를 쓰시오.

   답 보기

   인자 배치 → 제어 이동 → 저장 공간 확보 → 작업 수행 → 결과 배치 → 복귀. ② `jal x1, 라벨`, ⑥ `jalr x0, 0(x1)`

   ① 호출자가 ①②, 피호출자가 ③~⑥을 한다. [Ch2 p.33]

   ② jal은 점프하며 다음 주소를 x1에 남기고, jalr은 x1에 남은 주소로 돌아간다. [Ch2 p.35–36]

---

2. 처음 sp = 0x10000040이다. 레지스터 5개를 저장하려고 자리를 만든 뒤의 sp 값과, 가장 높은 칸(16(sp))의 주소를 쓰시오.

   답 보기

   sp = 0x1000002C, 16(sp) = 0x1000003C

   ① 5개 × 4바이트 = 20바이트 → `addi sp, sp, -20`.

   ② 끝 두 자리 0x40 = 4 × 16 = 64. 64 − 20 = 44 = 2 × 16 + 12 → 0x2C(C = 12). 앞자리는 빌릴 필요가 없어 그대로 → 0x1000002C.

   ③ 16(sp): 44 + 16 = 60 = 3 × 16 + 12 → 0x3C → 0x1000003C. 원래 sp − 4와 같다.

   검산: 0x1000002C + 0x14 = 0x10000040.

---

3. (21-2 유형) 빈칸 (가)~(마)를 채우시오. C: `int f(int g, int h) { return g + h - 7; }`, g·h는 x10·x11, 중간값은 x19, 결과는 x20, 두 레지스터를 스택에 저장한다.

```
f:  addi sp, sp, (가)
    sw   x19, 4(sp)
    sw   x20, (나)(sp)
    add  x19, x10, x11
    addi x20, x19, (다)
    addi x10, x20, 0
    lw   x20, 0(sp)
    lw   x19, (라)(sp)
    addi sp, sp, 8
    (마)
```

답 보기

(가) -8 (나) 0 (다) -7 (라) 4 (마) `jalr x0, 0(x1)`

① (가) 2개 × 4 = 8바이트, 아래로 자라므로 `-8`.

② (나) 복원 줄 `lw x20, 0(sp)`과 같은 오프셋 0. (라)도 저장 줄 `sw x19, 4(sp)`와 같은 4.

③ (다) 7을 빼는 즉시값 명령이 없으므로 addi에 `-7`.

④ (마) x1의 복귀 주소로 돌아간다.

---

4. O/X: (1) RISC-V는 x10~x17과 x1을 포함한 레지스터 19개를 caller-saved와 callee-saved 두 그룹으로 나눈다. (2) RISC-V 스택은 낮은 주소에서 높은 주소로 자라며 push 명령어로 값을 넣는다.

답 보기

(1) X (2) X

① (1) p.39의 19개는 x5~x7·x28~x31(3 + 4 = 7개)과 x8~x9·x18~x27(2 + 10 = 12개)이다. x10~x17과 x1은 p.41·43 호출 규약에서 추가된다. [Ch2 p.39, 43]

② (2) 높은 주소 → 낮은 주소로 자라고, push/pop 명령어 없이 sw/lw로 접근한다. [Ch2 p.37]

---

5. 주소 0x00400020에 `jal x1, proc`가 있다. 실행 직후 x1은? proc 끝의 `jalr x0, 0(x1)` 실행 직후 PC는? (proc 안에서 x1은 바뀌지 않음)

   답 보기

   x1 = 0x00400024, PC = 0x00400024

   ① 명령어 하나가 4바이트라 jal 다음 명령어 주소 = 0x00400020 + 4 = 0x00400024. 이것이 x1에 들어간다(link).

   ② jalr x0, 0(x1)은 PC = x1 + 0 = 0x00400024. x0에 쓴 링크 값은 버려진다.

6. 프로시저 P는 x7과 x21을 쓰고, 중간에 다른 프로시저 Q를 호출한다. Q 호출 뒤에는 x7 값이 필요 없다. 호출 규약에 따라 P가 시작할 때 저장해야 하는 레지스터와 sp 조정량은?

   답 보기

   x1과 x21, `addi sp, sp, -8`

   ① P는 자기를 부른 쪽에게는 피호출자다. x21은 callee-saved(x18~x27)라 피호출자인 P가 쓰기 전에 저장해야 한다.

   ② P는 Q를 부르므로 `jal x1, Q`가 P의 복귀 주소를 덮는다. 그래서 x1도 저장(nonleaf).

   ③ Q를 부를 때 P는 호출자다. x7은 caller-saved라 Q가 바꿀 수 있지만, Q 호출 뒤 필요 없으므로 P도 저장하지 않는다(필요했다면 P가 호출 전에 저장).

   ④ 2개 × 4 = 8바이트.

더 자세히: 이론 노트 Ch2 · 8, 9, 10, 11 / 실전 2

Ch2 · 4 슬라이드 p.21–24, 47–50

명령어 인코딩·디코딩

시험에 나오는 모양: 어셈블리 → 32비트 기계어(16진) → 리틀 엔디언 바이트, 기계어 → 어셈블리, 빈칸으로 branch offset과 형식(encoding) 이름. (기출: 26-1)

먼저 알아 둘 말

- **기계어(machine code)** = 명령어를 2진으로 바꾼 32비트 값. **형식(instruction format)** = 그 32비트를 어떤 칸(필드, field)으로 나누는지 정한 배치. R형·I형·S형·SB형·U형·UJ형이 있다. [Ch2 p.21]
- **opcode**(operation code) = 무슨 일을 하는 명령어인지 나타내는 7비트 칸. **funct3·funct7** = opcode가 같은 명령어들(예 add와 sub) 중 하나를 골라 주는 추가 칸(3비트·7비트). [Ch2 p.3, 22]
- **rd·rs1·rs2** = 레지스터 번호를 적는 5비트 칸(기초 · 3), **imm** = 즉시값을 적는 칸. imm[11:5]는 즉시값의 11~5번 비트라는 뜻이다(기초 · 3의 [hi:lo]).

쉽게 말하면

기계어 만들기는 32칸 양식지 채우기다. 형식마다 칸 순서·크기가 정해져 있고, 다 채운 뒤 4비트씩 끊어 16진수로 읽는다.

외울 것

형식별 칸 (왼쪽이 31번 비트) [Ch2 p.22–24, 46–47]

R형 (add, sub, sll, sra)

funct7 31–25	rs2 24–20	rs1 19–15	funct3 14–12	rd 11–7	opcode 6–0

I형 (lw, addi, slli, jalr), 즉시값 −2048 ~ 2047

imm[11:0] 31–20	rs1 19–15	funct3 14–12	rd 11–7	opcode 6–0

S형 (sw): rd 없음, rs2 = 저장할 값

imm[11:5] 31–25	rs2 24–20	rs1 19–15	funct3 14–12	imm[4:0] 11–7	opcode 6–0

SB형 (beq, bne, blt, bge): 오프셋 0번 비트는 저장 안 함

imm[12] 31	imm[10:5] 30–25	rs2 24–20	rs1 19–15	funct3 14–12	imm[4:1] 11–8	imm[11] 7	opcode 6–0

U형 (lui, opcode 0110111): 20비트 즉시값 · rd · opcode. UJ형 (jal, opcode 1101111): 20비트 즉시값 (조각냄) · rd · opcode. 교재 기준

rs1·rs2·rd·opcode·funct3는 그 칸이 있는 모든 형식에서 같은 자리(S형이 즉시값을 쪼갠 이유). [Ch2 p.24]

명령어	형식	opcode	funct3	funct7
add	R	0110011	000	0000000
sub	R	0110011	000	0100000
addi	I	0010011	000	n.a.
slli	I	0010011	001	0000000
lw	I	0000011	010	n.a.
jalr	I	1100111	000	n.a.
sw	S	0100011	010	n.a.
beq / bne	SB	1100011	000 / 001	n.a.
blt / bge	SB	1100011	100 / 101	n.a.
bltu / bgeu	SB	1100011	110 / 111	n.a.
lui	U	0110111	n.a.	n.a.
jal	UJ	1101111	n.a.	n.a.

p.50 인코딩 표의 일부. [Ch2 p.50] 26-1엔 시험지에 줬지만 계속 줄지는 장담 못 한다고 했으니 형식 이름, 칸 순서·폭, 주요 opcode는 외운다.

slli는 I형이지만 즉시값 칸 12비트를 둘로 나눠 쓴다: 위 7비트(31~25번)에 funct7 0000000, 아래 5비트(24~20번)에 시프트 양. 시프트 양은 0~31이라 5비트면 충분하다. 교재 기준

디코딩할 때는 거꾸로 opcode에서 형식을 찾는다. 위 표를 opcode 순으로 다시 쓰면:

opcode (6~0번)	형식	명령어 (funct3·funct7로 구별)
0110011	R	add, sub (funct7 0000000 / 0100000)
0010011	I	addi (funct3 000), slli (001)
0000011	I	lw (010)
1100111	I	jalr (000)
0100011	S	sw (010)
1100011	SB	beq 000, bne 001, blt 100, bge 101, bltu 110, bgeu 111
0110111	U	lui
1101111	UJ	jal

어셈블리에 레지스터를 쓰는 순서(칸 순서와 다르니 주의): R형 `add rd, rs1, rs2`, I형 `addi rd, rs1, 상수` · `lw rd, 상수(rs1)`, S형 `sw rs2, 상수(rs1)`, SB형 `bne rs1, rs2, 라벨`. 즉 분기는 앞 레지스터가 rs1, 뒤가 rs2다. [Ch2 p.14, 22, 47]

```
분기 오프셋(바이트) = 목표 주소 - 분기 명령어 자신의 주소
```

왜 자기 주소를 빼나: 분기 명령어를 실행하는 동안 PC에는 그 분기 명령어 자신의 주소가 들어 있고, 하드웨어는 목표 = PC + 오프셋으로 계산한다(PC 상대 주소 지정). [Ch2 p.48] 그래서 오프셋 = 목표 − PC. 단위는 바이트라 명령어 1개 = 4이고, 뒤(낮은 주소)로 가면 음수다.

왜 13비트이고 0번 비트를 버리나: 분기 목표는 짝수 주소뿐이라 오프셋의 0번 비트는 늘 0이다. [Ch2 p.47] 그래서 칸에는 imm[12:1] 12비트만 넣는다. 버린 0번 비트까지 치면 오프셋은 imm[12:0], 즉 13비트 2의 보수이고 12번 비트가 부호다. 음수 오프셋은 기초 · 2 방식(크기를 13비트로 쓰고 반전 + 1)으로 13비트를 만든다.

조각 배치: SB형 칸은 S형 칸과 자리가 같다. imm[10:5]는 S형 imm[11:5] 자리(30~25번)의 아래 6칸, imm[4:1]은 S형 imm[4:0] 자리(11~8번)의 위 4칸에 그대로 둔다. 남는 imm[11]은 S형에서 imm[0]이 있던 7번에, 부호 imm[12]는 모든 형식에서 부호 비트가 있는 31번에 둔다. 비트를 이렇게 같은 자리에 두면 하드웨어 배선이 단순해진다. 교재 기준

13비트를 왼쪽부터 12번 | 11번 | 10~5번(6개) | 4~1번(4개) | 0번으로 끊어 칸에 옮긴다. 예 오프셋 −8: 8 = 0 0000 0000 1000 → 반전 1 1111 1111 0111 → +1 → 1 1111 1111 1000. 끊으면 1 | 1 | 111111 | 1100 | 0 → imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 1100(0번 0은 버림).

되모으기(디코딩): 31번 → imm[12], 7번 → imm[11], 30~25번 → imm[10:5], 11~8번 → imm[4:1]을 꺼내 이 순서로 이어 쓰고 끝에 0을 붙여 13비트를 만든다. 12번 비트가 1이면 음수(반전 + 1로 크기). 목표 = 자기 주소 + 오프셋. 위 예를 되모으면 1 1 111111 1100 0 = 1 1111 1111 1000 → 반전 0 0000 0000 0111, +1 = 8 → −8.

범위	바이트 단위	word 단위
조건 분기(SB)	−4096 ~ 4094, 짝수만 [Ch2 p.47]	$\pm 2^{10}$ word [Ch2 p.48]
jal(UJ)	$\pm 2^{20}$ 바이트 = ±1 MiB	$\pm 2^{18}$ word [Ch2 p.48]

푸는 순서

인코딩 (어셈블리 → 메모리 바이트)

1. 표에서 형식·opcode·funct3·funct7을 옮긴다.
2. 레지스터 번호 → 5비트, 즉시값 → 2의 보수. 분기는 먼저 오프셋 계산.
3. 칸 순서대로 이어 쓴다(rs2가 rs1보다 왼쪽).
4. 4비트씩 끊어 16진수 8자리로.
5. 리틀 엔디언: 맨 오른쪽 바이트를 주소 +0에.

디코딩 (16진 → 어셈블리) [Ch2 p.50]

1. (메모리 덤프, 즉 메모리 바이트를 주소 순서로 늘어놓은 것이 주어지면 리틀 엔디언이라 바이트를 거꾸로 모아) 16진 → 2진.
2. 맨 오른쪽 7비트 opcode로 형식 결정(위 opcode 순 표).
3. 형식의 칸 폭대로 자른다.
4. funct3·funct7로 명령어를 고르고 위 "레지스터를 쓰는 순서"대로 쓴다(R형은 rd, rs1, rs2). SB형이면 즉시값 조각을 되모아 오프셋을 구한다.

SB 오프셋: 13비트 2의 보수로 쓴 뒤 imm[12] · imm[10:5] · imm[4:1] · imm[11]을 떼어 넣는다. 저장되는 imm[12:1] = 오프셋 ÷ 2.

함정

- `sw x9, 48(x22)` 의 x9는 rs2(저장할 값)다. imm[11:5]는 31~25번, imm[4:0]은 11~7번 비트.
- 오프셋은 바이트, 기준은 분기 명령어 자신의 주소. p.47 bne 칸 표의 opcode 1100111은 오타(정답 1100011, 1100111은 jalr).
- p.47은 바이트(−4096 ~ 4094), p.48은 word($\pm 2^{10}$, $\pm 2^{18}$). 위 끝은 4094(짝수만).

예제 (시험 수준)

(26-1 유형) 다음 두 명령어를 16진 기계어로 바꾸고, 주소 0x100 ~ 0x107에 리틀 엔디언으로 놓인 바이트를 쓰시오.

```
0x100  Loop: addi x5, x5, -1
0x104        bne  x5, x0, Loop
```

풀이 보기

addi = 0xFFF28293, bne = 0xFE029EE3. 바이트 0x100 ~ 0x107 = 93 82 F2 FF E3 9E 02 FE

① addi: I형, opcode 0010011, funct3 000. imm = −1 → 12비트 2의 보수 1111 1111 1111(1을 반전 1111 1111 1110, +1). rs1 = rd = 5 = 00101.

111111111111 imm = −1	00101 rs1 = 5	000 funct3	00101 rd = 5	0010011 opcode

② 이어 쓰고 4비트씩: 1111 1111 1111 0010 1000 0010 1001 0011 → F, F, F, 2, 8, 2, 9, 3 = 0xFFF28293.

③ bne: SB형, opcode 1100011, funct3 001. 오프셋 = Loop − 자기 주소 = 0x100 − 0x104 = −4(명령어 1개 뒤로).

④ −4를 13비트로: 4 = 0 0000 0000 0100, 반전 1 1111 1111 1011, +1 → 1 1111 1111 1100. 비트 번호별로 imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 1110, imm[0] = 0(저장 안 함). imm[12:1] = 111111111110(= −2 = −4 ÷ 2).

⑤ bne rs1, rs2이므로 rs1 = x5 = 00101, rs2 = x0 = 00000.

1 imm[12]	111111 imm[10:5]	00000 rs2 = 0	00101 rs1 = 5	001 funct3	1110 imm[4:1]	1 imm[11]	1100011 opcode

⑥ 이어 쓰고 4비트씩: 1111 1110 0000 0010 1001 1110 1110 0011 → F, E, 0, 2, 9, E, E, 3 = 0xFE029EE3.

⑦ 리틀 엔디언: 8자리를 두 자리씩(FF F2 82 93) 나눠 오른쪽 바이트부터 낮은 주소에. 0x100 = 93, 0x101 = 82, 0x102 = F2, 0x103 = FF, 0x104 = E3, 0x105 = 9E, 0x106 = 02, 0x107 = FE.

검산: 두 기계어를 bash 비트 연산(필드를 왼쪽으로 밀어 OR)으로 다시 만들어 같은 값을 얻었다.

회독 점검

1. (26-1 유형) `sw x18, 36(x2)`를 16진 기계어로 바꾸고, 주소 0x300부터 리틀 엔디언으로 놓인 바이트 4개를 쓰시오.

   답 보기

   0x03212223, 0x300 ~ 0x303 = 23 22 21 03

   ① S형, opcode 0100011, funct3 010. rs2 = 저장할 값 x18 = 10010, rs1 = 베이스 x2 = 00010.

   ② 36 = 32 + 4 → 12비트 0000 0010 0100. 아래 5비트 imm[4:0] = 00100, 위 7비트 imm[11:5] = 0000001. 검산 1 × 32 + 4 = 36.

   ③ 칸 순서 imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode: 0000001 10010 00010 010 00100 0100011.

   ④ 4비트씩: 0000 0011 0010 0001 0010 0010 0010 0011 → 0x03212223.

   ⑤ 바이트 03 21 22 23을 오른쪽부터 낮은 주소에: 0x300 = 23, 0x301 = 22, 0x302 = 21, 0x303 = 03.

---

2. (26-1 유형) RISC-V 메모리 0x500 ~ 0x503에 바이트 33 05 B5 40이 차례로 있다. 이 명령어를 어셈블리로 쓰시오.

   답 보기

   `sub x10, x10, x11`

   ① 리틀 엔디언이라 높은 주소부터 모은다: 40 B5 05 33 → 0x40B50533.

   ② 2진: 0100 0000 1011 0101 0000 0101 0011 0011. 맨 오른쪽 7비트 0110011 → R형.

   ③ R형 7 · 5 · 5 · 3 · 5 · 7로 자르기: 0100000 | 01011 | 01010 | 000 | 01010 | 0110011.

   ④ funct7 0100000 + funct3 000 → sub. rs2 = 01011 = 11, rs1 = 01010 = 10, rd = 01010 = 10.

   ⑤ 어셈블리 순서 sub rd, rs1, rs2 → sub x10, x10, x11.

---

3. (26-1 유형) 다음 코드에서 bge의 branch offset(바이트), 저장되는 imm[12:1], 16진 기계어를 쓰시오.

```
0x110  Loop: lw   x6, 0(x10)
...
0x124        bge  x6, x7, Loop
```

답 보기

오프셋 −20, imm[12:1] = 111111110110, 0xFE7356E3

① 오프셋 = 0x110 − 0x124. 0x110 = 272, 0x124 = 292 → −20(명령어 5개 뒤로).

② 13비트: 20 = 0 0000 0001 0100, 반전 1 1111 1110 1011, +1 → 1 1111 1110 1100. 0번 비트를 빼면 imm[12:1] = 111111110110(= −10 = −20 ÷ 2).

③ 조각: imm[12] = 1, imm[10:5] = 111111, imm[4:1] = 0110, imm[11] = 1. bge는 funct3 101, rs1 = x6 = 00110, rs2 = x7 = 00111.

④ 이어 쓰기: 1 111111 00111 00110 101 0110 1 1100011 → 1111 1110 0111 0011 0101 0110 1110 0011 = 0xFE7356E3.

---

4. (26-1 유형) 인코딩 형식 이름을 쓰시오: (a) slli (b) sra (c) jalr (d) jal (e) lui (f) sw (g) bltu

답 보기

(a) I형 (b) R형 (c) I형 (d) UJ형 (e) U형 (f) S형 (g) SB형

① slli는 시프트 양이 명령어 안의 상수(즉시값)라 I형. 표에 funct7 칸 값이 있어도 형식은 I형이다. sra는 시프트 양이 레지스터라 R형.

② jalr은 레지스터 + 12비트 상수라 I형, jal만 UJ형. lui는 U형.

③ sw는 store라 S형, bltu는 조건 분기라 SB형. [Ch2 p.50]

---

5. 빈칸: SB 형식 조건 분기는 (㉠) ~ (㉡) 바이트, (㉢)의 배수로만 분기한다. 슬라이드 p.48은 이를 $\pm 2^{(\text{㉣})}$ word로, jal은 $\pm 2^{(\text{㉤})}$ word로 적는다.

   답 보기

   ㉠ −4096 ㉡ 4094 ㉢ 2 ㉣ 10 ㉤ 18

   ① 즉시값 칸은 12비트지만 0번 비트(늘 0)를 빼고 imm[12:1]을 저장하므로 실제 오프셋은 13비트 2의 보수: $-2^{12} \sim 2^{12} - 1 = -4096 \sim 4095$. 짝수만 되므로 위 끝은 4094. [Ch2 p.47]

   ② word로 바꾸면 $4096 \div 4 = 1024 = 2^{10}$. jal은 21비트 오프셋 $\pm 2^{20}$ 바이트 $\div 4 = 2^{18}$ word. [Ch2 p.48]

6. 형식 칸 문제: (1) rd 칸이 없는 형식 두 가지 (2) 그 두 형식이 즉시값을 조각내 넣는 이유 (3) I형 즉시값 칸의 비트 수와 값의 범위 (4) 32비트 중 opcode 칸의 비트 번호

   답 보기

   (1) S형, SB형 (2) rs1·rs2·funct3·opcode 칸을 모든 형식에서 같은 자리에 두려고 (3) 12비트, −2048 ~ 2047 (4) 6~0번

   ① S형(sw)은 값을 메모리에 쓰기만 하고, SB형(beq 등)은 비교만 하므로 결과를 받을 레지스터 rd가 필요 없다. rd 자리(11~7번)는 즉시값 조각이 쓴다.

   ② 칸 자리가 같으면 하드웨어가 형식을 몰라도 레지스터 번호를 같은 곳에서 읽을 수 있다(설계 원칙 3, 형식은 최대한 비슷하게). [Ch2 p.23–24]

   ③ 12비트 2의 보수: $-2^{11} \sim 2^{11} - 1 = -2048 \sim 2047$.

   ④ opcode는 모든 형식에서 맨 오른쪽 7비트(6~0번)라, 디코딩은 여기부터 읽어 형식을 정한다.

더 자세히: 이론 노트 Ch2 · 5, 12, 13 / 실전 4

Ch2 · 5 슬라이드 p.2–51

Ch2 외울 문장

시험에 나오는 모양: 정의 빈칸, T/F, 나열(설계 원칙 3, 주소 지정 4, 번역 4단계), CISC·RISC 비교.

외울 것

- 명령어 = 프로세서 동작 하나를 기술하는 기본 명령, 하드웨어와 소프트웨어의 인터페이스. opcode(무엇을) + operand(대상). [Ch2 p.3]
- ISA = 프로세서가 이해하는 명령어 집합. 프로세서마다 달라도 비슷한 이유: ① 비슷한 원리의 하드웨어 기술 ② 모든 컴퓨터가 제공할 공통 기본 연산. [Ch2 p.3]
- 마이크로아키텍처 = 설계 제약·목표 아래 ISA의 구현. 소프트웨어에 드러나지 않음: 파이프라이닝, 추측 실행, 메모리 접근 스케줄링, 산술 장치. [Ch2 p.5]

	CISC (x86)	RISC (ARM, MIPS, SPARC, RISC-V)
명령어	복잡, 가변 길이, 마이크로코드(CPU 안에서 복잡한 명령어를 더 단순한 동작들로 풀어 놓은 것)로 변환	단순·규격화, 집합이 작음, 바로 실행
장점	어셈블리 프로그래밍 쉬움(컴파일러 단순), 명령어 메모리 적게 씀	CPU 설계 쉬움, 작은 집합 → 높은 클럭
단점	CPU 설계가 훨씬 어려움	코드가 길어짐(컴파일러 문제), 메모리를 많이 씀
기타	현대 x86 대부분은 RISC 기법으로 구현 [Ch2 p.6–7]	

- 설계 원칙 1: Simplicity favors regularity(단순성은 규칙성을 선호). 산술은 소스 2 + 목적지 1. [Ch2 p.9]
- 설계 원칙 2: Smaller is faster(작을수록 빠르다). 레지스터가 많으면 클럭 주기가 늘 수 있다. [Ch2 p.10]
- 설계 원칙 3: Good design demands good compromises(좋은 설계에는 좋은 타협). 형식이 여럿이면 해독은 복잡해도 모두 32비트, 형식은 최대한 비슷하게. [Ch2 p.23]
- addi는 가장 많이 쓰는 명령 중 하나. 즉시값은 상수를 메모리에서 읽는 것보다 훨씬 빠르고 에너지가 적다. [Ch2 p.16]
- 부호 있는 수 표현 3가지 = 부호-크기(sign and magnitude), 1의 보수(1's complement), 2의 보수(2's complement). 최종 채택 = 2의 보수(32비트 범위 -2^{31} ~ $2^{31} - 1$, MSB = 부호, 1이면 음수). [Ch2 p.19]
- 부정(= 부호 반전) = ~x + 1, 부호 확장 = 부호 비트를 왼쪽에 복제(값 보존). [Ch2 p.20]
- switch/case = 분기 주소 표(branch address table, branch table): 표에서 번호로 찾아 해당 코드로 바로 분기. [Ch2 p.31]
- lui(U형) = 20비트 상수를 12~31번 비트에, 오른쪽 12비트는 0. 먼 점프 = lui + jalr. [Ch2 p.46, 48]
- PC 상대를 쓰는 이유: 주소를 20비트 칸에 넣으면 프로그램이 2^{20} B(1 MiB)로 제한된다. 분기 목표는 대부분 가깝다(if/for/while) → PC가 이상적인 기준. [Ch2 p.48]

주소 지정 방식	피연산자·주소	명령어
즉시값(immediate)	명령어 안의 상수	addi, xori, srai
레지스터(register)	레지스터	add, sub, xor
베이스/변위(base or displacement)	메모리, 주소 = 레지스터 + 상수	lw, sw, **jalr**
PC 상대(PC-relative)	분기 주소 = PC + 상수	beq, bne, jal
한 연산이 여러 방식을 쓸 수 있다(덧셈 = add/addi) [Ch2 p.49]		

프로그램 번역 4단계: 컴파일러 → 어셈블러 → 링커 → 로더. 컴파일러 = C → 어셈블리, 어셈블러 = 어셈블리 → 기계어(목적 파일, object file), 링커 = 여러 목적 파일을 합쳐 실행 파일로, 로더 = 실행 파일을 메모리에 올려 실행을 시작. [Ch2 p.51]

함정

- jalr은 jump라는 이름과 달리 레지스터 + 상수라 베이스 방식. PC 상대는 jal.
- RISC 단점에 "메모리를 많이 씀"이 있다. 레지스터가 많을수록 무조건 빠른 것은 아니다(원칙 2).

회독 점검

1. O/X: 추측 실행(speculative execution)은 소프트웨어에 드러나는 ISA의 기능이다.

   답 보기

   X

   ① 소프트웨어에 드러나지 않고 하드웨어 안에서 하는 일이므로 마이크로아키텍처에 속한다. [Ch2 p.5]

2. 설계 원칙 3가지를 영어로 쓰고, 원칙 3의 예를 한 줄로 쓰시오.

   답 보기

   Simplicity favors regularity / Smaller is faster / Good design demands good compromises

   ① 원칙 3의 예: 형식을 여러 개(R·I·S …) 두어 해독은 복잡해졌지만 모든 명령어를 32비트로 같게 유지했다. [Ch2 p.9, 10, 23]

3. 다음 명령어의 주소 지정 방식은? (a) jalr (b) bne (c) xori (d) sw (e) sub

답 보기

(a) 베이스(변위) (b) PC 상대 (c) 즉시값 (d) 베이스(변위) (e) 레지스터

① jalr과 sw는 둘 다 "레지스터 + 상수"로 주소를 만든다. bne는 PC + 상수. [Ch2 p.49]

---

4. 분기에 PC 상대 주소 지정을 쓰는 이유를 두 가지 쓰시오.

답 보기

① 주소를 20비트 칸에 직접 넣으면 프로그램이 2^{20} B(1 MiB)보다 클 수 없다. ② 대부분의 분기 목표가 분기 명령어 가까이(if/for/while) 있어 PC를 기준으로 삼는 것이 이상적이다.

그래서 분기 $\pm 2^{10}$ word, 점프 $\pm 2^{18}$ word 안에서 이동한다. [Ch2 p.48]

---

5. CISC의 장점 두 가지와 RISC의 단점 두 가지를 쓰시오.

답 보기

CISC 장점: 어셈블리 프로그래밍이 쉬움(컴파일러도 단순), 명령어 메모리 사용 감소. RISC 단점: 어셈블리 코드가 길어짐, 메모리를 많이 씀.

① CISC는 명령어 하나가 일을 많이 해 코드가 짧고, RISC는 같은 일에 명령어가 더 필요하다. [Ch2 p.6–7]

---

6. 빈칸: C 코드가 실행되기까지 (㉠) → (㉡) → (㉢) → (㉣). 즉시값 명령어는 상수를 메모리에서 읽는 것보다 (㉤)고 에너지가 적게 든다.

답 보기

㉠ 컴파일러 ㉡ 어셈블러 ㉢ 링커 ㉣ 로더 ㉤ 훨씬 빠르

① 어셈블러가 목적 파일을 만들어야 링커가 합칠 수 있고, 메모리에 올리는 것은 로더다. [Ch2 p.51, 16]

더 자세히: 이론 노트 Ch2 · 1, 2, 4, 5, 12, 13, 14 / 실전 8

Ch3 · 1 슬라이드 p.4–20

오버플로·곱셈·나눗셈

시험에 나오는 모양: 오버플로 판정(O/X, 조건 빈칸), 곱셈·나눗셈 반복 표 빈칸, 곱셈 upper·lower 표, 부호 있는 나눗셈의 몫·나머지. (기출: 21-2 부호 없는/있는/부호 다른 곱셈 upper·lower 빈칸)

먼저 알아 둘 말

- A × B에서 곱해지는 수 A가 **피승수(multiplicand)**, 곱하는 수 B가 **승수(multiplier)**, 결과가 **곱(product)**. A ÷ B에서 나눠지는 수가 **피제수(dividend)**, 나누는 수가 **제수(divisor)**, 결과가 **몫(quotient)**과 **나머지(remainder)**. [Ch3 p.7, 14]
- 2진 손 곱셈: 승수의 비트를 오른쪽부터 보며 1인 자리마다 피승수를 그 자리만큼 왼쪽으로 민 값을 더한다(0인 자리는 더하지 않음). 곱셈기는 이것을 "LSB 검사 → 더하기 → 시프트"로 한 비트씩 반복하는 하드웨어다. [Ch3 p.7–8]
- **ALU(산술 논리 장치, Arithmetic Logic Unit)** = 덧셈·뺄셈·AND·OR 같은 계산을 하는 회로. [Ch3 p.8]
- "1판"은 슬라이드의 첫 번째 하드웨어 설계, "개선판"은 레지스터·ALU 크기를 줄인 다음 설계다. 표의 "반복"은 한 비트를 처리하는 한 바퀴다.

외울 것

오버플로(overflow) = 결과를 정해진 비트 수의 하드웨어로 표현할 수 없는 것. 2의 보수에서는 결과의 부호가 뒤집혀 보인다. [Ch3 p.4–5]

연산	A	B	이런 결과면 오버플로	기호
A + B	≥ 0	≥ 0	< 0	+ + + = −
A + B	< 0	< 0	≥ 0	− + − = +
A − B	≥ 0	< 0	< 0	+ − − = −
A − B	< 0	≥ 0	≥ 0	− − + = +

왜 이 네 경우인가 (8비트로 설명, 범위 −128 ~ 127은 기초 · 2)

1. **칸 밖 자리올림은 버린다**: 8비트 덧셈기는 결과도 8비트만 남긴다. 맨 왼쪽에서 생긴 9번째 비트(자리올림, carry)는 버린다. 2의 보수는 이렇게 버려야 맞는 답이 나오도록 만든 표현이다: −1 + 1 = 1111 1111 + 0000 0001 = 1 0000 0000 → 9번째 1을 버리면 0000 0000 = 0(정답). 그래서 자리올림이 나왔다는 것만으로는 오버플로가 아니다. PDF 외 보충
2. **오버플로 = 진짜 답이 범위 밖**: 양수 + 양수의 진짜 답이 127보다 크면, 8비트에 남는 패턴은 진짜 답 − 256이라 음수로 읽힌다. 예 100 + 100 = 200 → 0110 0100 + 0110 0100 = 1100 1000 = −56. 음수 + 음수의 진짜 답이 −128보다 작으면 진짜 답 + 256이 남아 양수로 읽힌다. 예 (−100) + (−100) = −200 → 56. 그래서 "부호가 같은 두 수를 더했는데 결과 부호가 다르다"가 오버플로의 표시다. [Ch3 p.5]
3. **부호가 다른 덧셈은 안전**: 합이 두 수 사이에 있다(−10 < −10 + 4 = −6 < 4). 두 수가 이미 범위 안이니 그 사이 값도 범위 안이다. [Ch3 p.4]
4. **뺄셈은 덧셈으로 바꿔 본다**: A − B = A + (−B)(−B는 기초 · 2의 부정). A ≥ 0, B < 0이면 −B > 0이라 양수 + 양수(표 셋째 줄). A < 0, B ≥ 0이면 −B ≤ 0이라 음수 + 음수(넷째 줄). A와 B 부호가 같으면 부호가 다른 덧셈이 되어 생기지 않는다. 그래서 표는 "부호가 같은 덧셈 2줄 + 그것으로 바뀌는 뺄셈 2줄"이다. [Ch3 p.4–5]

- **생기지 않는 경우**: 부호가 다른 두 수의 덧셈(합이 두 수 사이에 있음), 부호가 같은 두 수의 뺄셈(c − a = c + (−a)라 부호가 다른 덧셈이 됨). [Ch3 p.4]
- **무부호(unsigned)**(주소, 보통 무시): 덧셈 A + B = S에서 S < A 또는 S < B, 뺄셈 A − B = S에서 S > A면 오버플로. [Ch3 p.6] 이유: 무부호 8비트(0 ~ 255)에서 넘치면 진짜 답 − 256이 남아 원래 수보다 작아진다 (200 + 100 = 300 → 44 < 200). 뺄셈이 0 밑으로 내려가면 진짜 답 + 256이 남아 빼기 전보다 커진다(50 − 100 = −50 → 206 > 50). PDF 외 보충
- **언어**: C/C++·Java는 무시, Ada·Fortran은 통보 요구. [Ch3 p.6]

곱의 길이 = 피승수(multiplicand) 길이 + 승수(multiplier) 길이 (32 × 32 → 64비트) [Ch3 p.7]

부품	곱셈기 1판 [p.8–9]	개선판 [p.10–11]
승수 레지스터	32비트, 오른쪽 시프트	없음(곱 레지스터 오른쪽 절반에 넣음)
피승수 레지스터	64비트(오른쪽 절반에서 시작), 왼쪽 시프트	32비트, 시프트 안 함
ALU	64비트	32비트
곱 레지스터	64비트, 처음 0	64비트, 왼쪽 절반 0 + 오른쪽 절반 승수
한 번의 반복	Test-Add-Shift: 승수 LSB(맨 오른쪽 비트, 기초 · 2)가 1이면 곱 = 곱 + 피승수 → 피승수 왼쪽 시프트 → 승수 오른쪽 시프트	곱 레지스터 LSB가 1이면 왼쪽 절반에 피승수를 더함 → 곱 레지스터 전체 오른쪽 시프트

명령	돌려주는 것 (32 × 32 = 64비트 곱) [p.13]
`mul`	아래(lower) 32비트. 부호 해석과 상관없이 같음
`mulh`	위(upper) 32비트, 둘 다 부호 있음
`mulhu`	위 32비트, 둘 다 무부호
`mulhsu`	위 32비트, 하나는 부호 있음·하나는 무부호 (rs1 부호 있음, rs2 무부호 PDF 외 보충)

Mode (3비트 예)	A	B	A × B	Upper(2진)	Lower(2진)
Unsigned	5 (101_2)	5 (101_2)	25 (011001_2)	011	001
Signed	−3 (101_2)	−3 (101_2)	9 (001001_2)	001	001
Signed-Unsigned	−3 (101_2)	5 (101_2)	−15 (110001_2)	110	001

부호 있는 곱셈 = 양수로 바꿔 31번 반복, 부호가 다르면 곱을 부정(= 부호 반전). 31번인 이유: 32비트 중 맨 앞 부호 비트는 따로 처리하고 크기 31비트만 곱하기 때문이다(승수 비트 하나에 한 바퀴). 빠른 곱셈 = 여러 덧셈기의 병렬 트리(비용과 성능의 맞바꿈). [Ch3 p.12–13]

피제수 = 몫 × 제수 + 나머지, 제수 ≠ 0. 나눗셈기 1판 = 곱셈기 1판과 같은 하드웨어, 시프트 방향 반대, 뺄셈(Subtract-Test-Shift). 제수는 왼쪽 절반에서 시작, 나머지 레지스터는 피제수로 초기화. 한 번의 반복 = ① 나머지 − 제수 ② 결과가 음수(MSB 1)면 제수를 다시 더해 **복원**, 몫 왼쪽 시프트·끝 비트 0 / 0 이상이면 몫 왼쪽 시프트·끝 비트 1 ③ 제수 오른쪽 시프트. 몫의 끝 비트(0번 비트)를 아래 표에서 Q0라 쓴다. **n+1번** 반복(32비트 33번, 4비트 5번). [Ch3 p.14–16]

나눗셈기 1판의 각 동작이 왜 그런가 (n = 제수 비트 수, 아래 예는 n = 4)

1. **손 나눗셈과 같은 일**: 손으로 나눌 때는 제수를 피제수의 왼쪽 끝에 맞춰 대 보고, 들어가면 몫 자리에 1을 쓰고 빼고, 안 들어가면 0을 쓴 뒤 제수를 한 자리 오른쪽으로 옮겨 다시 대 본다. [Ch3 p.14] 하드웨어는 이것을 "일단 빼 보고, 안 들어갔으면 되돌린다"로 한다.
2. **제수를 왼쪽 절반에 넣는 이유**: 4비트 제수 0010을 8비트 레지스터 왼쪽 절반에 넣으면 0010 0000 = 2×2^4 = 32다(왼쪽 4칸 시프트 = $\times 2^4$, 기초 · 3). 가장 높은 자리부터 대 보기 위해서다. 매 반복 끝에 오른쪽으로 1칸 밀면(÷ 2) $\times 2^3$, $\times 2^2$, $\times 2^1$, $\times 2^0$ 자리로 차례로 내려온다.
3. **n+1번인 이유**: 제수가 $\times 2^4$부터 $\times 2^0$까지 자리 5개(4, 3, 2, 1, 0)를 하나씩 시험하므로 4 + 1 = 5번이다.
4. **음수 판정**: 하드웨어는 뺀 결과의 MSB(부호 비트)를 본다. 손으로는 10진으로 "지금 나머지 < 지금 제수"면 음수라고 바로 알 수 있다(7 < 32 → 음수). 8비트 패턴이 필요하면 기초 · 2의 일반형대로 256 − |값|을 2진으로 쓴다: 7 − 32 = −25 → 256 − 25 = 231 = 1110 0111.
5. **복원**: 음수면 제수가 "안 들어간" 것이므로 뺀 것을 취소한다. 제수를 다시 더하면 빼기 전 나머지로 돌아온다. 몫 비트는 0.
6. **몫 왼쪽 시프트**: 몫 비트는 높은 자리부터 하나씩 나온다. 새 비트를 오른쪽 끝(Q0)에 붙이고 먼저 나온 비트를 한 칸 왼쪽으로 밀면, 글씨를 왼쪽부터 써 나가는 것과 같은 순서가 된다.

1판 반복 표 (슬라이드 예 $0000\ 0111_2 \div 0010_2 = 7 \div 2$, 각 반복이 끝난 뒤 값) [Ch3 p.16]

반복	단계: 빼기 결과 → 판정	몫(2진)	제수(2진)	나머지(2진)
0	초기값	0000	0010 0000	0000 0111
1	7 − 32 = 1110 0111(음수) → 복원, Q0 = 0	0000	0001 0000	0000 0111
2	7 − 16 = 1111 0111(음수) → 복원, Q0 = 0	0000	0000 1000	0000 0111
3	7 − 8 = 1111 1111(음수) → 복원, Q0 = 0	0000	0000 0100	0000 0111
4	7 − 4 = 0000 0011(≥ 0) → Q0 = 1	0001	0000 0010	0000 0011
5	3 − 2 = 0000 0001(≥ 0) → Q0 = 1	0011	0000 0001	0000 0001

최종 몫 0011_2 = 3, 나머지 $0000\ 0001_2$ = 1 (3 × 2 + 1 = 7). 개선판: 몫은 나머지 레지스터 오른쪽 절반에 쌓이고, 끝나면 왼쪽 절반만 오른쪽으로 1칸. 같은 예의 최종 레지스터 0001 0011(왼쪽 나머지 1, 오른쪽 몫 3). 마지막에 왼쪽 절반만 되돌리는 이유: 개선판은 제수를 오른쪽으로 미는 대신 나머지 레지스터를 매번 왼쪽으로 미는데, 마지막 반복에서도 한 번 더 밀려 나머지가 한 칸 왼쪽에 와 있기 때문이다. [Ch3 p.17–18]

식	몫	나머지	검산
−7 ÷ 2	−3	−1	−3 × 2 − 1 = −7
7 ÷ −2	−3	1	−3 × (−2) + 1 = 7
−7 ÷ −2	3	−1	3 × (−2) − 1 = −7

규칙: 절댓값으로 나눔 → 두 부호가 다르면 몫 부정 → **나머지 부호 = 피제수 부호**. −7 ÷ 2를 몫 −4·나머지 1로 하지 않는다. [Ch3 p.19] 이 규칙의 좋은 점: 부호가 어떻게 섞여도 몫과 나머지의 **크기**는 양수끼리 나눈 7 ÷ 2 = 3 … 1과 같고 부호만 바뀐다(위 표의 세 줄 모두 3과 1). 그래서 절댓값으로 한 번 나누고 부호만 붙이면 된다. C의 % 연산도 이 규칙을 따른다. [Ch3 p.19] 나눗셈은 뺄셈이 앞 나머지 부호에 달린 조건부라 병렬화 불가. SRT = 표를 보고 한 단계에 몫 비트 여러 개 예측. 나눗셈 명령어: div(몫)·rem(나머지), 끝의 u는 무부호판(divu·remu). [Ch3 p.20]

푸는 순서

곱셈 반복 표 (슬라이드 예 $0010_2 \times 0011_2$, 각 반복이 끝난 뒤 값) [Ch3 p.9, 11]

반복	1판 승수(2진)	1판 피승수(2진)	1판 곱(2진)	개선판 곱 레지스터(2진)
0 (초기)	0011	0000 0010	0000 0000	0000 0011
1	0001	0000 0100	0000 0010	0001 0001
2	0000	0000 1000	0000 0110	0001 1000
3	0000	0001 0000	0000 0110	0000 1100
4	0000	0010 0000	0000 0110	0000 0110

1판 표 읽는 법: 피승수 0010(2), 승수 0011(3). 한 반복 = ① 승수의 LSB가 1이면 곱 = 곱 + 피승수 ② 피승수를 왼쪽 1칸 ③ 승수를 오른쪽 1칸. [Ch3 p.9]

- **피승수를 왼쪽으로 미는 이유**: 손 곱셈에서 승수의 다음 자리에 해당하는 피승수 복사본은 한 칸 더 왼쪽에 놓인다(위 "먼저 알아 둘 말"). 하드웨어는 매번 피승수를 왼쪽 1칸 밀어 그 위치를 미리 만들어 둔다. 최대 n칸까지 밀리므로 피승수 레지스터는 2n비트(오른쪽 절반에서 시작)다.
- **승수를 오른쪽으로 미는 이유**: 다음에 볼 승수 비트가 항상 LSB 자리로 오게 하려는 것이다. 그래서 하드웨어는 LSB 한 칸만 검사하면 된다.
- 반복 1: 승수 0011의 LSB 1 → 곱 0000 0000 + 0000 0010 = 0000 0010. 피승수 → 0000 0100, 승수 → 0001.

- 반복 2: 승수 0001의 LSB 1 → 곱 0000 0010 + 0000 0100 = 0000 0110. 피승수 → 0000 1000, 승수 → 0000.
- 반복 3, 4: LSB 0 → 더하지 않고 두 시프트만. 곱은 0000 0110 = 6 = 2 × 3 그대로. 승수 비트가 4개라 4번(n번)에서 끝난다.

개선판 표 읽는 법: 곱 레지스터 = 왼쪽 절반 0000 + 오른쪽 절반 승수 0011 → 0000 0011. 한 반복 = ① 곱 레지스터 LSB(= 지금 볼 승수 비트)가 1이면 왼쪽 절반 = 왼쪽 절반 + 피승수 ② 곱 레지스터 전체를 오른쪽 1칸(왼쪽 빈칸에 0). [Ch3 p.11]

- **피승수 대신 곱을 오른쪽으로 미는 이유**: 피승수를 왼쪽으로 미는 것과 곱을 오른쪽으로 미는 것은 둘 사이의 상대 위치가 같아서 결과가 같다. 그러면 피승수는 움직이지 않으니 32비트 레지스터·32비트 ALU면 된다. 오른쪽으로 밀 때 다 쓴 승수 비트가 오른쪽 끝으로 빠지고 다음 승수 비트가 LSB에 온다. 곱은 왼쪽부터 채워져 내려온다.
- 반복 1: LSB 1 → 왼쪽 0000 + 0010 = 0010 → 0010 0011 → 오른쪽 1칸 → 0001 0001.
- 반복 2: LSB 1 → 왼쪽 0001 + 0010 = 0011 → 0011 0001 → 0001 1000.
- 반복 3: LSB 0 → 시프트만 → 0000 1100. 반복 4: LSB 0 → 0000 0110 = 6.
- 왼쪽 절반을 더할 때 4비트를 넘는 자리올림이 생기면(예 1111 × 1111) 그 올림 비트는 버리지 않고 시프트 때 맨 왼쪽 빈칸으로 들어간다. 시험 예처럼 올림이 없으면 빈칸은 0이다. 교재 기준

upper·lower 표

1. n비트를 무부호 값과 부호 값으로 읽는다(맨 앞 1이면 -2^n). 이유: 기초 · 2 일반형대로 −a의 패턴을 무부호로 읽으면 $2^n - a$이므로, 부호 값 = 무부호 값 − 2^n이다. 3비트 101_2은 무부호 5, 부호 5 − 8 = −3.
2. 모드별 10진 곱: Unsigned = 무부호 × 무부호, Signed = 부호 × 부호, Signed-Unsigned = A(부호) × B(무부호).
3. 곱을 2n비트로 쓴다(곱의 길이 = 두 수 길이의 합이라 2n비트면 어느 모드든 다 들어간다). 양수는 자릿값 분해로 쓰고 왼쪽 빈자리를 0으로 채운다. 음수는 2n비트 2의 보수로 써야 하므로 같은 일반형을 2n비트에 적용해 2^{2n} + 곱(6비트면 64 + 곱)을 2진으로 쓴다. 예 −15 → 64 − 15 = 49 = 32 + 16 + 1 = 110001_2.
4. 왼쪽 n비트 = Upper, 오른쪽 n비트 = Lower. 검산: Lower가 세 줄 모두 같아야 한다. 이유: 부호 값과 무부호 값은 2^n(3비트면 8)만큼 차이 나므로 모드별 곱은 8의 배수만큼만 다르다. 8의 배수를 더하거나 빼도 2진 아래 3자리는 변하지 않는다(10진에서 1000의 배수를 더해도 아래 세 자리가 그대로인 것과 같음). 예 25 − 9 = 16 = 2 × 8.

함정

- 2의 보수(부호 있는 수)에서 맨 왼쪽 밖으로 나가는 자리올림은 오버플로가 아니다. 피연산자 부호와 결과 부호로 판정한다.
- 곱셈은 n번(부호 있는 곱셈은 31번), 나눗셈은 n+1번. 나머지가 음수면 몫 비트 0과 함께 반드시 복원한다.
- `mulhu`는 위 32비트다. 아래 32비트는 부호와 관계없이 `mul` 하나.

예제 (시험 수준)

3비트 A = 110_2, B = 101_2을 곱해 6비트 곱을 얻는다. Unsigned, Signed, Signed-Unsigned(A 부호 있음, B 무부호) 각각의 A × B(10진, 6비트)와 Upper, Lower를 쓰시오.

풀이 보기

Unsigned 30 (011110_2) Upper 011_2 Lower 110_2 / Signed 6 (000110_2) Upper 000_2 Lower 110_2 / Signed-Unsigned −10 (110110_2) Upper 110_2 Lower 110_2

① 비트 읽기: 110_2의 무부호 값 = 4 + 2 = 6. 맨 앞이 1이라 부호 값 = 6 − 8 = −2. 101_2의 무부호 값 = 4 + 1 = 5, 부호 값 = 5 − 8 = −3.

② 모드별 곱: Unsigned 6 × 5 = 30. Signed (−2) × (−3) = 6. Signed-Unsigned는 A를 부호(−2), B를 무부호(5)로 읽어 (−2) × 5 = −10.

③ 6비트로: 30 = 16 + 8 + 4 + 2 = 011110_2. 6 = 4 + 2 = 000110_2. −10은 음수라 2^6 + (−10) = 64 − 10 = 54 = 32 + 16 + 4 + 2 = 110110_2.

④ 반으로 자르기: 011|110_2, 000|110_2, 110|110_2. 왼쪽 3비트가 Upper, 오른쪽 3비트가 Lower.

검산: Lower가 셋 다 110_2으로 같다. 30을 8로 나눈 나머지 6 = 110_2과도 맞다. 32비트였다면 Upper는 차례로 `mulhu`, `mulh`, `mulhsu`, Lower는 `mul`의 결과다. [Ch3 p.13]

회독 점검

1. 8비트 2의 보수에서 (1) 1001 0000 + 1010 0000 (2) 0101 0000 − 1100 0000의 8비트 결과와 오버플로 여부를 쓰시오.

답 보기

(1) 0011 0000, 오버플로 (2) 1001 0000, 오버플로

① (1) 부호: 두 수 모두 MSB 1이라 음수(−112, −96). 음수 + 음수라 오버플로를 의심한다.

② 더하기: 1001 0000 + 1010 0000 = 1 0011 0000. 9번째 비트는 칸 밖이라 버리고 0011 0000(= 48, 양수)이 남는다. 음수끼리 더했는데 양수(− + − = +) → 오버플로. 실제 답 −208은 8비트 최솟값 −128보다 작다.

③ (2) 부호: 0101 0000(80, 양수) − 1100 0000(−64, 음수). 양수 − 음수라 표의 셋째 줄을 의심한다.

④ 뺄셈을 덧셈으로: −(−64) = 64 = 0100 0000. 0101 0000 + 0100 0000 = 1001 0000. MSB 1이라 음수(−112)로 읽힌다. + − − = − → 오버플로. 실제 답 144는 최댓값 127보다 크다. [Ch3 p.5]

---

2. 3비트 A = 101_2, B = 111_2의 곱셈 표에서 빈칸을 채우시오. Unsigned: A × B = (ㄱ), Upper (ㄴ), Lower 011_2 / Signed: A × B = (ㄷ), Upper 000_2 / Signed-Unsigned: A × B = (ㄹ), Upper (ㅁ), Lower (ㅂ)

답 보기

(ㄱ) 35 (100011_2) (ㄴ) 100_2 (ㄷ) 3 (000011_2) (ㄹ) −21 (101011_2) (ㅁ) 101_2 (ㅂ) 011_2

① 읽기: 101_2 = 무부호 5, 부호 5 − 8 = −3. 111_2 = 무부호 7, 부호 7 − 8 = −1.

② Unsigned 5 × 7 = 35 = 32 + 2 + 1 = 100011_2 → Upper 100_2, Lower 011_2. 무부호라 맨 앞 1이어도 음수가 아니다.

③ Signed (−3) × (−1) = 3 = 000011_2 → Upper 000_2.

④ Signed-Unsigned (−3) × 7 = −21. 음수라 64 − 21 = 43 = 32 + 8 + 2 + 1 = 101011_2 → Upper 101_2, Lower 011_2.

검산: Lower 011_2이 세 줄 공통. [Ch3 p.13]

---

3. 슬라이드 규칙으로 −11 ÷ 4, 11 ÷ −4, −11 ÷ −4의 몫과 나머지를 쓰시오.

답 보기

−11 ÷ 4 → 몫 −2, 나머지 −3 / 11 ÷ −4 → 몫 −2, 나머지 3 / −11 ÷ −4 → 몫 2, 나머지 −3

① 절댓값으로 나눈다: 11 ÷ 4. 4 × 2 = 8 ≤ 11 < 12 = 4 × 3이라 몫 2, 나머지 11 − 8 = 3.

② 몫 부호: 앞의 두 식은 피제수와 제수 부호가 달라 −2, 셋째 식은 둘 다 음수라 2.

③ 나머지 부호는 피제수를 따른다: 피제수가 −11이면 −3, 11이면 3.

검산(몫 × 제수 + 나머지): −2 × 4 − 3 = −11, −2 × (−4) + 3 = 11, 2 × (−4) − 3 = −11. [Ch3 p.19]

4. 개선판 곱셈기로 $0101_2 \times 0011_2$(5 × 3)을 계산한다. 반복 2가 끝난 뒤(시프트 후) 곱 레지스터 값과 최종 값을 쓰시오.

답 보기

반복 2 뒤 0011 1100, 최종 0000 1111 (= 15)

① 초기값: 곱 레지스터 = 왼쪽 0000 + 오른쪽 승수 0011 → 0000 0011.

② 반복 1: LSB 1 → 왼쪽 절반 0000_2 + 피승수 0101_2 = 0101_2 → 0101 0011. 오른쪽 시프트(왼쪽에 0) → 0010 1001.

③ 반복 2: LSB 1 → 왼쪽 절반 0010_2 + 0101_2 = 0111_2(2 + 5 = 7) → 0111 1001. 오른쪽 시프트 → 0011 1100.

④ 반복 3: LSB 0 → 더하지 않고 시프트 → 0001 1110. 반복 4: LSB 0 → 시프트 → 0000 1111.

검산: 0000 1111 = 8 + 4 + 2 + 1 = 15 = 5 × 3. [Ch3 p.11]

5. O/X: (1) 부호가 다른 두 수의 덧셈은 오버플로가 생길 수 있다. (2) 나눗셈기 1판으로 4비트 나눗셈을 하면 5번 반복한다. (3) 나눗셈도 곱셈처럼 여러 덧셈기(뺄셈기)를 병렬로 써서 빠르게 할 수 있다.

   답 보기

   (1) X (2) O (3) X

   ① (1) 부호가 다른 두 수의 합은 두 수 사이에 있다(예 −10 + 4 = −6). 두 수가 이미 칸에 들어가 있으니 그 사이 값도 들어간다. [Ch3 p.4]

   ② (2) 제수가 n칸 왼쪽(왼쪽 절반)에서 출발해 n칸부터 0칸까지 n+1개 자리를 시험하므로 4 + 1 = 5번. [Ch3 p.16]

   ③ (3) 다음 뺄셈을 할지가 앞 단계 나머지의 부호에 달려 있어(조건부) 단계를 동시에 할 수 없다. 대신 SRT로 몫 비트 여러 개를 예측한다. [Ch3 p.20]

6. 나눗셈기 1판으로 $0000\ 1101_2 \div 0011_2$(13 ÷ 3)을 계산한다. 반복 표에서 반복 3이 끝난 뒤의 몫·제수·나머지 값과, 최종 몫·나머지를 쓰시오.

   답 보기

   반복 3 뒤 몫 0001_2, 제수 0000 0110, 나머지 0000 0001 / 최종 몫 0100_2(4), 나머지 0000 0001(1)

   ① 초기값: 몫 0000, 제수 0011_2을 왼쪽 절반에 넣어 0011 0000(48), 나머지 = 피제수 0000 1101(13).

   ② 반복 1: 13 < 48이라 음수가 나온다. 8비트 패턴은 256 − 35 = 221 = 128 + 64 + 16 + 8 + 4 + 1이라 13 − 48 = −35 = 1101 1101(MSB 1, 음수) → 제수를 다시 더해 0000 1101로 복원, 몫 왼쪽 시프트·끝 비트 0 → 0000. 제수 오른쪽 시프트 → 0001 1000(24).

   ③ 반복 2: 13 − 24 = −11 = 1111 0101(음수) → 복원, 몫 0000. 제수 → 0000 1100(12).

   ④ 반복 3: 13 − 12 = 1 = 0000 0001(MSB 0, ≥ 0) → 복원하지 않음, 몫 왼쪽 시프트·끝 비트 1 → 0001. 제수 → 0000 0110(6). 여기까지가 반복 3의 답.

   ⑤ 반복 4: 1 − 6 = −5(음수) → 복원, 몫 0010, 제수 0000 0011. 반복 5: 1 − 3 = −2(음수) → 복원, 몫 0100, 제수 0000 0001. n + 1 = 5번에서 끝.

   검산: 몫 0100_2 = 4, 나머지 1. 4 × 3 + 1 = 13. [Ch3 p.16]

더 자세히: 이론 노트 Ch3 · 1~5 / 실전 5

Ch3 · 2 슬라이드 p.27–31

IEEE 754 변환

시험에 나오는 모양: 10진 → 2진 정규화 → IEEE 754 16진수, 16진수 → 10진, 지수 칸 값 계산, 16비트(1-5-10) 표현. (기출: 26-1 10진 → 2진, A = −0.5·B = 2.625 표현, 16비트 1-5-10)

먼저 알아 둘 말

- **정규화(normalized)** = 소수점 왼쪽에 0이 아닌 숫자가 딱 한 자리인 과학적 표기. 10진 1.0×10^{-9}처럼, 2진이면 맨 앞이 늘 1이라 $1.xxx_2 \times 2^n$ 꼴이 된다. [Ch3 p.23–24]
- **부동소수점(floating point)** = 소수점 위치를 고정하지 않고 지수 n으로 나타내는 수 표현. 정규화 꼴의 xxx 부분이 **가수(fraction)**, n이 **지수(exponent)**다. [Ch3 p.24]
- **IEEE 754** = 부동소수점을 비트로 저장하는 표준. 32비트 **단정도(single precision)**와 64비트 **배정도(double precision)**가 있다. [Ch3 p.27]
- **bias** = 지수를 저장할 때 더해 두는 고정값(단정도 127). 저장된 값을 **지수 칸**, 원래 n을 **실제 지수**라 구별해 부른다. [Ch3 p.28]

외울 것

형식	부호	지수	가수(Fraction)	bias	정규화 수의 지수 칸 / 실제 지수
단정도(32비트)	1	8	23	127	1 ~ 254 / −126 ~ 127
배정도(64비트)	1	11	52	1023	1 ~ 2046 / −1022 ~ 1023
16비트 PDF 외 보충	1	5	10	15	1 ~ 30 / −14 ~ 15

$$\text{값} = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{지수 칸} - \text{bias})}$$

- **숨은 1(hidden bit)**: 정규화된 2진수는 소수점 앞이 항상 1이라 저장하지 않는다. 가수 칸 = 소수점 뒤 비트만. 그래서 단정도 정밀도는 1 + 23 = 24비트. 유효숫자(significand) = 1 + Fraction. [Ch3 p.28]
- **지수 칸 = 실제 지수 + bias**. 지수 칸이 늘 무부호가 된다. [Ch3 p.28]
- **예약**: 지수 칸 전부 0과 전부 1(단정도 0·255, 16비트 0·31)은 예약값이다. 특수 기호 예: NaN(0/0 같은 잘못된 연산). [Ch3 p.30] 나머지 쓰임은 0·비정규화 수(전부 0), ±무한대(전부 1). PDF 외 보충
- **16비트**는 슬라이드에 없지만 26-1에 나왔다. 규칙은 같고 bias = $2^{(k-1)} - 1$(k = 지수 칸 비트 수, 5 → 15), 16진 4자리. PDF 외 보충

슬라이드 예: $-0.75 = -0.11_2 = -1.1_2 \times 2^{-1}$ → S = 1, 지수 칸 −1 + 127 = 126, 가수 칸 1000…0 → **0xBF400000** [Ch3 p.31]

1	01111110	10000000000000000000000
S	지수 칸 126	Fraction

푸는 순서

10진 → IEEE 754

1. 부호를 뗀다. 음수면 S = 1.
2. 크기를 2진으로. 정수 부분은 ÷2 반복(몫 0까지, 나머지를 아래 → 위로 읽기) 또는 자릿값 합으로 분해(예 9 = 8 + 1). 소수 부분은 ×2 반복, 나온 정수 부분(0 또는 1)을 위 → 아래로 읽기. 소수 부분이 0이 되면 끝.

3. 정규화 $1.xxx_2 \times 2^n$. 소수점을 **맨 앞 1의 바로 뒤**로 옮긴다. 소수점을 왼쪽으로 k칸 옮기면 n = +k, 오른쪽으로 k칸이면 n = −k.
   - 왜: 2진에서 소수점을 왼쪽으로 1칸 옮기면 모든 자릿값이 절반이 되어 수가 2배 작아진다(10진에서 왼쪽 1칸 = ÷ 10과 같음). 값이 그대로여야 하므로 줄어든 만큼 × 2를 붙인다. k칸이면 2^k배 작아졌으니 × 2^k. 오른쪽으로 k칸이면 2^k배 커졌으니 × 2^{-k}.
   - 몇 칸인지 세기: 1001.101_2은 맨 앞 1 뒤에 정수 자리가 3개(0, 0, 1) 있으니 왼쪽 3칸 → $1.001101_2 \times 2^3$. 0.00101_2은 맨 앞 1이 소수 셋째 자리라 그 1 뒤까지 오른쪽 3칸 → $1.01_2 \times 2^{-3}$.
4. 지수 칸 = n + bias(127, 16비트는 15)를 8비트(16비트는 5비트) 2진으로. bias를 더하면 실제 지수 −126 ~ 127이 1 ~ 254로 바뀌어 지수 칸에 음수가 없어진다(정렬·비교가 쉬워지는 이유는 Ch3 · 3). 자릿값 분해로 2진을 구한 뒤 8비트가 안 되면 **왼쪽**을 0으로 채운다: 124 = $111\ 1100_2$(7자리) → 0111 1100.
5. 가수 칸 = 1.xxx의 소수점 뒤 비트를 왼쪽부터 채우고 나머지 칸(단정도 23비트, 16비트 10비트까지)은 **오른쪽**에 0. 소수점 뒤 끝에 0을 붙여도 값이 같기 때문이다($0.01_2 = 0.0100_2$). 맨 앞 1은 숨은 비트라 넣지 않는다.
6. S | 지수 | 가수를 모두 이어 붙인 뒤 왼쪽부터 4비트씩 끊어 16진으로. 칸별로 끊으면 안 되는 이유: S 1비트 + 지수 8비트 = 9비트라 4비트 경계가 칸 경계와 맞지 않는다.

IEEE 754 → 10진: ① 16진 한 자리를 4비트로 ② 1 | 8 | 23(16비트는 1 | 5 | 10)으로 자르기 ③ 실제 지수 = 지수 칸 − bias ④ $(-1)^S \times 1.\text{가수}_2 \times 2^{\text{실제 지수}}$. 계산은 정규화의 거꾸로: 실제 지수가 +e면 소수점을 오른쪽으로 e칸, −e면 왼쪽으로 e칸 옮긴 뒤(빈자리는 0) 자릿값을 더한다. 예 $1.01_2 \times 2^2 = 101_2 = 5$, $1.1_2 \times 2^{-1} = 0.11_2 = 0.5 + 0.25 = 0.75$. $(-1)^0 = 1$이라 S = 0이면 양수, $(-1)^1 = -1$이라 S = 1이면 음수. PDF 외 보충

함정

- 가수 칸에 숨은 1을 넣지 않는다. −0.75의 가수 칸은 1100…이 아니라 1000…이다.
- 지수 칸에 실제 지수를 2의 보수로 넣지 않는다(−1 → 1111 1111은 틀림). 16비트에 bias 127을 쓰지 않는다.
- S 1비트 + 지수 8비트(16비트는 5비트)는 4의 배수가 아니다. 칸별로 끊지 말고 전부 이어 붙인 뒤 4비트씩 끊는다.

예제 (시험 수준)

−9.625를 IEEE 754 단정도(32비트)와 16비트(1-5-10, bias 15) 16진수로 쓰시오.

풀이 보기

$-1.001101_2 \times 2^3$, 32비트 0xC11A0000, 16비트 0xC8D0

① 부호: 음수라 S = 1. 크기 9.625만 2진으로 바꾼다.

② 정수 부분 9 = 8 + 1 = 1001_2. 소수 부분: 0.625 × 2 = 1.25 → 1(남은 0.25), 0.25 × 2 = 0.5 → 0, 0.5 × 2 = 1.0 → 1(끝). 위 → 아래로 읽어 0.101_2. 합치면 1001.101_2.

③ 정규화: 첫 1 바로 뒤로 소수점을 왼쪽으로 3칸 옮기면 1.001101_2. 값이 2^3배 작아졌으니 × 2^3 → 실제 지수 3.

④ 32비트 지수 칸 = 3 + 127 = 130 = 128 + 2 → 1000 0010. 가수 칸 = 소수점 뒤 001101(6비트) 뒤에 0 17개를 채워 23비트. 맨 앞 1은 숨은 비트라 넣지 않는다.

⑤ 이어 붙이기: 1 10000010 0011010…0 → 4비트씩 1100 0001 0001 1010 0000 0000 0000 0000 → C, 1, 1, A, 0, 0, 0, 0 = 0xC11A0000.

⑥ 16비트: 지수 칸 = 3 + 15 = 18 = 16 + 2 → 10010. 가수 10비트 0011010000. 1 10010 0011010000 → 1100 1000 1101 0000 → C, 8, D, 0 = 0xC8D0.

검산: 1.001101_2 = 1 + 1/8 + 1/16 + 1/64 = 1.203125, × 8 = 9.625. 16비트 거꾸로: 지수 칸 18 − 15 = 3, 같은 가수 → −9.625.

회독 점검

1. 0.15625를 IEEE 754 단정도 16진수로 쓰시오.

답 보기

0x3E200000

① 소수 부분 ×2: 0.15625 × 2 = 0.3125 → 0, 0.3125 × 2 = 0.625 → 0, 0.625 × 2 = 1.25 → 1, 0.25 × 2 = 0.5 → 0, 0.5 × 2 = 1.0 → 1(끝). 위 → 아래로 읽어 0.00101_2.

② 정규화: 첫 1이 소수점 뒤 셋째 자리 → 소수점을 오른쪽으로 3칸 → $1.01_2 \times 2^{-3}$.

③ S = 0. 지수 칸 = −3 + 127 = 124 = 64 + 32 + 16 + 8 + 4 → 111 1100(7자리)이라 왼쪽에 0을 하나 채워 8비트 0111 1100. 가수 칸 = 소수점 뒤 01 뒤에 0 21개(23비트).

④ 0 01111100 0100…0 → 0011 1110 0010 0000 0000 0000 0000 0000 → 0x3E200000.

검산: 1.01_2 = 1.25, 1.25 ÷ 8 = 0.15625.

---

2. 13.0을 16비트 형식(부호 1, 지수 5, 가수 10, bias 15)의 16진수로 쓰시오.

답 보기

0x4A80

① 13 = 8 + 4 + 1 = 1101_2. 소수점을 왼쪽으로 3칸 → $1.101_2 \times 2^3$.

② S = 0. 지수 칸 = 3 + 15 = 18 = 16 + 2 → 10010.

③ 가수 칸 10비트 = 101 뒤에 0 → 1010000000.

④ 0 10010 1010000000 → 4비트씩 0100 1010 1000 0000 → 0x4A80.

검산: 1.101_2 = 1.625, × 8 = 13.

---

3. 단정도 0xC1C80000을 10진수로 바꾸시오.

답 보기

−25.0

① 2진: C = 1100, 1 = 0001, C = 1100, 8 = 1000, 나머지 0 → 1100 0001 1100 1000 0000 ....

② 자르기: S = 1(음수). 다음 8비트 지수 칸 = 1000 0011 = 128 + 2 + 1 = 131. 남은 가수 = 1001 0...0.

③ 실제 지수 = 131 − 127 = 4.

④ 유효숫자 = 1.1001_2. 소수점을 오른쪽으로 4칸 → 11001_2 = 16 + 8 + 1 = 25 → −25.0.

---

4. 0.03125를 저장할 때 지수 칸의 값을 단정도, 배정도, 16비트 형식 각각 10진수와 2진수로 쓰시오.

답 보기

단정도 122 (0111 1010), 배정도 1018 (011 1111 1010), 16비트 10 (01010)

① 0.03125 = 1/32 = 2^{-5} = $1.0_2 \times 2^{-5}$. 실제 지수 −5.

② 지수 칸 = 실제 지수 + bias: −5 + 127 = 122, −5 + 1023 = 1018, −5 + 15 = 10.

③ 2진: 122 = 64 + 32 + 16 + 8 + 2 → 8비트 0111 1010. 1018 = 512 + 256 + 128 + 64 + 32 + 16 + 8 + 2 → 11비트 011 1111 1010. 10 = 8 + 2 → 5비트 01010.

---

5. O/X: (1) 1.0을 단정도로 저장하면 지수 칸은 0000 0000이다. (2) 단정도 가수 칸 23비트에는 정규화된 수의 맨 앞 1도 들어간다. (3) 16비트(1-5-10) 형식의 bias는 15이다. (4) 단정도 지수 칸 255는 가장 큰 정규화 수에 쓰인다.

답 보기

(1) X (2) X (3) O (4) X

① (1) 1.0 = $1.0_2 \times 2^0$, 실제 지수 0이라 지수 칸 = 0 + 127 = 127 = 0111 1111. 0000 0000은 예약값이다. [Ch3 p.28]

② (2) 맨 앞 1은 숨은 비트라 저장하지 않는다. 그래서 정밀도가 24비트가 된다. [Ch3 p.28]

③ (3) $2^{(5-1)} - 1 = 16 - 1 = 15$. PDF 외 보충

④ (4) 255는 무한대·NaN용 예약값. 가장 큰 정규화 수의 지수 칸은 254(실제 지수 127). [Ch3 p.30]

더 자세히: 이론 노트 Ch3 · 6, 7 / 실전 6

Ch3 · 3 슬라이드 p.23–40

부동소수점 연산과 외울 문장

시험에 나오는 모양: FP 덧셈 4단계·곱셈 5단계의 순서와 단계별 값, 부동소수점 문장 T/F·빈칸, GRS 반올림 판정. (기출: 26-1 A = −0.5, B = 2.625로 A+B, A×B)

외울 것

FP = 부동소수점(floating point). 단계 ③의 오버플로 = 지수가 너무 커서, 언더플로(underflow) = 음의 지수가 너무 커서(0에 너무 가까워서) 지수 칸에 담을 수 없는 것(아래 외울 문장). [Ch3 p.26]

슬라이드 예: $0.5 = 1.000_2 \times 2^{-1}$, $-0.4375 = -1.110_2 \times 2^{-2}$ (유효숫자 4비트) [Ch3 p.33, 36]

덧셈 4단계	0.5 + (−0.4375)
① 정렬: 지수가 작은 쪽 유효숫자를 오른쪽 시프트	$-1.110_2 \times 2^{-2} = -0.111_2 \times 2^{-1}$
② 유효숫자 더하기	$1.000_2 + (-0.111_2) = 0.001_2 \times 2^{-1}$
③ 정규화, 오버플로·언더플로 검사	$1.000_2 \times 2^{-4}$, $-126 \le -4 \le 127$
④ 반올림 (정규화가 깨지면 ③으로)	필요 없음 → 0.0625

덧셈 단계별 이유

- **유효숫자 4비트** = 소수점 앞 1 + 소수점 뒤 3비트(1.110_2). 계산 중에 생긴 그보다 뒤의 비트는 ④ 반올림에서 정리한다. 실제 단정도는 1 + 23 = 24비트(Ch3 · 2).
- **① 정렬은 몇 칸, 왜 작은 쪽**: 지수가 같아야 자리를 맞춰 더할 수 있다. 유효숫자를 오른쪽으로 1칸 밀면 값이 절반이 되므로 지수를 1 올려 값을 유지한다(Ch3 · 2 정규화와 같은 원리). 그래서 **두 지수의 차이만큼** 민다: −2와 −1은 차이 1 → 1칸, $1.110_2 \times 2^{-2} = 0.111_2 \times 2^{-1}$. 큰 쪽을 왼쪽으로 밀면 맨 앞 1이 소수점 앞 한 자리 칸을 넘어 앞자리를 잃는다. 작은 쪽을 오른쪽으로 밀면 잃는 것은 끝의 작은 비트뿐이다. [Ch3 p.33]
- **② 부호가 다르면 빼기**: 부동소수점은 부호-크기 표현(부호 비트 1개 + 크기, 음수라고 크기를 2의 보수로 바꾸지 않음)이라 크기와 부호를 따로 다룬다. [Ch3 p.25] 부호가 같으면 크기를 더하고 그 부호를 쓴다. 부호가 다르면 **큰 크기 − 작은 크기**를 하고 크기가 큰 쪽의 부호를 쓴다. 0.5 + (−0.4375)는 $1.000_2 - 0.111_2$, 0.5 쪽이 커서 결과는 +.
- **2진 소수 더하기·빼기 요령**: 소수점 뒤 자리 수를 같게 맞춘 뒤 소수점을 떼고 정수로 읽어 10진으로 계산하고, 같은 자리 수로 소수점을 다시 찍는다. $1.000_2 - 0.111_2 \rightarrow 1000_2 - 0111_2 = 8 - 7 = 1 = 0001_2 \rightarrow 0.001_2$. 자리별로 직접 빼도 된다: 1 − 0 = 1, 1 − 1 = 0, 0 − 1은 왼쪽에서 1을 빌려(빌려 온 1은 이 자리에서 2) 2 − 1 = 1, 빌려 준 자리는 1 줄어든다. PDF 외 보충
- **③ 정규화**: 결과를 다시 $1.xxx_2$ 꼴로. $0.001_2 \times 2^{-1}$은 맨 앞 1이 소수 셋째 자리라 소수점을 오른쪽으로 3칸 → 지수 −1 − 3 = −4 → $1.000_2 \times 2^{-4}$. 결과가 $10.xxx_2$처럼 정수 자리가 둘이면 왼쪽으로 1칸, 지수 +1. 그다음 지수가 단정도 −126 ~ 127 안인지 본다(벗어나면 오버플로·언더플로).

곱셈 5단계	0.5 × (−0.4375)
① 지수 더하기 (지수 칸끼리면 bias 한 번 빼기)	−1 + (−2) = −3, 칸으로 126 + 125 − 127 = 124
② 유효숫자 곱하기	$1.000_2 \times 1.110_2 = 1.110_2$
③ 정규화, 오버플로·언더플로 검사	$1.110_2 \times 2^{-3}$, 범위 안
④ 반올림	필요 없음
⑤ 부호 결정	+ × − = − → $-1.110_2 \times 2^{-3} = -0.21875$

② 유효숫자 곱하기 요령: 덧셈처럼 소수점을 떼고 정수로 곱한 뒤, 두 수의 소수점 뒤 자리 수를 더한 만큼 오른쪽에서 소수점을 찍는다(10진 1.5 × 1.5 = 15 × 15 = 225 → 소수 1 + 1 = 2자리 → 2.25와 같음). $1.000_2 \times 1.110_2$ → 8 × 14 = 112 = 1110000_2, 소수 3 + 3 = 6자리 → $1.110000_2 = 1.110_2$. 한쪽이 1.0_2이면 곱은 다른 쪽 그대로다. 결과가 $10.xxx_2$가 되면 ③에서 왼쪽 1칸·지수 +1로 정규화한다. 부호는 크기를 다 구한 뒤 ⑤에서 정한다(같으면 +, 다르면 −). PDF 외 보충

bias를 빼는 이유: 지수 칸 = 실제 지수 + 127이라 칸끼리 더하면 127이 두 번 들어간다. 식으로: (a + 127) + (b + 127) = (a + b) + 254이므로 127을 한 번 빼야 (a + b) + 127, 즉 곱의 올바른 지수 칸이 된다. 실제 지수로 계산하면(−1 + (−2) = −3) bias를 뺄 일이 없고, 지수 칸으로 계산할 때만 뺀다. 덧셈은 정렬하고 곱셈은 정렬하지 않는다. [Ch3 p.36]

외울 문장

- 정규화 = 소수점 왼쪽에 0이 아닌 숫자 한 자리. 1.0×10^{-9} O, 10.0×10^{-10}·0.1×10^{-8} X. 2진 소수점 = binary point. [Ch3 p.23]
- 정규화 표기의 장점 3: ① 부동소수점 데이터 교환 단순화 ② 부동소수점 알고리즘 단순화 ③ 한 word에 저장되는 수의 정확도 증가(쓸모없는 앞자리 0 대신 진짜 숫자). [Ch3 p.24]
- 가수(fraction, mantissa) 칸을 늘리면 정밀도(precision), 지수 칸을 늘리면 범위(range)가 좋아진다. 둘은 맞바꿈. [Ch3 p.25]
- 부호는 부호-크기 표현(0 양수, 1 음수). 음수라고 가수를 2의 보수로 바꾸지 않는다. [Ch3 p.25]
- FP 오버플로 = 지수가 너무 커서 지수 칸에 담을 수 없음. 언더플로 = 음의 지수가 너무 커서 담을 수 없음(0에 너무 가까움). 줄이려면 64비트 double. [Ch3 p.26]
- p.25는 지수 칸을 "2's complement"라 적었지만 IEEE 754 지수는 bias로 저장한다. 시험에서는 bias로 답한다. [Ch3 p.25, 28]
- bias를 쓰는 이유(정수 비교로 정렬): ① 부호가 MSB라 비교가 빠름 ② 지수를 가수 앞에 둬 정렬이 단순 ③ 음수 지수를 2의 보수로 쓰면 크게 보이는 문제를 bias로 해결. [Ch3 p.29]
- IEEE 754-1985는 이식성 문제를 해결했다. 설계자 William Kahan은 튜링상 수상. [Ch3 p.27]
- FP 덧셈기는 정수 덧셈기보다 훨씬 복잡하다. [Ch3 p.34]
- 중간 계산 때 오른쪽 여분 2비트 guard·round를 유지한다. $2.56 \times 10^{0} + 2.34 \times 10^{2}$ → 없으면 2.36×10^{2}, 있으면 2.37×10^{2}. [Ch3 p.38]
- 반올림 4모드: 0 쪽(버림), +∞ 쪽, −∞ 쪽, 가장 가까운 짝수(기본). −2.75 → −2, −2, −3, −3. 3.8 → 3, 4, 3, 4. [Ch3 p.39]
- G = 남길 LSB 바로 다음 비트, R = 그다음, S(sticky) = 나머지 전체 OR. GRS 000~011 내림, 101~111 올림, 100이면 LSB가 0인 쪽. [Ch3 p.40] 이유: G 자리의 자릿값은 남길 LSB 자릿값의 정확히 절반이다. G = 0이면 버릴 부분이 절반보다 작아 내림. G = 1이고 R, S가 모두 0(100)이면 딱 절반이라 두 후보의 거리가 같으므로, 기본 모드 "가장 가까운 짝수"대로 LSB가 0(짝수)이 되는 쪽을 고른다. G = 1이고 R이나 S 중 하나라도 1이면 절반보다 커서 올림. S를 OR로 만드는 이유는 R 뒤에 1이 하나라도 있었는지를 1비트로 기억하기 위해서다. "올림" = 남길 부분의 LSB에 1을 더하는 2진 덧셈이다(자리올림이 생기면 왼쪽으로 전파: 1011 + 1 = 1100). "내림" = G, R, S를 그냥 버린다.
- FP 레지스터 f0~f31(**f0은 0으로 고정 아님**), 적재·저장 flw/fsw(단정도)·fld/fsd(배정도). feq·flt·fle(.s/.d)는 결과를 **정수 레지스터**에 0(거짓)·1(참)으로 쓰고 beq·bne로 분기. [Ch3 p.37]

함정

- 정렬은 지수가 작은 쪽을 오른쪽으로 민다. 큰 쪽을 왼쪽으로 밀면 앞자리를 잃는다.
- 곱셈에서 지수 칸끼리 더하기만 하면 bias가 두 번 들어간다.
- GRS = 100이면 무조건 올리지 않는다. LSB가 0이면 그대로, 1이면 올린다.

예제 (시험 수준, 26-1 기출)

A = −0.5, B = 2.625일 때 A + B와 A × B를 2진 부동소수점 단계대로 계산하고, 두 결과를 32비트 IEEE 754와 16비트(1-5-10) 16진수로 쓰시오.

풀이 보기

A + B = $1.0001_2 \times 2^1$ = 2.125 (0x40080000, 0x4040) / A × B = $-1.0101_2 \times 2^0$ = −1.3125 (0xBFA80000, 0xBD40)

① 2진 정규화: 0.5 × 2 = 1.0 → 1이라 0.5 = 0.1_2 → A = $-1.0_2 \times 2^{-1}$. 2.625는 정수 부분 2 = 10_2, 소수 부분 0.625 × 2 = 1.25 → 1(남은 0.25), 0.25 × 2 = 0.5 → 0, 0.5 × 2 = 1.0 → 1(끝). 위 → 아래로 읽어 0.101_2. 10.101_2의 소수점을 왼쪽으로 1칸 → B = $1.0101_2 \times 2^1$.

② 덧셈 정렬: 지수 −1과 1, 차이 2. 지수가 작은 A를 오른쪽으로 2칸 밀고 지수를 1로: $-1.0_2 \times 2^{-1} = -0.0100_2 \times 2^1$.

③ 유효숫자 더하기: 부호가 다르므로 큰 크기(B 1.0101_2)에서 작은 크기(A 0.0100_2)를 빼고, 크기가 큰 B의 부호(+)를 쓴다. $1.0101_2 - 0.0100_2 = 1.0001_2$(소수 둘째 자리 1 − 1 = 0, 나머지 그대로). 정수로 검산: $10101_2 - 00100_2$ = 21 − 4 = 17 = 10001_2. → $1.0001_2 \times 2^1$.

④ 정규화·검사: 이미 1.xxxx 꼴이고 지수 1은 범위 안. 반올림: 가수 0001이 칸에 다 들어가 필요 없음. 값 = 10.001_2 = 2 + 1/8 = 2.125.

⑤ A + B 비트: S = 0, 가수 0001 뒤 0. 32비트 지수 칸 1 + 127 = 128 = 1000 0000 → 0 10000000 0001…0 → 0100 0000 0000 1000 0…0 = 0x40080000. 16비트 지수 칸 1 + 15 = 16 = 10000 → 0 10000 0001000000 → 0100 0000 0100 0000 = 0x4040.

⑥ 곱셈 1단계 지수 더하기: −1 + 1 = 0. 지수 칸으로는 bias를 한 번 뺀다: 32비트 126 + 128 − 127 = 127, 16비트 14 + 16 − 15 = 15.

⑦ 2단계 유효숫자 곱: $1.0_2 \times 1.0101_2 = 1.0101_2$. 3단계 정규화: 이미 정규화, 지수 0 범위 안. 4단계 반올림: 필요 없음. 5단계 부호: (−) × (+) = − → $-1.0101_2 \times 2^0 = -(1 + 1/4 + 1/16) = -1.3125$.

⑧ A × B 비트: S = 1, 가수 0101 뒤 0. 32비트 지수 칸 127 = 0111 1111 → 1 01111111 0101…0 → 1011 1111 1010 1000 0…0 = 0xBFA80000. 16비트 지수 칸 15 = 01111 → 1 01111 0101000000 → 1011 1101 0100 0000 = 0xBD40.

검산: −0.5 + 2.625 = 2.125, −0.5 × 2.625 = −1.3125. [Ch3 p.32–36]

회독 점검

1. 빈칸: FP 덧셈은 (ㄱ) → 유효숫자 더하기 → (ㄴ) → (ㄷ) 순서이고, (ㄷ) 뒤에 정규화가 깨지면 (ㄹ)번째 단계로 돌아간다. FP 곱셈의 마지막 단계는 (ㅁ)이다.

   답 보기

   (ㄱ) 정렬(지수가 작은 쪽을 오른쪽 시프트) (ㄴ) 정규화·오버플로/언더플로 검사 (ㄷ) 반올림 (ㄹ) 3 (ㅁ) 부호 결정

   ① 반올림으로 9.9999×10^2가 10.000×10^2가 되는 것처럼 정규화가 다시 깨질 수 있어 3단계로 돌아간다. [Ch3 p.32]

   ② 곱셈은 부호-크기 표현이라 크기를 다 구한 뒤 부호를 따로 정한다. [Ch3 p.35]

2. 유효숫자 4비트로 0.875 + 0.25를 덧셈 4단계로 계산하고 2진 정규화 결과와 10진 값을 쓰시오.

   답 보기

   $1.001_2 \times 2^0 = 1.125$

   ① 2진: 0.875 × 2 = 1.75 → 1(남은 0.75), 0.75 × 2 = 1.5 → 1(남은 0.5), 0.5 × 2 = 1.0 → 1(끝). 위 → 아래로 읽어 $0.111_2 = 1.110_2 \times 2^{-1}$. $0.25 = 0.01_2 = 1.000_2 \times 2^{-2}$.

   ② 정렬: 지수 차이 1. 작은 쪽 0.25를 오른쪽으로 1칸 → $0.100_2 \times 2^{-1}$.

   ③ 더하기: $1.110_2 + 0.100_2$. 소수 첫째 자리 $1 + 1 = 10_2$(0 쓰고 올림), 정수 $1 + 0 + 1 = 10_2$ → $10.010_2 \times 2^{-1}$.

   ④ 정규화: 정수 자리가 두 자리라 소수점을 왼쪽으로 1칸, 지수 +1 → $1.0010_2 \times 2^0$. 범위 안.

   ⑤ 반올림: 4비트 1.001_2만 남긴다. 잘리는 비트 0이라 그대로. 1 + 1/8 = 1.125. 검산 0.875 + 0.25 = 1.125.

3. 단정도 곱셈에서 두 수의 지수 칸이 130과 124이다. 곱의 지수 칸(정규화 전)과 실제 지수를 쓰시오.

답 보기

지수 칸 127, 실제 지수 0

① 실제 지수: 130 − 127 = 3, 124 − 127 = −3. 곱의 실제 지수 = 3 + (−3) = 0.

② 칸끼리 더하면 130 + 124 = 254 = 0 + 254로 bias가 두 번 들어간다. 127을 한 번 빼서 254 − 127 = 127.

검산: 127 − 127 = 0. [Ch3 p.36]

---

4. 가수(fraction)를 4비트만 남길 때 가장 가까운 짝수로 반올림하시오. 띄어 쓴 뒤 3비트는 G, R, S이다. (1) 1.1011 100 (2) 1.0010 101 (3) 1.0100 100

답 보기

(1) 1.1100_2 (2) 1.0011_2 (3) 1.0100_2

① (1) GRS 100 = 정확히 절반. LSB(가수 4번째 비트)가 1(홀수)이라 짝수가 되도록 올림: 1011 + 1 = 1100.

② (2) GRS 101 > 100, 절반보다 크다 → 올림: 0010 + 1 = 0011.

③ (3) GRS 100 = 절반, LSB가 이미 0(짝수) → 그대로 1.0100_2. [Ch3 p.40]

---

5. O/X: (1) RISC-V의 f0은 항상 0.0이다. (2) flt.s의 결과는 FP 레지스터에 저장된다. (3) IEEE 754 지수 칸은 2의 보수로 저장한다. (4) 지수 칸 비트 수를 늘리면 정밀도가 좋아진다. (5) −3.5를 round toward zero로 정수로 바꾸면 −3이다.

답 보기

(1) X (2) X (3) X (4) X (5) O

① (1) f0은 상수 0으로 고정되지 않는다(x0과 다름). (2) 비교 결과는 정수 레지스터에 0·1로 들어간다. [Ch3 p.37]

② (3) bias 방식(실제 지수 + 127)이다. p.25의 "2's complement"는 일반형 설명. [Ch3 p.28–29]

③ (4) 지수는 범위, 가수가 정밀도다. [Ch3 p.25]

④ (5) 0 쪽 반올림은 버림이라 −3.5 → −3(0에 가까운 쪽). [Ch3 p.39]

더 자세히: 이론 노트 Ch3 · 6~9 / 실전 6, 8

Ch4 · 1 슬라이드 p.3–17, 21

데이터패스 구성과 그림

시험에 나오는 모양: 그림 빈칸(ALU, 가산기 등), 기계어를 주고 각 선(rs1·rs2·rd)의 레지스터 번호 쓰기, "레지스터가 있는 하드웨어" 이름. (기출: 21-2 그림 빈칸·프로세서 두 가지, 26-1 control line·라인별 레지스터 번호)

먼저 알아 둘 말

- **인출(fetch)** = PC가 가리키는 주소에서 명령어를 메모리로부터 가져오는 일. [Ch4 p.3] **포트(port)** = 장치에 값이 들어가고 나오는 입구·출구.
- **제어 신호(control signal)** = 제어 유닛이 장치마다 보내는 0/1 스위치. 이 단원 문제에 필요한 뜻만: RegWrite = 1이면 레지스터 파일에 결과를 쓴다(0이면 안 씀). MemRead·MemWrite = 1이면 데이터 메모리를 읽는다·쓴다. ALUSrc = 0이면 ALU 둘째 입력이 레지스터 값, 1이면 즉시값. 명령어가 하는 일에서 바로 나온다: sw·beq는 레지스터에 결과를 쓰지 않으니 RegWrite = 0, lw는 메모리를 읽기만 하니 MemRead = 1·MemWrite = 0, lw는 주소 = 레지스터 + 즉시값이라 ALUSrc = 1이다. 전체 표와 이유는 Ch4 · 2. [Ch4 p.24–25]
- **Branch와 Zero, PCSrc** = 다음 PC를 고르는 세 신호. Branch = 1이면 지금 명령어가 분기(beq)라는 뜻이다(제어 유닛이 opcode를 보고 냄). Zero = ALU 결과가 0이면 1이 되는 ALU 출력이다. beq는 ALU에서 두 레지스터를 빼므로 두 값이 같을 때 Zero = 1이 된다. **PCSrc = Branch AND Zero**: AND 게이트 하나가 두 신호를 받아, 둘 다 1일 때만 1을 내서 다음 PC를 분기 목표 주소로 바꾼다. 하나라도 0이면 다음 PC = PC + 4. [Ch4 p.25]
- **레지스터 파일의 입구·출구 이름**: 입구 **읽기 레지스터 1·2(Read register 1·2)**와 **쓰기 레지스터(Write register)**에는 레지스터 **번호**(5비트, 0~31)가 들어온다. 출구 **읽기 데이터 1·2(Read data 1·2)**로는 그 번호 레지스터에 든 **값**(32비트)이 나오고, 입구 **쓰기 데이터(Write data)**에는 쓸 값이 들어온다. 예 `add x9, x20, x21` : 읽기 레지스터 1 = 20, 읽기 레지스터 2 = 21, 쓰기 레지스터 = 9가 들어가고, 읽기 데이터 1·2로 x20·x21의 값이 나오며, 쓰기 데이터로 그 합이 들어간다. 그래서 "선에 실리는 번호"를 물으면 값이 아닌 번호를 답한다. [Ch4 p.12, 24]
- **클럭 엣지(clock edge)** = 클럭 신호가 0에서 1(또는 1에서 0)로 바뀌는 순간. 클럭 사이클 하나는 엣지 하나에서 다음 엣지까지다. [Ch4 p.10] PDF 외 보충

외울 것

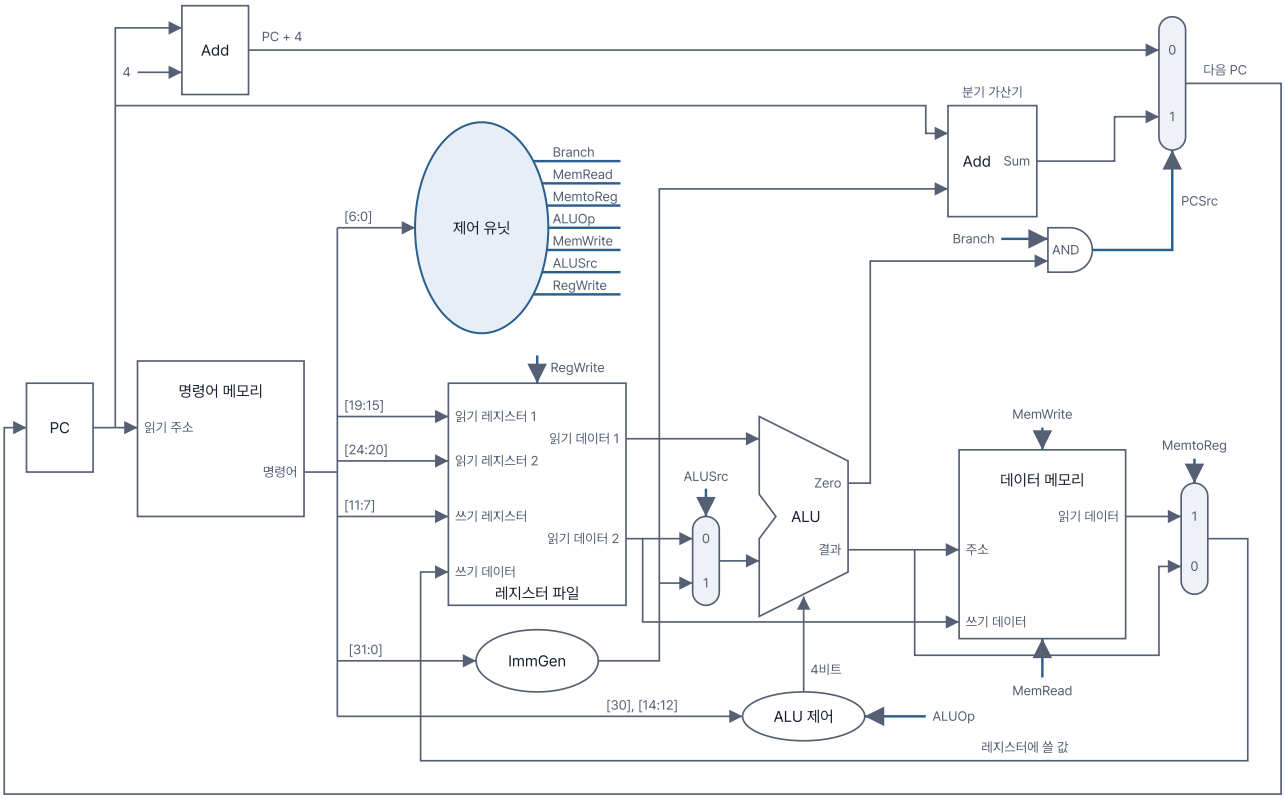
- 구현 대상 7개: 메모리 참조 `lw`, `sw` / 산술·논리 `add`, `sub`, `and`, `or` / 분기 `beq`. [Ch4 p.3]
- 공통 두 단계: ① PC로 명령어 메모리에서 명령어 인출 ② 명령어 필드로 레지스터 1~2개 읽기. 이후는 명령어 종류마다 다름. [Ch4 p.3]
- 멀티플렉서(MUX) = 선택 신호에 따라 입력 여러 개 중 하나를 골라 내보내는 장치. 데이터 선을 그냥 합칠 수 없어서 필요하다. [Ch4 p.7]
- 프로세서 = 데이터패스 + 제어. 기본 구현 = 단순 그림 + MUX 3개 + 제어 유닛. [Ch4 p.8]

구성 요소	역할	종류
PC(프로그램 카운터)	현재 명령어 주소를 담은 32비트 레지스터	상태
명령어 메모리	주소(PC)를 받아 32비트 명령어를 내보냄	상태
가산기(PC + 4)	다음 명령어 주소. 명령어가 4바이트라 +4	조합
레지스터 파일(register file)	범용 레지스터 32개를 모은 구조. 읽기 포트 2, 쓰기 포트 1, RegWrite	상태
ALU	32비트 입력 2개, 4비트 제어, 결과가 0이면 Zero = 1	조합
ImmGen(즉시값 생성기)	명령어 전체를 받아 12비트 즉시값을 32비트로 부호 확장(분기용 1비트 시프트 포함)	조합
데이터 메모리	주소 = ALU 결과, 쓰기 데이터 = 읽기 데이터 2. MemRead·MemWrite	상태
분기 가산기	PC + (오프셋 << 1) = 분기 목표 주소. ALU와 별개	조합

[Ch4 p.11–15] 가산기·MUX·ImmGen을 조합 소자로 분류한 것은 PDF 외 보충.

- MUX 3개의 자리: ALUSrc = ALU 둘째 입력 앞, MemtoReg = 레지스터 쓰기 데이터 앞, PCSrc = PC 입력 앞. 0/1 뜻은 Ch4 · 2 신호표. [Ch4 p.16–17, 24]
- 교재 기준 슬라이드 그림의 MUX 숫자는 **MemtoReg만 위 1·아래 0**이다(나머지는 위 0). 위치가 아니라 숫자로 읽는다.
- 명령어 비트 → 포트: [6:0] opcode → 제어 유닛, **[19:15] rs1 → 읽기 레지스터 1**, **[24:20] rs2 → 읽기 레지스터 2**, **[11:7] rd → 쓰기 레지스터**, [31:0] → ImmGen, [30]·[14:12] → ALU 제어. [Ch4 p.21, 24] [14:12]는 funct3 전체, [30]은 funct7(31~25번) 중 한 비트다. 구현하는 R형 넷 중 funct7이 다른 것은 sub(0100000)뿐이고 그 차이가 30번 비트 하나라서, 그 비트만 보내면 add와 sub를 가를 수 있다. 교재 기준
- 필드 역할: funct3·funct7 = opcode에 따라 확장 opcode. rd = R형·load의 목적지 레지스터. [Ch4 p.21]
- rs1 = R형·분기의 첫째 피연산자, load·store의 베이스. rs2 = R형·분기의 둘째 피연산자, store에서 메모리로 복사될 레지스터. [Ch4 p.21]

- 조합 소자(combinational) = 출력이 입력만의 함수(AND 게이트, ALU). 상태 소자(state) = 저장(레지스터, 메모리), 입력 = 쓸 데이터·클럭, 출력 = 이전 사이클에 쓴 값. [Ch4 p.9]
- 분류하는 질문 하나: **"값을 기억하는가?"** "출력이 입력만의 함수"는 지금 들어온 입력만 보면 출력이 정해진다는 뜻이다. 같은 입력이면 언제나 같은 출력이 나오고, 기억하는 것이 없다. 예 가산기에 3과 4가 들어오면 언제나 7이다. 반대로 레지스터 파일은 같은 번호 5를 넣어도 전에 x5에 무엇을 써 두었는지에 따라 다른 값이 나온다. 기억이 있으니 상태 소자다. PC도 주소를 담아 두는 레지스터라 상태 소자다. MUX는 지금 들어온 입력 중 하나를 선택 신호대로 내보낼 뿐이고, ImmGen은 지금 명령어의 비트를 늘려 내보낼 뿐이라 조합 소자다. PDF 외 보충
- 클로킹 방식 = 언제 읽고 쓰는지 정한 규칙. 하드웨어 동작을 예측할 수 있게 하려고 정한다. **엣지 트리거(edge-triggered)** = 상태 소자는 클럭 엣지에서만 갱신, 가장 긴 지연이 클럭 주기를 정함. [Ch4 p.10]
- 엣지 트리거에서 한 사이클이 흐르는 순서: ① 엣지에서 상태 소자(PC, 레지스터 파일 등)가 값을 내보낸다. ② 엣지와 엣지 사이에 조합 소자(가산기, ALU, MUX)가 그 값으로 계산한다. ③ 다음 엣지에서 계산 결과가 상태 소자에 저장된다. 그래서 상태 소자의 값은 사이클 동안 고정이고, 계산 도중에 입력이 바뀌어 꼬이는 일이 없다. 예 PC는 사이클 내내 현재 주소를 내보내고, 가산기가 만든 PC + 4는 다음 엣지에야 PC에 들어간다. 다음 엣지 전에 가장 느린 계산까지 끝나야 하므로 가장 긴 지연이 클럭 주기를 정한다. [Ch4 p.10]
- 메모리가 둘인 이유: 단일 사이클은 한 명령어가 자원을 한 번만 쓴다. lw·sw는 인출과 데이터 접근에 메모리를 두 번 쓴다. [Ch4 p.16] 레지스터 파일도 한 명령어에서 읽고(처음) 쓰지만(끝) 괜찮은 이유: 읽기 포트 2개와 쓰기 포트 1개가 따로 달린 부품이라 각 포트는 한 번씩만 쓰이고, 쓰기는 사이클 끝 엣지에 일어난다. PDF 외 보충



단일 사이클 데이터패스(슬라이드 p.17·p.24에 p.26–28의 제어 유닛과 AND 게이트를 더함). 제어 유닛 출력은 위부터 Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite. 분기용 1비트 시프트는 ImmGen이 함께 한다(p.15). T자 분기점만 연결이다.

그림 읽는 법 (빈칸 문제)

시험 그림은 상자 안 이름을 지우고 낸다. 이름은 **모양**으로 후보를 좁히고, 모양이 같은 상자(Add 둘, 타원 셋, MUX 셋)는 **들어오고 나가는 선**으로 가른다. 위 그림에서 강조된 선이 제어 신호다. [Ch4 p.17, 24]

그림 속 위치·모양	구별하는 단서(선)	이름
맨 왼쪽 작은 세로 상자	출력이 명령어 메모리와 두 Add로 감. 입력은 오른쪽 끝 MUX에서 아래로 돌아오는 긴 선	PC
PC 바로 오른쪽 큰 상자	입력 = 읽기 주소(PC), 출력 = 명령어(32비트 버스로 갈라짐)	명령어 메모리
왼쪽 위 작은 Add	입력 = PC와 상수 4	PC + 4 가산기
오른쪽 위 Add(Sum)	입력 = PC와 ImmGen 출력	분기 가산기
가운데 위 큰 타원	입력 = [6:0](opcode), 출력 = 신호 7개	제어 유닛
가운데 큰 사각형(입구 4, 출구 2)	입구 = [19:15], [24:20], [11:7], 쓰기 데이터. 출구 = 읽기 데이터 1·2. 위에 RegWrite	레지스터 파일
레지스터 파일 아래 작은 타원	입력 = [31:0](명령어 전체), 출력 = 32비트(분기 가산기와 ALU 앞 MUX로)	ImmGen
왼쪽 변 가운데가 V자로 파인 도형	입력 2개, 출력 = Zero와 결과	ALU
ALU 아래 작은 타원	입력 = ALUOp와 [30]·[14:12], 출력 = 4비트(ALU로)	ALU 제어
오른쪽 큰 상자	입구 = 주소(ALU 결과)·쓰기 데이터(읽기 데이터 2), 출구 = 읽기 데이터. 위·아래에 MemWrite·MemRead	데이터 메모리
한쪽이 둥근 작은 게이트	입력 = Branch와 Zero, 출력 = PCSrc	AND 게이트
위아래로 길쭉한 둥근 상자	입력 2개(0·1 표시) + 위나 아래에서 꽂히는 선택 신호 1개. ALU 앞 = ALUSrc, 데이터 메모리 오른쪽 = MemtoReg, 오른쪽 위(PC로 돌아가는 선 앞) = PCSrc	MUX

교재 그림 중에는 ImmGen과 분기 가산기 사이에 "Shift left 1" 상자가 따로 있는 것도 있다. 그 상자가 있으면 "왼쪽 1비트 시프트"(오프셋 × 2)로 답한다. 이 그림과 슬라이드 p.15는 그 일을 ImmGen이 함께 한다. [Ch4 p.14–15] 교재 기준

푸는 순서 (기계어 → 선에 실리는 번호)

1. 16진 한 자리를 4비트로 바꿔 32비트를 쓴다.
2. 맨 오른쪽 7비트(opcode)로 형식을 정한다: `0110011` R형, `0000011` lw, `0100011` sw, `1100011` beq. 교재 기준 R형이면 funct7(31~25번)·funct3(14~12번)으로 명령어를 고른다: add = 0000000·000, sub = 0100000·000, and = 0000000·111, or = 0000000·110. 교재 기준

3. 형식과 상관없이 [19:15] → 읽기 레지스터 1, [24:20] → 읽기 레지스터 2, [11:7] → 쓰기 레지스터에 그대로 실린다. 5비트씩 10진으로. 이유: 그림에서 이 세 선은 명령어 비트에 고정으로 연결된 전선이라, 어떤 명령어가 오든 그 자리의 비트가 그대로 포트로 간다. 그 번호를 실제로 쓸지 말지는 제어 신호(RegWrite, ALUSrc)가 정한다. Ch2 · 4에서 rs1·rs2·rd 칸을 모든 형식에서 같은 자리에 둔 것도 이 전선을 하나로 쓰기 위해서다. [Ch2 p.24] [Ch4 p.21, 24]
4. lw의 [24:20], sw·beq의 [11:7]은 즉시값 조각이다. 선에 실리긴 하지만 ALUSrc·RegWrite 때문에 결과에 안 쓰인다.

함정

- 분기 목표 주소는 ALU가 아니라 별도의 분기 가산기가 계산한다. ALU는 두 레지스터를 빼서 Zero로 같은지만 본다.
- sw가 저장할 값은 읽기 데이터 2(rs2)에서 ALU를 거치지 않고 데이터 메모리 쓰기 데이터로 바로 간다.
- "메모리가 둘인 이유 = 속도"는 틀린 답이다. 한 사이클에 자원을 두 번 쓸 수 없어서다.

예제 (시험 수준)

`lw x14, 20(x10)`의 기계어는 `0x01452703`이다. 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터 선에 실리는 번호와 ImmGen 출력(10진)을 쓰시오. `x10 = 0x2000`이면 데이터 메모리 주소는?

풀이 보기

읽기 레지스터 1 = 10, 읽기 레지스터 2 = 20, 쓰기 레지스터 = 14, ImmGen 출력 = 20, 주소 = 0x2014.

① 4비트씩: 0000 0001 0100 0101 0010 0111 0000 0011.

② 오른쪽 7비트 = `0000011` → lw(I형). I형 칸 imm[11:0] 12 · rs1 5 · funct3 3 · rd 5 · opcode 7로 끊으면 000000010100 · 01010 · 010 · 01110 · 0000011.

③ [19:15] = 01010_2 = 8 + 2 = 10 → 읽기 레지스터 1 = 10(x10, 베이스). 선에는 값(0x2000)이 아닌 번호 10이 실린다.

④ [24:20]은 I형에서 imm[11:0]의 아래 5비트(imm[4:0])다: 10100_2 = 16 + 4 = 20. 전선은 고정이라 이 선에도 20이 실려 읽기 데이터 2로 x20의 값이 나오지만, ALUSrc = 1이라 ALU는 즉시값을 고르고, MemWrite = 0이라 데이터 메모리에도 쓰이지 않는다. 그래서 결과에 영향이 없다.

⑤ [11:7] = 01110_2 = 8 + 4 + 2 = 14 → 쓰기 레지스터 = 14(x14, 목적지).

⑥ ImmGen은 I형 즉시값 칸 [31:20] = 000000010100_2 = 16 + 4 = 20을 받는다. 맨 앞(부호 비트)이 0이라 왼쪽을 0으로 채워 부호 확장해도 20이다.

⑦ 데이터 메모리 주소 = ALU 결과 = 읽기 데이터 1(x10의 값) + ImmGen 출력 = 0x2000 + 20. 20 = 1 × 16 + 4 = 0x14이므로 0x2000 + 0x14 = 0x2014.

검산: 문제의 어셈블리 `lw x14, 20(x10)`과 rd = 14, rs1 = 10, 오프셋 20이 맞는다.

회독 점검

1. 그림 빈칸: (가) PC와 ImmGen 출력을 더하는 상자 (나) 결과가 0이면 1을 내보내는 출력을 가진 상자 (다) [6:0]을 받는 타원 (라) 12비트를 32비트로 늘리는 타원 (마) Branch와 Zero를 받는 게이트의 이름은?

   답 보기

   (가) 분기 가산기(adder) (나) ALU (다) 제어 유닛 (라) ImmGen (마) AND 게이트

   ① 분기 목표 = PC + (오프셋 << 1)은 ALU와 별도인 가산기가 계산한다. [Ch4 p.15]

   ② Zero(결과가 0이면 1)는 ALU 출력이다. 그림에서는 왼쪽 변이 V자로 파인 도형이다.

   ③ opcode [6:0]은 주 제어 유닛 입력이다(큰 타원). 같은 타원이라도 [31:0]을 받으면 ImmGen, ALUOp를 받으면 ALU 제어다.

   ④ 12비트 즉시값을 32비트로 부호 확장하는 것은 ImmGen이다.

   ⑤ PCSrc = Branch AND Zero라서 두 신호를 받는 게이트는 AND 게이트다. [Ch4 p.25]

---

2. `0x01DE73B3`이 실행될 때 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터 선의 번호와 명령어를 쓰시오.

   답 보기

   28, 29, 7. `and x7, x28, x29`

   ① 4비트씩: 0000 0001 1101 1110 0111 0011 1011 0011.

   ② 오른쪽 7비트 `0110011` → R형. 7 · 5 · 5 · 3 · 5 · 7로 끊으면 0000000 · 11101 · 11100 · 111 · 00111 · 0110011.

   ③ [24:20] = 11101_2 = 16 + 8 + 4 + 1 = 29, [19:15] = 11100_2 = 16 + 8 + 4 = 28, [11:7] = 00111_2 = 7.

   ④ 선에는 이 번호가 그대로 실린다: 읽기 레지스터 1 ← [19:15] = 28, 읽기 레지스터 2 ← [24:20] = 29, 쓰기 레지스터 ← [11:7] = 7.

   ⑤ funct7 0000000, funct3 111 → and(위 푸는 순서 2의 목록). 어셈블리 순서는 rd, rs1, rs2 → `and x7, x28, x29`.

   검산: R형은 RegWrite = 1이라 쓰기 레지스터 7이 실제로 쓰인다(x7 ← x28 AND x29).

---

3. `sw x8, 12(x9)` (기계어 `0x0084A623`)에서 읽기 레지스터 1·2, 쓰기 레지스터 선의 번호를 쓰고, 쓰기 레지스터 선의 값이 문제가 되지 않는 이유를 쓰시오.

   답 보기

   읽기 레지스터 1 = 9, 읽기 레지스터 2 = 8, 쓰기 레지스터 = 12. RegWrite = 0이라 쓰지 않는다.

   ① 4비트씩: 0000 0000 1000 0100 1010 0110 0010 0011. 오른쪽 7비트 `0100011` → sw(S형). S형 칸 imm[11:5] 7 · rs2 5 · rs1 5 · funct3 3 · imm[4:0] 5 · opcode 7로 끊으면 0000000 · 01000 · 01001 · 010 · 01100 · 0100011.

   ② [19:15] = 01001_2 = 9(베이스 x9), [24:20] = 01000_2 = 8(저장할 값 x8).

   ③ [11:7] = 01100_2 = 8 + 4 = 12는 rd가 아니라 imm[4:0]이다. 전선은 고정이라 쓰기 레지스터 선에 12가 실리지만, sw는 레지스터에 결과를 쓰지 않아 RegWrite = 0이므로 x12는 바뀌지 않는다.

   검산: 즉시값 = imm[11:5] · imm[4:0] = 0000000 · 01100 = 12로 문제의 오프셋 12와 맞는다.

---

4. 조합 소자(C)와 상태 소자(S)로 분류: PC, MUX, 레지스터 파일, ImmGen, 데이터 메모리, 분기 가산기.

   답 보기

   PC = S, MUX = C, 레지스터 파일 = S, ImmGen = C, 데이터 메모리 = S, 분기 가산기 = C.

   ① 기준은 저장 여부다. 값을 기억하면 상태 소자, 출력이 지금 입력만으로 정해지면 조합 소자. [Ch4 p.9]

   ② PC는 현재 명령어 주소를 담아 두는 레지스터, 레지스터 파일은 레지스터 32개, 데이터 메모리는 메모리라 모두 기억한다 → S.

   ③ MUX는 지금 들어온 입력 중 하나를 고를 뿐, ImmGen은 지금 명령어 비트를 늘릴 뿐, 분기 가산기는 지금 들어온 두 수를 더할 뿐이라 기억하는 것이 없다 → C.

---

5. 빈칸: 프로세서는 (　　)와 (　　)로 이루어진다. 범용 레지스터 32개를 모아 번호로 읽고 쓰는 하드웨어는 (　　)다.

답 보기

데이터패스, 제어. 레지스터 파일.

① 데이터패스 = 데이터를 담고 계산하는 부품들(메모리, 레지스터 파일, ALU, 가산기), 제어 = 그 부품에 0/1 신호를 보내 무엇을 할지 정하는 쪽. [Ch4 p.8, 11]

② 32개 범용 레지스터를 모아 번호(읽기 레지스터·쓰기 레지스터 입구)로 읽고 쓰는 구조가 레지스터 파일이다. [Ch4 p.12] (21-2 기출 유형)

6. O/X: 엣지 트리거 방식에서 레지스터 파일의 값은 클럭 사이클 내내 계속 바뀔 수 있다.

답 보기

X

① 레지스터 파일은 값을 기억하는 상태 소자다.

② 엣지 트리거에서 상태 소자의 값은 클럭 엣지에서만 갱신된다. 엣지 사이에 값이 움직이는 것은 조합 소자(ALU, MUX 등)다. [Ch4 p.10]

③ 그래서 레지스터 파일 값은 한 사이클 동안 고정이고, 새로 쓸 값은 다음 엣지에 한 번 들어간다.

더 자세히: 이론 노트 Ch4 · 1~4 / 실전 7

Ch4 · 2 슬라이드 p.18–28

제어 신호와 ALU 제어

시험에 나오는 모양: 명령어별 ALUSrc·Branch·MemtoReg 등 0/1/X 표 채우기, ALUOp와 funct로 ALU 제어 4비트 구하기. (기출: 21-2 ld, sd, beq, add의 0/1/X 표)

외울 것

신호	0일 때	1일 때
RegWrite	아무 일 없음	쓰기 레지스터(rd)에 쓰기 데이터 기록
ALUSrc	ALU 둘째 입력 = 읽기 데이터 2	ALU 둘째 입력 = 부호 확장 즉시값
PCSrc	PC ← PC + 4	PC ← 분기 목표 주소
MemRead	아무 일 없음	주소의 메모리 내용이 읽기 데이터로 나옴
MemWrite	아무 일 없음	주소의 메모리 내용이 쓰기 데이터로 바뀜
MemtoReg	레지스터 쓰기 데이터 = ALU 결과	레지스터 쓰기 데이터 = 메모리 읽기 데이터
Branch	분기 명령어 아님	분기 명령어(beq), Zero와 AND해서 PCSrc

[Ch4 p.24–25] 교재 기준

아래 표의 **ALUOp**(2비트, ALUOp1·ALUOp0) = 주 제어 유닛이 ALU 제어 유닛에 주는 힌트: 00 = 덧셈, 01 = 뺄셈, 10 = funct 칸을 보고 정함(맨 아래 표). [Ch4 p.18–20] **X** = don't care(0이든 1이든 결과가 같음, 표 아래 규칙).

명령어	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
R형	0	0	1	0	0	0	1	0
lw (= ld)	1	1	1	1	0	0	0	0
sw (= sd)	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

[Ch4 p.25] 교재 기준 ld(load doubleword)·sd(store doubleword)는 64비트판 이름이고 신호 값은 lw·sw와 같다. PDF 외 보충

- **X(don't care) 규칙**: sw·beq는 RegWrite = 0이라 레지스터에 쓰지 않는다 → MemtoReg가 무엇을 골라도 상관없다. 반대로 X로 둘 수 없는 것: RegWrite·MemWrite(잘못 1이면 레지스터·메모리 값이 망가짐), Branch(엉뚱한 분기), MemRead(불필요한 메모리 읽기를 막으려고 PDF 외 보충).

- **X가 되는 신호를 스스로 찾는 법**: X가 될 수 있는 것은 **MUX의 선택 신호**뿐이고, 그 MUX의 출력을 아무도 쓰지 않을 때만 X다. 이유: MUX는 둘 중 하나를 골라 넘길 뿐 아무것도 바꾸지 않으므로, 받는 쪽이 버리는 값이면 어느 쪽을 골라도 결과가 같다. 반면 RegWrite·MemWrite·MemRead·Branch는 레지스터·메모리·PC에 일을 시키는 스위치라 0이어야 할 때 1이 되면 실제로 무언가가 바뀐다. 네 명령어에 대입하면: MemtoReg MUX의 출력은 레지스터 쓰기 데이터로만 가므로 RegWrite = 0인 sw·beq에서 X. ALUSrc MUX의 출력은 ALU로 가는데 ALU 결과는 네 명령어 모두 쓴다(R형 결과, lw·sw 주소, beq 비교) → ALUSrc는 언제나 0 또는 1로 정해진다. PDF 외 보충
- **Zero는 언제 1인가**: ALU는 모든 명령어에서 계산하고 결과가 0이면 언제나 Zero = 1을 낸다. add의 결과가 우연히 0이어도 Zero = 1이다. 그래서 Zero만으로 분기하면 안 되고, beq일 때만 1인 Branch와 AND한다. beq는 ALU가 rs1 − rs2를 계산하므로 Zero = 1 ⇔ 두 레지스터 값이 같음이다. [Ch4 p.12, 15]
- **PCSrc = Branch AND Zero**: opcode만으로 정할 수 없는 유일한 신호. "beq이고 ALU Zero가 1"일 때 1. 다른 신호는 명령어 종류만 알면 정해지지만, Zero는 실행 중 레지스터에 든 값을 빼 봐야 알 수 있어서 opcode만 보는 제어 유닛이 미리 정할 수 없다. [Ch4 p.25]
- 주 제어 유닛 출력(그림 위부터): Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite. PCSrc는 바깥 AND 게이트가 만든다. [Ch4 p.26–28]

명령어	ALUOp	funct7	funct3	ALU 동작	ALU 제어
lw / sw	00	X	X	add (주소 계산)	`0010`
beq	01	X	X	subtract (같은지 검사)	`0110`
add	10	0000000	000	add	`0010`
sub	10	0100000	000	subtract	`0110`
and	10	0000000	111	AND	`0000`
or	10	0000000	110	OR	`0001`

[Ch4 p.18, 20] 다단 디코딩: 주 제어 → ALUOp → ALU 제어. 주 제어 유닛이 작아지고 지연도 줄 수 있다. [Ch4 p.19] 입력 2 + 7 + 3 = 12비트라 전체 2^{12} = 4096 조합, don't care로 압축. [Ch4 p.20]

ALU 제어 표 읽는 법: ① 명령어의 ALUOp를 위 신호표에서 찾는다. ② ALUOp가 00이나 01이면 funct 열이 X라 funct는 보지 않고 그 줄의 ALU 제어가 답이다(lw·sw는 늘 덧셈, beq는 늘 뺄셈이라 funct를 볼 필요가 없음). ③ ALUOp가 10(R형)일 때만 funct7과 funct3가 둘 다 맞는 줄을 찾는다. 예 sub: ALUOp 10 → funct7 0100000·funct3 000 줄 → 0110. [Ch4 p.18, 20]

이 표의 X는 **입력** 열의 X다. "출력이 그 입력 값에 달려 있지 않다"(ALUOp가 00이면 funct가 무엇이든 0010)는 뜻이다. 위 신호표의 X는 **출력**(제어 신호) 값이 0이든 1이든 실행 결과가 같다는 뜻이다. 둘 다 "상관없음"이지만

가리키는 쪽이 다르다. [Ch4 p.20] ALU 제어 4비트 값(0000 AND, 0001 OR, 0010 add, 0110 sub)은 정해진 약속이라 이유 없이 외운다. [Ch4 p.18]

ALUOp 두 비트는 ALUOp1(왼쪽)·ALUOp0(오른쪽)이다. "ALUOp = 10"은 ALUOp1 = 1, ALUOp0 = 0이라는 뜻이다.

푸는 순서 (한 명령어의 신호 줄)

1. 명령어 종류를 정한다: R형 / lw(ld) / sw(sd) / beq. 각자 하는 일을 말로 쓴다: R형 rd ← rs1 연산 rs2, lw rd ← 메모리[rs1 + imm], sw 메모리[rs1 + imm] ← rs2, beq rs1 = rs2면 분기.
2. 레지스터에 쓰나(말로 쓴 식의 왼쪽이 rd인가)? 쓰면 RegWrite = 1. 쓰지 않으면 RegWrite = 0, 그러면 쓰기 데이터를 고르는 MemtoReg = X.
3. 쓴다면 그 값이 메모리 값인가? 메모리면 MemtoReg = 1(lw), ALU 결과면 0(R형).
4. 메모리를 읽나(MemRead, lw만), 쓰나(MemWrite, sw만)? 아니면 0(X 아님).
5. ALU 둘째 입력이 즉시값인가? 주소 계산(lw·sw)이면 ALUSrc = 1, 레지스터끼리(R형·beq)면 0.
6. 분기인가? beq만 Branch = 1, 나머지는 0(X 아님). ALUOp는 ALU가 할 일로 정한다: 주소 덧셈(lw·sw) 00, 비교 뺄셈(beq) 01, funct로 정함(R형) 10.

함정

- beq에도 오프셋이 있지만 ALUSrc = 0이다. 오프셋은 분기 가산기로 가고 ALU는 두 레지스터를 뺀다.
- X는 0과 같은 뜻이 아니다. 0이든 1이든 결과가 같다는 뜻이다.
- ALUOp(2비트, 주 제어 출력)와 ALU 제어(4비트, ALU 제어 유닛 출력)를 섞지 말 것.

예제 (시험 수준)

표를 채우시오(열 순서는 제어 유닛 출력 순서). 명령어: `sub`, `sd`, `ld`, `beq`. 열: Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite.

풀이 보기

sub: 0 0 0 10 0 0 1 / sd: 0 0 X 00 1 1 0 / ld: 0 1 1 00 0 1 1 / beq: 1 0 X 01 0 0 0

① sub(R형): 결과를 rd에 쓴다 → RegWrite 1, 값은 ALU 결과 → MemtoReg 0. 메모리 안 씀 → MemRead 0, MemWrite 0. 두 레지스터끼리 → ALUSrc 0. 분기 아님 → Branch 0. R형 → ALUOp 10.

② sd(= sw): 레지스터에 쓰지 않음 → RegWrite 0, MemtoReg X. 메모리에 씀 → MemWrite 1, MemRead 0. 주소 = 베이스 + 오프셋 → ALUSrc 1, ALUOp 00(덧셈). Branch 0.

③ ld(= lw): 메모리 값을 rd에 → RegWrite 1, MemtoReg 1, MemRead 1, MemWrite 0. 주소 계산 → ALUSrc 1, ALUOp 00. Branch 0.

④ beq: 레지스터에 안 씀 → RegWrite 0, MemtoReg X. 메모리 안 씀 → 0, 0. 두 레지스터를 빼서 비교 → ALUSrc 0, ALUOp 01. Branch 1.

검산: 위 외울 것 표의 열 순서(ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp)로 바꿔 읽으면 lw 1 1 1 1 0 0 00, sw 1 X 0 0 1 0 00, beq 0 X 0 0 0 1 01, R형 0 0 1 0 0 0 10과 같다. [Ch4 p.25]

회독 점검

1. Branch = 0, MemWrite = 0, RegWrite = 1, ALUSrc = 1인 명령어는 R형, lw, sw, beq 중 무엇인가? 이 명령어의 MemtoReg는?

   답 보기

   lw, MemtoReg = 1

   ① RegWrite = 1 → R형 또는 lw. ② ALUSrc = 1(즉시값) → R형은 0이므로 lw. ③ lw는 메모리 값을 레지스터에 쓰므로 MemtoReg = 1.

2. O/X: sw의 MemtoReg를 0으로 정해도 실행 결과는 같지만, RegWrite를 X로 두면 안 된다.

답 보기

O

① sw는 레지스터에 쓰지 않으므로 MemtoReg MUX의 선택은 결과에 영향이 없다(X라서 0이어도 됨).

② RegWrite가 1이 되면 [11:7](imm[4:0])을 번호로 하는 레지스터가 덮어써진다. 그래서 반드시 0이다.

---

3. `ld`와 `and`의 ALUOp와 ALU 제어 4비트를 각각 쓰시오.

답 보기

ld: ALUOp 00, ALU 제어 0010 / and: ALUOp 10, ALU 제어 0000

① ld는 lw와 같다. 주소 계산은 늘 덧셈 → ALUOp 00 → funct를 보지 않고 0010(add).

② and는 R형 → ALUOp 10 → funct7 0000000, funct3 111 줄 → 0000(AND). [Ch4 p.20]

---

4. `add x5, x6, x7`에서 x6 = 4, x7 = −4다. ALU의 Zero와 PCSrc는?

답 보기

Zero = 1, PCSrc = 0

① ALU 결과 4 + (−4) = 0 → Zero = 1.

② add는 분기가 아니라 Branch = 0. PCSrc = Branch AND Zero = 0 AND 1 = 0 → 다음 PC = PC + 4.

Zero만 보고 분기한다고 답하면 틀린다. 그래서 Branch 신호가 따로 있다.

---

5. 빈칸: PCSrc는 제어 신호 중 유일하게 ( )만으로 정할 수 없다. 명령어가 ( )이고 ALU의 ( ) 출력이 1일 때 1이 된다.

답 보기

opcode(명령어 필드), beq, Zero

Zero는 실행 중 뺄셈 결과로 정해지므로 opcode만 봐서는 알 수 없다. [Ch4 p.25]

더 자세히: 이론 노트 Ch4 · 5, 6, 7 / 실전 7

Ch4 · 3 슬라이드 p.2–10, 16–29

단일 사이클의 한계와 외울 문장

시험에 나오는 모양: "단일 사이클을 쓰지 않는 이유" 서술, 가장 긴 경로의 장치 순서, 클럭 주기·CPU 시간 계산, T/F 문장.

외울 것

- 성능 3요소(IC, CPI, 클럭 주기)와 결정 주체는 Ch1 · 2. 이 장은 CPI와 주기를 정하는 CPU 하드웨어를 만든다. [Ch4 p.2]
- 구현은 두 가지: 단순한(단일 사이클) 버전, 파이프라인 버전. [Ch4 p.3]
- add x5, x6, x7: 인출 → x6·x7 읽기 → 덧셈 → x5에 쓰기. [Ch4 p.5]
- lw x5, 40(x6): 인출 → x6 읽기 → x6 + 40 주소 계산(덧셈) → 메모리 읽어 x5에 쓰기. [Ch4 p.6]
- 선을 그냥 합칠 수 없다 → MUX. 데이터패스를 조종해야 한다 → 제어 유닛. [Ch4 p.7]
- 단일 사이클 = 모든 명령어를 한 클럭에(메모리를 둘로 나눈 이유는 Ch4 · 1). [Ch4 p.16]
- SB·UJ형은 즉시값 비트 배치를 섞어 ImmGen의 MUX를 줄였고, 대가는 어셈블러가 진다. [Ch4 p.22–23]
- 단일 사이클은 동작은 맞지만 너무 비효율적이다. [Ch4 p.29]
- 모든 명령어가 같은 길이의 클럭 사이클을 가져야 하고, 가장 긴 경로가 클럭 주기를 정한다. [Ch4 p.29]
- 가장 긴 경로 = load: **명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일**(5개 직렬). [Ch4 p.29]
- 명령어마다 주기를 다르게 하는 것은 불가능(not feasible) → "자주 생기는 경우를 빠르게" 위반 → 파이프라이닝으로 개선. [Ch4 p.29]

```
단일 사이클 CPU 시간 = IC × 1 × (가장 긴 경로 지연)
```

CPI는 1로 최소지만 주기가 lw에 맞춰 길어진다. 짧은 명령어도 그만큼 기다린다. [Ch1 p.28] [Ch4 p.29]

경로 지연을 더하는 이유: 한 명령어 안에서 장치들은 앞 장치의 출력을 받아야 일을 시작할 수 있다(직렬). ALU는 레지스터 파일이 값을 내놓기 전에는 계산할 수 없고, 레지스터 파일은 명령어 메모리가 명령어(번호가 든 비트)를 내놓기 전에는 무엇을 읽을지 모른다. 그래서 한 명령어에 걸리는 시간 = 지나는 장치 지연의 합이다. 반대로 서로를 기다리지 않는 장치(PC + 4 가산기, 분기 가산기는 ALU와 동시에 일함)는 더하지 않는다. 엣지 트리거에서 이 합이 다음 엣지 전에 끝나야 하므로, 명령어 중 가장 긴 합이 클럭 주기가 된다. [Ch4 p.10, 29]

명령어	지나는 장치(차례대로)	빠지는 장치와 이유
lw	명령어 메모리 → 레지스터 파일(읽기) → ALU → 데이터 메모리 → 레지스터 파일(쓰기)	없음(5개, 가장 긺)
sw	명령어 메모리 → 레지스터 파일(읽기) → ALU → 데이터 메모리	레지스터 쓰기(RegWrite = 0)
R형	명령어 메모리 → 레지스터 파일(읽기) → ALU → 레지스터 파일(쓰기)	데이터 메모리(MemRead·MemWrite = 0)
beq	명령어 메모리 → 레지스터 파일(읽기) → ALU(뺄셈, Zero)	데이터 메모리, 레지스터 쓰기

표는 앞 단원의 제어 신호(0이면 그 장치를 안 씀)로 만든 것이다. 시험 문제가 "나머지 무시"라고 하면 MUX·가산기·제어 유닛 지연은 0으로 본다. [Ch4 p.29] PDF 외 보충

함정

- "CPI가 1이니 단일 사이클이 가장 빠르다"는 X. 클럭 주기가 가장 긴 명령어에 맞춰진다.
- 가장 긴 경로는 sw가 아니라 lw. sw는 레지스터 쓰기가 없다.
- 위반하는 원칙은 8대 아이디어 중 "자주 생기는 경우를 빠르게(Make the common case fast)".

회독 점검

1. 명령어 메모리 300 ps, 레지스터 파일 읽기·쓰기 각 120 ps, ALU 180 ps, 데이터 메모리 300 ps다(나머지 무시). 단일 사이클 클럭 주기와 beq에 실제로 필요한 시간은? 명령어 10^6개를 실행하면 CPU 시간은?

   답 보기

   주기 1020 ps, beq 600 ps, CPU 시간 1.02 ms

   ① 주기 = 가장 긴 lw 경로(명령어 메모리 → 레지스터 파일 읽기 → ALU → 데이터 메모리 → 레지스터 파일 쓰기)의 지연 합. 장치들이 앞 장치를 기다리는 직렬이라 더한다: 300 + 120 = 420, + 180 = 600, + 300 = 900, + 120 = 1020 ps.

   ② beq는 인출, 레지스터 읽기, ALU만 쓴다(데이터 메모리·레지스터 쓰기 없음): 300 + 120 + 180 = 600 ps. 그래도 단일 사이클에서는 모든 명령어가 같은 주기라 1020 ps를 쓴다.

   ③ CPU 시간 = IC × CPI × 주기 = $10^6 \times 1 \times 1020$ ps = $1020 \times 10^6 \times 10^{-12}$ s = 1.02×10^{-3} s = 1.02 ms.

2. O/X: 단일 사이클 구현에서 클럭 주기는 가장 자주 실행되는 명령어의 경로로 정한다.

   답 보기

   X

   가장 긴 경로(lw)로 정한다. 자주 쓰는 짧은 명령어도 그 시간을 기다리므로 "자주 생기는 경우를 빠르게"를 어긴다. [Ch4 p.29]

3. 빈칸: 단일 사이클에서 가장 긴 경로는 ( ) 명령어이고, 명령어 메모리 → ( ) → ( ) → ( ) → 레지스터 파일 순으로 지난다. sw의 경로가 이보다 짧은 이유는?

   답 보기

   lw. 레지스터 파일, ALU, 데이터 메모리. sw는 마지막 레지스터 쓰기가 없다.

   ① lw는 인출(명령어 메모리) → 베이스 레지스터 읽기(레지스터 파일) → 주소 덧셈(ALU) → 메모리 읽기(데이터 메모리) → 읽은 값을 rd에 쓰기(레지스터 파일)로 다섯 장치를 차례로 지난다.

   ② sw는 데이터 메모리에 쓰고 끝난다(RegWrite = 0). 그래서 마지막 레지스터 파일 단계가 빠진다.

   [Ch4 p.29]

4. `add x9, x20, x21`의 실행 4단계를 쓰고, `lw x9, 8(x20)`과 달라지는 곳 두 군데를 쓰시오.

   답 보기

   인출 → x20·x21 읽기 → x20 + x21 덧셈 → 합을 x9에 쓰기. 다른 곳: ALU 둘째 입력(레지스터 x21 / 즉시값 8), x9에 쓰는 값(ALU 결과 / 메모리에서 읽은 값).

   ① 두 명령어 모두 인출과 레지스터 읽기(공통 두 단계) 뒤에 ALU가 덧셈을 한다. lw의 덧셈은 주소 계산이다.

   ② 그래서 ALU 둘째 입력 앞(ALUSrc)과 레지스터 쓰기 데이터 앞(MemtoReg)에 MUX가 필요하다.

   [Ch4 p.5–7]

더 자세히: 이론 노트 Ch4 · 1, 2, 6, 8 / 실전 8

마무리 기초~Ch4 전 범위

시험 전날 한 장

시험 전날과 당일 아침에 이 화면만 훑는다. 막히는 줄이 있으면 그 단원으로 돌아간다. 아래 점검 10문항은 종이에 풀고 답을 연다.

기초

- 16진 한 자리 = 2진 4비트(0xB7 = 1011 0111_2 = 11 × 16 + 7 = 183).
- 2의 보수 부정 = ~x + 1, n비트 범위 -2^{n-1} ~ $2^{n-1} - 1$(32비트 -2^{31} ~ $2^{31} - 1$). [Ch2 p.19–20]
- 부호 확장 = 부호 비트를 왼쪽에 복제(1111 1110 → 1111…1110, 여전히 −2). [Ch2 p.20]
- 왼쪽 시프트 k = × 2^k, sra = 부호 비트로 채우며 오른쪽 시프트. [Ch2 p.25]
- 1 ps = 10^{-12} s, 4 GHz ↔ 250 ps(역수). KB = 10^3, KiB = 2^{10}.

Ch1

공식·목록	근거
CPU 시간 = IC × CPI × 클럭 주기 = IC × CPI / 클럭 속도	[Ch1 p.28]
성능 = 1 / 실행 시간. "X가 n배 빠름" ⇔ $시간_Y$ / $시간_X$ = n	[Ch1 p.24]
MIPS = 클럭 속도 / (CPI × 10^6). 한계: ISA 차이, 명령어 복잡도 반영 NOT, 프로그램마다 CPI 다름	[Ch1 p.34]
암달: $T_{개선}$ = $T_{영향}$ / n + $T_{비영향}$ (80 s 중 2.5배 → n = 4)	[Ch1 p.33]
전력 ∝ 용량성 부하 × 전압^2 × 주파수. 전력 장벽 → 멀티코어	[Ch1 p.30–32]
8대 아이디어: 무어의 법칙, 추상화, 자주 생기는 경우를 빠르게, 병렬성, 파이프라이닝, 예측, 메모리 계층, 중복을 통한 신뢰성	[Ch1 p.12]
5대 구성 요소: 입력, 출력, 메모리, 데이터패스, 제어(프로세서 = 데이터패스 + 제어)	[Ch1 p.15]

Ch2

형식	칸(비트 수, 31 → 0)	예
R	funct7 7 · rs2 5 · rs1 5 · funct3 3 · rd 5 · opcode 7	add, sub, and, or
I	imm[11:0] 12 · rs1 · funct3 · rd · opcode	lw, addi, jalr
S	imm[11:5] 7 · rs2 · rs1 · funct3 · imm[4:0] 5 · opcode	sw
SB	imm[12|10:5] · rs2 · rs1 · funct3 · imm[4:1|11] · opcode	beq, bne
U / UJ	imm 20 · rd · opcode	lui / jal

- x0 = 항상 0, x1 = 복귀 주소, x2 = sp, x10~x17 = 인자(x10·x11 반환값). [Ch2 p.16, 34, 37]
- word 배열 A[i]의 오프셋 = 4 × i, 리틀 엔디언 = 최하위 바이트가 가장 낮은 주소. [Ch2 p.12–13]
- 설계 원칙 3: 단순성은 규칙성을 선호 / 작을수록 빠르다 / 좋은 설계는 좋은 타협. [Ch2 p.9–10, 23]
- 주소 지정 4: 즉시값, 레지스터, 베이스(lw·sw·jalr), PC 상대(beq·bne·jal). [Ch2 p.49]
- 분기 목표 = PC + 오프셋 필드 × 2, 범위 −4096 ~ 4094바이트. 스택은 아래로 자라고 레지스터 하나당 sp 4. [Ch2 p.37–38, 47]

Ch3

- 오버플로: + + + = −, − + − = +, + − − = −, − − + = +. 부호 다른 덧셈·부호 같은 뺄셈은 없음. [Ch3 p.4–5]
- mul = 하위 32비트, mulh(부호 × 부호)·mulhu(무 × 무)·mulhsu(부호 × 무) = 상위 32비트. [Ch3 p.13]
- 부호 있는 나눗셈: 부호가 다르면 몫 음수, 나머지 부호 = 피제수 부호(−7 ÷ 2 → 몫 −3, 나머지 −1). [Ch3 p.19]

	부호	지수	가수(F)	bias
단정도 32	1	8	23	127
배정도 64	1	11	52	1023
16비트 PDF 외 보충	1	5	10	15

- 값 = $(-1)^S \times (1 + F) \times 2^{(\text{지수 칸} - \text{bias})}$. −0.75 = 0xBF400000. [Ch3 p.28, 31]
- FP 덧셈 4단계: 소수점 맞추기(작은 지수 쪽 정렬) → 유효숫자 더하기 → 정규화·오버/언더플로 검사 → 반올림. 곱셈 5단계: 지수 더하기(bias 한 번 빼기) → 유효숫자 곱하기 → 정규화·검사 → 반올림 → 부호 결정. [Ch3 p.32, 35]

Ch4

명령어	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R형	0	0	1	0	0	0	10
lw	1	1	1	1	0	0	00
sw	1	X	0	0	1	0	00
beq	0	X	0	0	0	1	01

- ALU 제어: 0000 AND, 0001 OR, 0010 add, 0110 sub. PCSrc = Branch AND Zero. [Ch4 p.18, 25]
  교재 기준
- rs1 [19:15], rs2 [24:20], rd [11:7], opcode [6:0]. 메모리 둘 = 한 사이클에 자원 한 번. [Ch4 p.16, 21]
- 최장 경로 lw: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일. [Ch4 p.29]

명령어 모음

이름 읽는 법: 끝의 **i** = immediate(세 번째 값이 명령어에 적힌 상수), 끝의 **u** = unsigned(무부호), **l/s**로 시작 = load(메모리에서 읽기)/store(메모리에 쓰기), **w/d** = word 4바이트/doubleword 8바이트.

분류	명령어	이름 풀이	하는 일	형식
산술 [Ch2 p.9–10, 16]	`add rd, rs1, rs2`	add	rd = rs1 + rs2	R
	`sub rd, rs1, rs2`	subtract	rd = rs1 − rs2	R
	`addi rd, rs1, imm`	add immediate	rd = rs1 + 상수(subi는 없음, 음수 상수로 대신)	I
메모리 [Ch2 p.11–15]	`lw rd, imm(rs1)`	load word	rd = 메모리[rs1 + imm]의 4바이트	I
	`sw rs2, imm(rs1)`	store word	메모리[rs1 + imm] = rs2의 4바이트	S
	`ld`, `sd`	load / store doubleword	8바이트 읽기·쓰기. 21-2 기출, 제어 신호는 lw·sw와 같음 PDF 외 보충	I / S
논리 [Ch2 p.26–27]	`and`, `andi`	and (immediate)	비트별 AND, 일부 비트만 남기기(마스크)	R / I
	`or`, `ori`	or (immediate)	비트별 OR, 일부 비트를 1로	R / I
	`xor`, `xori`	exclusive or (immediate)	비트별 XOR, 다르면 1. 111… 1과 XOR = NOT	R / I
시프트 [Ch2 p.25]	`sll`, `slli`	shift left logical	왼쪽으로, 빈칸 0. k칸 = × 2^k	R / I
	`srl`, `srli`	shift right logical	오른쪽으로, 빈칸 0	R / I
	`sra`, `srai`	shift right arithmetic	오른쪽으로, 빈칸 부호 비트 (1100 0011 → 1110 0001)	R / I
조건 분기 [Ch2 p.28–31]	`beq rs1, rs2, L`	branch if equal	같으면 L로	SB
	`bne`	branch if not equal	다르면 L로	SB
	`blt`	branch if less than	rs1 < rs2(부호 있게)면 L로	SB
	`bge`	branch if greater or equal	rs1 ≥ rs2(부호 있게)면 L로	SB

분류	명령어	이름 풀이	하는 일	형식
	`bltu`, `bgeu`	위 둘의 unsigned판	무부호로 비교	SB
점프 [Ch2 p.35]	`jal x1, L`	jump and link	PC + 4를 x1에 저장하고 L로 (함수 호출)	UJ
	`jalr x0, 0(x1)`	jump and link register	x1 + 0 주소로(함수 복귀), 주소 지정은 베이스 방식	I
큰 상수 [Ch2 p.46]	`lui rd, imm`	load upper immediate	20비트 상수를 31~12번 비트에, 아래 12비트는 0. 이어서 addi	U
곱셈·나눗셈 [Ch3 p.13, 20]	`mul`	multiply	곱의 하위 32비트	R
	`mulh`, `mulhu`, `mulhsu`	multiply high (unsigned / signed-unsigned)	곱의 상위 32비트(부호×부호 / 무×무 / 부호×무)	R
	`div`, `divu`, `rem`, `remu`	divide / remainder (unsigned)	몫 / 나머지	R
부동소수점 [Ch3 p.37]	`fadd`, `fsub`, `fmul`, `fdiv`, `fsqrt` (`.s` / `.d`)	FP 사칙연산·제곱근	.s = 단정도, .d = 배정도, 레지스터 f0~f31(f0은 0 고정 아님)	
	`feq`, `flt`, `fle`	FP equal / less than / less or equal	결과 0 또는 1을 **정수 레지스터**에, 이어서 beq·bne로 분기	
	`flw`, `fsw` / `fld`, `fsd`	FP load / store word·doubleword	단정도 / 배정도 값 메모리 읽기·쓰기	

Ch4 단일 사이클이 구현하는 명령어는 **lw, sw, add, sub, and, or, beq** 일곱 개뿐이다. 제어 신호 표 문제는 이 일곱 개로 나온다. [Ch4 p.3]

외울 것은 뿌리 10개(add/sub, lw/sw, and/or/xor, sll/srl/sra, beq/bne/blt/bge, jal/jalr, lui) + i·u 규칙. 곱셈은 상위·하위 구분, FP는 위 두 줄(정수 레지스터, f0)만.

회독 점검

1. −20을 8비트 2의 보수로 쓰고 16진으로 나타내시오.

   답 보기

   $1110\ 1100_2$ = 0xEC

   ① 20 = 16 + 4 → $0001\ 0100_2$.

   ② 음수는 ~x + 1: 비트를 뒤집으면 $1110\ 1011_2$, 1을 더하면 $1110\ 1100_2$.

   ③ 4비트씩 16진: 1110 = E, 1100 = C → 0xEC.

   검산: 무부호로 읽으면 0xEC = 14 × 16 + 12 = 236, 236 − 256 = −20(8비트 2의 보수는 256을 빼면 음수 값).

---

2. 명령어 4×10^9개, CPI 1.25, 클럭 2.5 GHz다. CPU 시간과 MIPS는?

   답 보기

   2 s, 2000 MIPS

   ① CPU 시간 = IC × CPI / 클럭 속도 = $4 \times 10^9 \times 1.25 / (2.5 \times 10^9) = 5 \times 10^9 / (2.5 \times 10^9)$ = 2 s.

   ② MIPS = 클럭 속도 / (CPI × 10^6) = $2.5 \times 10^9 / (1.25 \times 10^6) = 2 \times 10^3$ = 2000.

   검산: MIPS = IC / (시간 × 10^6) = $4 \times 10^9 / (2 \times 10^6)$ = 2000. [Ch1 p.28, 34]

---

3. 120 s 걸리는 프로그램 중 메모리 접근이 90 s다. 전체를 3배 빠르게 하려면 메모리 접근을 몇 배 빠르게 해야 하나? 메모리 외 부분만 개선하면 최대 몇 배인가?

   답 보기

   9배. 메모리 외만 개선하면 최대 약 1.33배(4/3배).

   ① 목표 시간 = 120 / 3 = 40 s.

   ② 암달: 40 = 90 / n + 30(개선 안 되는 나머지 30 s). 90 / n = 10 → n = 9.

   ③ 메모리 외 30 s를 0으로 줄여도 90 s가 남는다 → 120 / 90 = 4/3 ≈ 1.33배. [Ch1 p.33]

---

4. O/X: ISA가 다른 두 컴퓨터는 MIPS 값만으로 성능을 비교할 수 있다.

답 보기

X

ISA가 다르면 같은 프로그램의 명령어 수가 다르고, 명령어 복잡도 차이도 MIPS에 반영되지 않는다. [Ch1 p.34]

---

5. `x5 = 0xA1B2C3D4`를 `sw x5, 0(x6)`로 저장했다(x6 = 0x100). 0x100~0x103 각 주소의 바이트는?

답 보기

0x100: D4, 0x101: C3, 0x102: B2, 0x103: A1

① 바이트로 나누면 A1 B2 C3 D4, 최하위 바이트는 맨 오른쪽 D4.

② RISC-V는 리틀 엔디언이라 최하위 바이트를 가장 낮은 주소에 둔다. 그다음 바이트부터 주소가 1씩 커진다. [Ch2 p.13]

---

6. 0x200에 있는 `beq`가 0x1E8로 분기한다. 바이트 오프셋과 명령어에 저장되는 오프셋 필드 값(10진)은?

답 보기

바이트 오프셋 −24, 오프셋 필드 −12

① 목표 − PC = 0x1E8 − 0x200. 0x200 = 512, 0x1E8 = 1 × 256 + 14 × 16 + 8 = 488. 488 − 512 = −24.

② 분기 목표 = PC + (오프셋 필드 × 2)이므로 필드 = −24 / 2 = −12. 0번 비트는 항상 0이라 저장하지 않는다. [Ch2 p.47] [Ch4 p.14]

---

7. 프로시저가 레지스터 5개를 스택에 저장한다. 맨 처음 sp를 조정하는 명령어를 쓰시오.

답 보기

`addi sp, sp, -20`

① 레지스터 하나는 4바이트 → 5 × 4 = 20바이트.

② 스택은 높은 주소에서 낮은 주소로 자라므로 공간을 확보할 때 sp에서 뺀다. [Ch2 p.37–38]

---

8. O/X: 부호가 다른 두 수의 뺄셈에서는 오버플로가 생기지 않는다.

   답 보기

   X

   오버플로가 없는 경우는 부호가 다른 수의 **덧셈**과 부호가 같은 수의 뺄셈이다. 부호가 다른 뺄셈은 + − − = −, − − + = + 처럼 생길 수 있다. [Ch3 p.4–5]

---

9. −5.5를 IEEE 754 단정도 16진수로, 그리고 16비트(1-5-10, bias 15) 16진수로 쓰시오.

   답 보기

   단정도 0xC0B00000, 16비트 0xC580

   ① $5 = 101_2$, $0.5 = 0.1_2$ → $5.5 = 101.1_2 = 1.011_2 \times 2^2$. 음수라 S = 1.

   ② 단정도 지수 칸 = 2 + 127 = 129 = $1000\ 0001_2$. 가수 = 011 뒤에 0(앞의 1은 숨은 비트).

   ③ 1 | 10000001 | 0110…0 → 4비트씩 1100 0000 1011 0000 0000 … → 0xC0B00000.

   ④ 16비트 지수 칸 = 2 + 15 = 17 = 10001_2. 1 | 10001 | 0110000000 → 1100 0101 1000 0000 → 0xC580.

---

10. 빈칸: beq의 ALUOp는 ( )이고 ALU 제어 출력은 ( )이다. 다음 PC를 고르는 신호는 PCSrc = ( ) AND ( )이다.

    답 보기

    01, 0110(뺄셈), Branch, Zero

    beq는 두 레지스터를 빼서 0인지 본다. 차가 0이면 Zero = 1, Branch = 1과 AND해서 분기 목표를 고른다. [Ch4 p.18, 25]

더 자세히: 이론 노트 한눈에 정리 표 1~9 / 실전 8

틀린 문제 모아 보기

오답 노트

회독 점검 문항 아래의 **[틀림]**을 누른 문제가 단원별로 여기에 모입니다. 다시 풀어 맞히면 **[다시 풀어 맞음]**을 눌러 목록에서 빼세요. 다음 회독을 시작하기 전에 여기부터 다시 풉니다.